PROMPT 1:

ETAPA 1 — PLANIFICACIÓN Y LEVANTAMIENTO DE REQUISITOS

Sistema web de gestión de inventario para una ferretería

1. Rol y enfoque profesional

Actúa como un **analista de sistemas e ingeniero de requisitos de software**, con experiencia en análisis de negocios, desarrollo de aplicaciones web y proyectos construidos con Java, Spring Boot, React y MySQL.

Trabaja con un enfoque profesional, sistemático y pragmático. El objetivo es definir correctamente el problema, el alcance del producto, los usuarios, las reglas de negocio y los requisitos antes de iniciar el diseño y la implementación.

Toma como referencias:

- **ISO/IEC/IEEE 29148:2018**, para orientar la ingeniería, especificación y documentación de requisitos.
- **ISO/IEC 25010:2023**, para identificar y especificar requisitos de calidad del producto de software.

Utiliza estas referencias de manera práctica y proporcional al tamaño del proyecto. No afirmes que el proyecto está certificado ni que cumple integralmente una norma por utilizar sus plantillas o recomendaciones.

No te limites a generar documentos genéricos. Cada decisión y requisito debe relacionarse con las características concretas del negocio descrito en este prompt.

2. Contexto del negocio

Se desarrollará un sistema web para administrar las operaciones principales de una ferretería.

Actualmente, el propósito del sistema es centralizar la información de productos, existencias, compras, ventas, clientes, proveedores, devoluciones y ajustes de inventario.

El sistema operará en una sola sede y manejará un único inventario. El proyecto está orientado a una demostración académica pequeña, pero debe desarrollarse con buenas prácticas de ingeniería de software y una estructura que permita comprenderlo, probarlo, desplegarlo y mantenerlo.

El sistema debe ayudar a mantener información consistente, reducir errores operativos y conservar la trazabilidad de las operaciones.

3. Tecnologías de referencia

El proyecto utilizará las siguientes tecnologías:

Backend

- Java 21.
- Spring Boot 3.
- Spring MVC para la API REST.
- Spring Data JPA y Hibernate para la persistencia.
- Spring Security para autenticación y autorización.
- Maven para la gestión de dependencias y construcción.

Base de datos

- MySQL como sistema gestor de base de datos relacional.

Frontend

- React.
- Material UI para los componentes de interfaz.
- Formik y Yup como herramientas de referencia para formularios y validaciones, cuando resulten apropiadas.

Comunicación y documentación

- API REST.
- Intercambio de datos mediante JSON.
- OpenAPI/Swagger para la documentación de la API en la etapa correspondiente.

Pruebas

- JUnit y Mockito para pruebas del backend.
- Herramientas de pruebas de integración de Spring.
- Playwright para pruebas del frontend.

Estas tecnologías constituyen las decisiones de referencia del proyecto. No propongas sustituirlas sin identificar primero un problema concreto que justifique el cambio.

En esta etapa no debes definir todavía la arquitectura interna definitiva, las entidades JPA, las tablas de MySQL, los endpoints detallados ni la estructura final de paquetes. Esos aspectos corresponden a etapas posteriores.

4. Usuarios y actores del sistema

El sistema tendrá dos roles de autorización:

Administrador

- Gestiona y supervisa las operaciones del sistema.
- Es el único autorizado para ejecutar y autorizar ajustes de inventario.
- Puede consultar la información y la auditoría que correspondan a sus permisos.

Usuario operativo

- Realiza las operaciones comerciales y de inventario para las que tenga autorización.
- Puede registrar compras, ventas, clientes, proveedores y devoluciones según los permisos definidos.
- Puede modificar precios de venta cuando tenga autorización para gestionar inventario.

Debes elaborar una matriz de permisos que detalle las operaciones permitidas para cada rol.

La descripción anterior establece responsabilidades generales, pero no implica que todos los permisos estén completamente definidos. Identifica los que deban validarse con el responsable del negocio.

La autorización debe contemplarse tanto en el backend como en la interfaz. Ocultar una opción en el frontend no constituye por sí solo un control de seguridad.

Además de los dos roles, identifica los actores externos que realmente intervengan en los procesos, como clientes y proveedores, distinguiendo entre actores que utilizan directamente el sistema y personas o entidades cuyos datos son gestionados por los usuarios.

5. Alcance funcional confirmado

El sistema debe contemplar los siguientes módulos:

1. **Productos:** registro, consulta, actualización y desactivación de productos.
2. **Inventario:** consulta de existencias, registro de movimientos y control de cantidades disponibles.
3. **Compras:** registro de compras con múltiples productos y recepción completa en una sola operación.
4. **Costos:** cálculo del costo promedio ponderado del inventario.
5. **Ventas:** registro de ventas vinculadas obligatoriamente a un cliente registrado.
6. **Clientes y proveedores:** registro y administración de terceros. Una misma persona o empresa puede ser cliente y proveedor.
7. **Pagos:** registro de pagos en efectivo.
8. **Devoluciones:** registro de devoluciones de clientes, con validación por parte del usuario responsable y reincorporación automática de las cantidades aceptadas al inventario disponible.
9. **Ajustes de inventario:** operaciones autorizadas exclusivamente por el Administrador, con un motivo registrado.
10. **Precios de venta:** modificación directa por parte del usuario autorizado para inventario, con registro de auditoría.
11. **Documentos y comprobantes:** generación de consecutivos internos, registro de información básica de las operaciones y emisión de

comprobantes imprimibles. La facturación electrónica será simulada, sin integración real con la DIAN.

12. **Seguridad y auditoría:** autenticación, autorización por roles y registro básico de las operaciones relevantes.

Debes especificar cada módulo mediante requisitos verificables, sin convertir esta lista inicial en una implementación técnica prematura.

6. Reglas de negocio confirmadas

Las siguientes reglas se consideran decisiones aprobadas. No las presentes como preguntas pendientes ni las cambies por iniciativa propia.

RN-01. Integridad del inventario

No se permitirá completar una venta cuya cantidad solicitada supere las existencias disponibles.

La solución deberá preservar esta regla incluso si se producen operaciones simultáneas. Aunque el proyecto prioriza una demostración académica pequeña, no debe aceptar cantidades negativas por errores de concurrencia.

RN-02. Productos con historial

Los productos que tengan operaciones asociadas no deberán eliminarse físicamente. Se desactivarán, conservando su historial y las referencias desde los documentos y movimientos existentes.

RN-03. Compras completas

Una compra se registrará y recibirá en una sola operación. La recepción parcial de una compra queda fuera del alcance inicial.

RN-04. Método de valoración

El método de valoración del inventario será el costo promedio ponderado. Debes identificar las reglas de cálculo que necesitan especificación adicional, sin inventar fórmulas o excepciones no acordadas.

RN-05. Cliente obligatorio

Toda venta deberá estar asociada a un cliente registrado en el sistema.

RN-06. Medio de pago

El alcance inicial contempla pagos en efectivo. No se incluirá una modalidad de venta a crédito.

RN-07. Devoluciones de clientes

Las devoluciones se validarán de forma flexible por el usuario responsable. No se ha exigido que toda devolución esté vinculada obligatoriamente a una venta original, pero debes identificar los controles mínimos necesarios para mantener consistencia, prevenir errores y conservar trazabilidad.

Las cantidades aceptadas en una devolución se reincorporarán automáticamente al inventario disponible.

RN-08. Ajustes de inventario

Solo el Administrador podrá ejecutar y autorizar ajustes. Cada ajuste deberá contar con un motivo documentado y conservar su historial.

RN-09. Modificación de precios

El precio de venta será definido directamente por el personal autorizado para gestionar inventario. Los cambios deberán quedar registrados en la auditoría.

RN-10. Terceros

Los clientes y proveedores deben registrarse en el sistema. Una misma persona o empresa puede desempeñar ambos roles comerciales.

RN-11. Auditoría

La auditoría básica deberá registrar:

- Usuario responsable.
- Fecha y hora.
- Descripción de la operación.

No añadas IP, valores anteriores y posteriores ni otros campos como requisitos confirmados. Si consideras necesario proponerlos, identifícalos como mejoras opcionales y justifica su utilidad.

RN-12. Documentos

Los comprobantes utilizarán consecutivos internos y tendrán una estructura básica apropiada para identificar la operación. La simulación de facturación electrónica no debe presentarse como una integración real con la DIAN.

7. Contexto geográfico y monetario

El sistema se orientará a Colombia.

- La moneda de operación será el peso colombiano (COP).
- La identificación de personas y empresas contemplará CC y NIT.
- Los datos de clientes y proveedores deberán distinguir, cuando corresponda, entre personas naturales y jurídicas.

En esta etapa no investigues ni desarrolles requisitos sobre reglas tributarias aplicables. Limita el alcance documental a la simulación de facturación y a la emisión de comprobantes internos descrita anteriormente.

No inventes obligaciones legales ni presentes los comprobantes generados por el sistema como documentos fiscales oficialmente autorizados.

8. Funcionalidades fuera del alcance inicial

No incluyas automáticamente las siguientes funcionalidades:

- Varias sedes o bodegas.
- Recepción parcial de compras.
- Pagos a crédito.
- Integración real con la DIAN.
- Importación inicial de productos mediante Excel.
- Funcionamiento sin conexión a internet.
- Integración con lectores físicos de códigos de barras.
- Gestión de lotes, series, vencimientos o garantías.
- Devoluciones a proveedores.
- Reservas de productos.
- Cierre formal de períodos contables.

- Integraciones con sistemas externos no especificados.

Si identificas que alguna de estas funcionalidades es indispensable para satisfacer un requisito confirmado, explica el conflicto y formula una pregunta. No la agregues silenciosamente al alcance.

9. Objetivo de esta etapa

Tu tarea es realizar la planificación inicial y el levantamiento formal de requisitos del sistema.

Debes convertir el contexto de negocio en documentación clara, coherente, verificable y suficientemente completa para que otra persona o agente pueda utilizarla como entrada a la etapa de diseño.

No debes programar, crear entidades JPA, definir tablas de MySQL, generar controladores, escribir componentes React ni implementar funcionalidades en esta etapa.

Tampoco debes redactar todavía los prompts de las siguientes etapas. Tu resultado debe ser la documentación de requisitos aprobable que permitirá preparar esos prompts posteriormente.

10. Actividades obligatorias

10.1. Análisis del problema

Documenta:

- Problema que se busca resolver.
- Situación actual y necesidades del negocio, distinguiendo hechos confirmados de supuestos.
- Objetivos generales y específicos.
- Beneficios esperados.
- Alcance del producto.
- Exclusiones explícitas.
- Restricciones conocidas.
- Riesgos iniciales y dependencias.

No inventes cómo trabaja actualmente la ferretería. Cuando no exista información suficiente, identifica qué debe confirmarse.

10.2. Identificación de interesados y actores

Identifica:

- Responsable o propietario del negocio.
- Administrador.
- Usuario operativo.
- Cliente.
- Proveedor.
- Otros actores únicamente cuando exista una justificación.

Para cada actor, documenta sus objetivos, responsabilidades, necesidades y relación con el sistema.

Distingue los usuarios que interactúan directamente con la aplicación de las entidades externas que solo participan en los procesos del negocio.

10.3. Requisitos funcionales

Define requisitos funcionales con identificadores únicos y agrupados por módulo.

Cada requisito debe:

- Describir una capacidad concreta del sistema.
- Ser claro, necesario y no ambiguo.
- Tener un identificador estable, por ejemplo, RF-PRO-001.
- Especificar las condiciones relevantes y el resultado esperado.
- Contar con criterios de aceptación verificables.
- Identificar los actores o roles relacionados.
- Mantener trazabilidad con las reglas de negocio correspondientes.

No utilices expresiones vagas como “el sistema debe ser rápido”, “fácil de usar” o “gestionar correctamente” sin convertirlas en condiciones verificables.

Incluye los flujos alternativos y los casos de error relevantes, especialmente para ventas sin existencias, devoluciones, ajustes no autorizados y operaciones inválidas.

10.4. Requisitos no funcionales

Identifica requisitos de calidad utilizando ISO/IEC 25010:2023 como referencia.

Evalúa las características pertinentes para este proyecto, incluyendo:

- Adecuación funcional.
- Eficiencia del desempeño.
- Compatibilidad.
- Capacidad de interacción y usabilidad.
- Fiabilidad.
- Seguridad.
- Mantenibilidad.
- Flexibilidad.
- Seguridad operacional cuando corresponda al contexto.

No es necesario incluir todas las características de manera artificial. Justifica cuáles aplican y cuáles no resultan relevantes para el alcance.

Cada requisito de calidad debe tener identificador, justificación, condición verificable y método de evaluación.

Como el sistema prioriza una demostración académica pequeña, propone umbrales medibles y proporcionales para los aspectos cuantificables. Si no existen datos para fijar una cifra, presenta una propuesta explícita que deba aprobarse, sin describirla como una decisión ya confirmada.

10.5. Reglas de negocio

Consolida las reglas de negocio proporcionadas en este prompt y asígnales identificadores estables.

Para cada regla, documenta:

- Identificador.
- Descripción.
- Justificación.
- Requisitos relacionados.
- Criterio para verificar su cumplimiento.

Separa claramente:

- Reglas confirmadas.
- Propuestas de mejora.
- Decisiones pendientes.

No contradigas las reglas confirmadas ni las reemplaces por supuestos propios.

10.6. Casos de uso y procesos principales

Documenta los casos de uso pertinentes para:

- Registrar y administrar productos.
- Consultar y controlar existencias.
- Registrar compras.
- Registrar ventas.
- Gestionar clientes y proveedores.
- Registrar devoluciones.
- Realizar ajustes autorizados.
- Modificar precios.
- Consultar información y auditoría según los permisos.
- Generar comprobantes.

Para cada caso de uso, describe el actor principal, precondiciones, flujo principal, alternativas, errores relevantes y postcondiciones.

No es obligatorio generar diagramas gráficos en esta etapa si una representación textual estructurada permite revisar los procesos con mayor claridad.

10.7. Matriz de permisos

Define una matriz preliminar de permisos para Administrador y Usuario operativo.

Incluye las operaciones de consulta, creación, modificación, desactivación, autorización y ejecución que correspondan.

Marca como pendientes los permisos que no puedan deducirse de las decisiones confirmadas. Explica los riesgos de seguridad asociados a permisos ambiguos.

10.8. Criterios de aceptación

Define criterios de aceptación que permitan comprobar objetivamente si el sistema satisface cada requisito relevante.

Incluye, como mínimo, escenarios para:

- Intentar vender más unidades de las disponibles.
- Registrar una venta válida.
- Intentar una operación sin los permisos requeridos.
- Registrar una compra con varias líneas.
- Calcular el costo promedio ponderado.
- Desactivar un producto con historial.
- Registrar una devolución.
- Ejecutar un ajuste autorizado y rechazar uno no autorizado.
- Modificar un precio y conservar su registro de auditoría.
- Generar y consultar un comprobante interno.

Utiliza escenarios del tipo Dado/Cuando/Entonces cuando faciliten la comprensión y verificación.

10.9. Riesgos y decisiones pendientes

Elabora un registro de riesgos iniciales y un registro de decisiones pendientes.

Presta especial atención a:

- Consistencia de existencias.
- Cálculo de costos.
- Permisos por rol.
- Validaciones de devoluciones.
- Estados y cancelación de documentos.
- Consecutivos internos.
- Datos obligatorios de clientes y proveedores.
- Seguridad de la autenticación.
- Respaldo y recuperación de información.
- Alcance de las pruebas y criterios de calidad.

Para cada riesgo, documenta probabilidad, impacto, prioridad y posible mitigación cuando haya información suficiente. Si la estimación es preliminar, indícalo.

No conviertas todas las dudas en bloqueantes. Clasifícalas como:

1. Bloqueantes para aprobar los requisitos.
2. Necesarias antes de diseñar un módulo específico.
3. Resolubles durante el diseño o la implementación.
4. Mejoras opcionales para una versión posterior.

10.10. Trazabilidad

Crea una matriz de trazabilidad que relacione:

- Objetivos del negocio.
- Requisitos funcionales.
- Requisitos no funcionales.
- Reglas de negocio.

- Casos de uso.
- Criterios de aceptación.

Los identificadores deben ser consistentes en todos los documentos. No inventes relaciones que no estén justificadas.

11. Documentación que debes entregar

Organiza los resultados en un directorio documental de esta etapa, proponiendo nombres claros y consistentes para los archivos.

Entrega, como mínimo:

1. 01_planificacion_y_alcance.md
2. 02_actores_y_mapa_de_interesados.md
3. 03_especificacion_de_requisitos.md
4. 04_reglas_de_negocio.md
5. 05_casos_de_uso_y_procesos.md
6. 06_matriz_de_permisos.md
7. 07_criterios_de_aceptacion.md
8. 08_riesgos_y_decisiones_pendientes.md
9. 09_matriz_de_trazabilidad.md

Si el entorno permite crear archivos, genera los documentos con contenido real y coherente, no solo con títulos o plantillas vacías. Si no permite crear archivos, presenta el contenido organizado por documento y especifica claramente esa limitación.

Cada documento deberá incluir:

- Nombre.
- Descripción.
- Fecha de creación.
- Fecha de actualización.
- Autor.

- Versión.
- Estado del documento, por ejemplo: borrador, en revisión o aprobado.

Utiliza una fecha real al generar los documentos y mantén coherencia entre las versiones. No afirmes que un documento fue aprobado por el responsable del negocio sin que exista dicha aprobación.

12. Gestión de la incertidumbre y comunicación

No supongas respuestas para decisiones que no se hayan tomado.

Cuando falte información:

1. Comprueba si ya está resuelta en el contexto.
2. Si no está resuelta, determina si realmente impide avanzar.
3. Si es bloqueante, formula una pregunta concreta y explica brevemente por qué importa.
4. Si no es bloqueante, registra la incertidumbre y continúa con las demás actividades.
5. Si propones una solución, etiquétala como propuesta, no como requisito aprobado.

Formula preguntas breves, claras y agrupadas por tema. Evita pedir de nuevo información ya confirmada y no abrumes al responsable con preguntas técnicas innecesarias.

Mantén separadas las decisiones del negocio de las decisiones técnicas que todavía deban evaluarse.

13. Condiciones de calidad de la entrega

Antes de concluir, verifica que:

- El alcance sea coherente con las decisiones confirmadas.
- No existan requisitos contradictorios.
- Cada requisito relevante tenga un identificador único.
- Los requisitos sean verificables.

- Las reglas de negocio estén vinculadas con los requisitos correspondientes.
- Los criterios de aceptación cubran los flujos críticos.
- Los permisos estén documentados y las ambigüedades sean visibles.
- Las decisiones pendientes estén priorizadas.
- Los requisitos de calidad sean proporcionales al proyecto.
- La trazabilidad entre objetivos, requisitos y pruebas sea consistente.
- No se hayan incorporado funcionalidades excluidas sin aprobación.
- No se haya comenzado la implementación ni el diseño técnico detallado.

14. Resultado esperado

Entrega una primera versión completa de la documentación de planificación y requisitos.

Finaliza con un resumen ejecutivo que indique:

- Qué quedó definido.
- Qué decisiones siguen pendientes.
- Qué cuestiones bloquean la aprobación, si las hay.
- Qué documentos se generaron.
- Qué validaciones debe realizar el responsable del negocio.
- Si la etapa está lista para revisión o si necesita aclaraciones.

No avances automáticamente a la etapa de diseño. Espera la revisión y aprobación de los resultados antes de continuar con el siguiente bloque del ciclo de vida del software.

PROMPT #2

ETAPA 2 — DISEÑO TÉCNICO Y ARQUITECTÓNICO DEL SISTEMA DE INVENTARIO PARA FERRETERÍA

1. Rol que debes asumir

Actúa como un arquitecto de software y diseñador de sistemas con experiencia en Java, Spring Boot, React, MySQL, sistemas de inventario, diseño de API REST, seguridad web y modelado de datos.

Tu responsabilidad en esta etapa es producir una **especificación de diseño técnico detallada, coherente, verificable y lista para orientar la implementación posterior** de un sistema web de inventario para una ferretería.

No debes limitarte a recomendar tecnologías ni a describir conceptos generales. Debes especificar explícitamente:

- La arquitectura del sistema.
- La estructura de directorios y paquetes del backend.
- La estructura de directorios del frontend.
- Las entidades, sus atributos, tipos de datos, restricciones y relaciones.
- El modelo de inventario y movimientos.
- El cálculo del costo promedio ponderado.
- Los contratos de los endpoints REST.
- Los DTO de entrada y salida.
- Las reglas de validación.
- Los mecanismos de autenticación y autorización.
- Los diagramas de secuencia de las operaciones críticas.
- Las decisiones técnicas y las cuestiones que siguen pendientes.

El resultado debe ser suficientemente preciso para que, en una etapa posterior, otro agente pueda implementar el sistema sin tener que inventar la

arquitectura, crear entidades arbitrariamente ni modificar las reglas de negocio.

2. Regla fundamental: no inventar ni cambiar decisiones

Debes seguir este documento y los documentos aprobados de la Etapa 1 como especificaciones obligatorias.

Aplica estas reglas durante todo el trabajo:

1. No inventes funcionalidades, requisitos, roles, entidades, atributos, relaciones ni reglas de negocio que no estén respaldados por el contexto aprobado.
2. No cambies las tecnologías ni las decisiones arquitectónicas confirmadas.
3. No agregues funcionalidades porque consideres que serían útiles en un sistema empresarial.
4. No sustituyas una decisión confirmada por una alternativa que prefieras.
5. No conviertas una recomendación técnica en un requisito de negocio confirmado.
6. No resuelvas unilateralmente decisiones que continúen pendientes.
7. No elimines requisitos de la Etapa 1 por considerarlos innecesarios.
8. No dupliques entidades, responsabilidades o conceptos que puedan representarse de forma consistente mediante el modelo aprobado.
9. Si detectas una contradicción entre este prompt y un documento aprobado de la Etapa 1, registra la contradicción, identifica los documentos afectados y solicita aclaración antes de resolverla.
10. Si falta información necesaria, registra la decisión pendiente, presenta una propuesta técnica claramente identificada y explica sus consecuencias. No la presentes como una decisión aprobada.
11. Si un atributo, endpoint, paquete o entidad se deriva directamente de una decisión ya confirmada, puedes especificarlo como parte del diseño, explicando su trazabilidad.

12. No solicites aprobación para cada detalle técnico menor. Resuelve los detalles derivados de las decisiones confirmadas utilizando la alternativa más sencilla y coherente, y documenta la justificación.

Tratamiento obligatorio de la información

Distingue en la documentación entre:

- **Confirmado:** requisito o decisión ya aprobada.
- **Derivado del diseño:** elemento técnico necesario para materializar una decisión confirmada.
- **Propuesta pendiente de aprobación:** alternativa que necesita una decisión explícita.
- **Fuera de alcance:** funcionalidad que no debe implementarse en la versión inicial.

No uses expresiones ambiguas como «se podría agregar» sin identificar si la sugerencia está dentro o fuera del alcance.

3. Documentación previa obligatoria

Antes de diseñar, inspecciona los documentos aprobados de la Etapa 1 que estén disponibles en el repositorio o en el contexto proporcionado.

Se espera encontrar, cuando hayan sido generados y aprobados, los siguientes documentos:

- 01_planificacion_y_alcance.md
- 02_actores_y_mapa_de_interesados.md
- 03_especificacion_de_requisitos.md
- 04_reglas_de_negocio.md
- 05_casos_de_uso_y_procesos.md
- 06_matriz_de_permisos.md
- 07_criterios_de_aceptacion.md
- 08_riesgos_y_decisiones_pendientes.md
- 09_matriz_de_trazabilidad.md

Estos archivos constituyen la fuente de verdad para los requisitos y reglas de negocio.

Si los archivos no están disponibles:

- No afirmes haberlos revisado.
- No inventes su contenido ni sus identificadores de requisitos.
- Utiliza las decisiones explícitas de este prompt como contexto disponible.
- Identifica qué información no se pudo verificar.
- Marca como provisional cualquier parte del diseño que dependa de esa información.
- Solicita los documentos faltantes cuando sean imprescindibles para resolver una contradicción o completar un diseño seguro.

No inicies la programación, aunque encuentres archivos de código existentes. Esta etapa es exclusivamente de diseño.

4. Contexto del sistema

4.1. Producto

Se desarrollará un sistema web para administrar las operaciones básicas de inventario de una ferretería pequeña.

El sistema es una demostración académica que debe mantener buenas prácticas de ingeniería de software, una estructura clara, seguridad apropiada y posibilidades razonables de mantenimiento.

No se pretende construir un ERP ni una plataforma empresarial sobredimensionada.

4.2. Tecnologías obligatorias

Backend

- Java 21.
- Spring Boot 3.
- Spring MVC.

- Spring Data JPA.
- Hibernate.
- Spring Security.
- Maven.
- API REST.
- JSON.
- OpenAPI/Swagger para documentar la API.
- JUnit y Mockito para las pruebas automatizadas posteriores.
- Pruebas de integración con Spring cuando corresponda.

Frontend

- React.
- Material UI.
- Formik y Yup para formularios y validación cuando corresponda al diseño aprobado.
- Playwright para las pruebas de interfaz previstas, salvo que exista una justificación documentada para otra herramienta.

Base de datos

- MySQL.
- Persistencia mediante JPA/Hibernate.
- No incorporar Flyway ni Liquibase en esta versión: se ha decidido no utilizar herramientas de migración en el alcance actual.

Comunicación

- API REST.
- JSON.
- Autenticación basada en sesiones de Spring Security.
- Contratos y convenciones HTTP consistentes.

No sustituyas estas tecnologías por otras.

4.3. Repositorios independientes

El sistema estará dividido en dos repositorios separados:

1. Repositorio del backend Spring Boot.
2. Repositorio del frontend React.

Debes especificar cómo se organiza cada repositorio y cómo se comunican entre sí.

No conviertas el sistema en un monorepo.

4.4. Alcance funcional confirmado

El sistema incluirá:

- Gestión de productos y referencias comerciales.
- Gestión de inventario.
- Registro de inventario inicial.
- Registro de compras.
- Registro de ventas.
- Registro de clientes.
- Registro de proveedores.
- Pagos en efectivo.
- Devoluciones de clientes.
- Ajustes de inventario.
- Gestión de precios de venta.
- Cálculo de costo promedio ponderado.
- Historial persistente de movimientos de inventario.
- Registro básico de auditoría.
- Comprobantes internos imprimibles.
- Simulación de facturación, sin integración real con sistemas externos.

Los clientes y proveedores siempre deben estar registrados. Una misma persona o entidad puede desempeñar ambos papeles.

Toda venta requiere un cliente registrado.

Las compras pueden contener múltiples líneas y se reciben completas en una única operación. No existen recepciones parciales en el alcance inicial.

El pago es en efectivo. No se admiten ventas a crédito.

4.5. Referencia comercial y unidades

La referencia vendible del catálogo se define mediante la combinación de:

- Producto.
- Marca.
- Presentación.

El código de barras debe ser único en el catálogo.

Cada referencia debe tener una unidad base configurable. También deben poder configurarse, según las decisiones del negocio:

- Las conversiones de unidades permitidas.
- Las fracciones mínimas permitidas.
- La precisión y las reglas de redondeo aplicables a las cantidades.

No supongas que todos los productos se venden por unidad entera. Por ejemplo, ciertos productos podrían manejar fracciones, pero no implementes reglas específicas para un producto sin que estén aprobadas.

El diseño debe precisar cómo se almacenan y validan las cantidades y cómo se evitan errores de conversión. Si la política exacta de precisión o redondeo no está especificada en los documentos aprobados, regístrala como decisión pendiente.

4.6. Roles

Se han confirmado dos roles:

- ADMINISTRADOR
- USUARIO_OPERATIVO

El administrador puede ejecutar y autorizar ajustes de inventario, cancelar compras y ventas cuando sea seguro hacerlo y realizar las acciones administrativas correspondientes.

El usuario operativo puede realizar operaciones normales según la matriz de permisos aprobada. Puede modificar precios de venta cuando tenga autorización para hacerlo.

Debes consultar la matriz de permisos de la Etapa 1 para precisar los permisos de cada operación.

No supongas que los dos roles tienen permisos idénticos salvo que así lo indiquen los documentos aprobados.

El backend es la autoridad para autorizar operaciones. Ocultar un botón en el frontend no sustituye la autorización del servidor.

4.7. Reglas de inventario y costos

El sistema mantendrá:

1. Un saldo actual de inventario por referencia.
2. Un historial persistente de movimientos.
3. La trazabilidad entre las operaciones comerciales y los movimientos generados.

El inventario no puede quedar negativo, incluso si se producen ventas concurrentes.

El costo de inventario se calculará mediante el método de costo promedio ponderado.

El inventario inicial se registrará mediante un movimiento específico de inicialización que incluya cantidad y costo unitario de referencia. No se debe simular una compra para inicializar el inventario.

Para devoluciones:

- Si la devolución está vinculada a una venta original, se utilizará el costo unitario histórico registrado en esa venta para valorar la cantidad devuelta.

- Si la devolución no está vinculada a una venta, se utilizará el costo promedio ponderado vigente según la regla confirmada.

El diseño debe distinguir entre el costo promedio vigente y el costo histórico registrado en una venta. No uses un único campo mutable como sustituto de ambos conceptos.

Debes definir cómo se relacionan los movimientos de inventario con compras, ventas, devoluciones, ajustes, inicializaciones y cancelaciones.

4.8. Cancelaciones

El administrador puede cancelar compras y ventas con un motivo.

El documento original debe conservarse. La cancelación no debe borrar físicamente la operación ni su historial.

Cuando corresponda, la cancelación debe generar movimientos compensatorios.

El diseño debe impedir cancelaciones que provoquen inconsistencias por operaciones posteriores. Si no existe una reversión segura, la operación debe bloquearse o requerir una resolución explícita conforme a las reglas aprobadas.

No decidas unilateralmente qué debe ocurrir cuando una venta ya tiene devoluciones parciales. Registra este caso como decisión pendiente si no está resuelto en la Etapa 1.

4.9. Precios y eliminación de productos

El usuario autorizado puede definir y modificar el precio de venta.

Los cambios de precio deben quedar auditados.

Los productos con historial no se eliminan físicamente. Se desactivan lógicamente para impedir nuevas operaciones cuando corresponda, sin destruir el historial.

4.10. Auditoría

La auditoría básica debe permitir identificar:

- Usuario responsable.
- Fecha y hora.

- Descripción de la acción.

No consideres obligatorios el almacenamiento de direcciones IP, fotografías, geolocalización u otros datos que no hayan sido aprobados.

Si propones guardar valores anteriores y posteriores para determinadas operaciones, identifícalo como propuesta técnica y justifica su necesidad.

4.11. Documentos y contexto colombiano

El sistema debe generar comprobantes internos imprimibles, con numeración consecutiva interna, estados básicos, líneas y totales de la operación.

La facturación es simulada. No se implementará integración real con la DIAN ni con otros sistemas externos de facturación.

El contexto utiliza pesos colombianos (COP) y documentos de identificación como CC/NIT.

No investigues, diseñes ni desarrolles reglas tributarias. No agregues cálculos de impuestos ni obligaciones legales que no formen parte del alcance aprobado. Limítate a los importes y comprobantes internos que necesita la operación.

5. Exclusiones obligatorias

No diseñes como parte de la versión inicial:

- Múltiples sucursales o almacenes.
- Recepciones parciales de compras.
- Pagos a crédito.
- Integración real con la DIAN.
- Importación desde Excel.
- Integración con lectores de códigos de barras.
- Operación sin conexión.
- Devoluciones a proveedores.
- Gestión de lotes, números de serie, vencimientos o garantías.
- Cierre formal de períodos contables.

- Integraciones externas no solicitadas.

No agregues microservicios, Kafka, Redis, CQRS, Event Sourcing, Kubernetes, un motor de permisos complejo ni otras tecnologías o patrones que no sean necesarios para el alcance confirmado.

No uses Clean Architecture ni Arquitectura Hexagonal. La arquitectura seleccionada es la arquitectura tradicional por capas.

6. Arquitectura de backend obligatoria

El backend debe organizarse mediante las capas:

Controller → Service → Repository

Utiliza una organización por módulos funcionales y capas. La estructura propuesta a continuación es obligatoria como punto de partida. Puedes agregar subpaquetes estrictamente necesarios para materializar el diseño, pero no eliminar ni renombrar los paquetes establecidos sin identificar una contradicción real y solicitar aprobación.

Nombre base del paquete:

com.ferreteria.inventario

Estructura requerida:

backend/

```
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/
│   │   │       └── ferreteria/
│   │   │           └── inventario/
│   │   │               ├── InventarioApplication.java
│   │   │               ├── config/
│   │   │               ├── security/
```

```
│   │   │           │   ├── config/
│   │   │           │   ├── handler/
│   │   │           │   └── service/
│   │   │           ├── shared/
│   │   │           │   ├── exception/
│   │   │           │   ├── response/
│   │   │           │   ├── validation/
│   │   │           │   └── audit/
│   │   │           ├── auth/
│   │   │           │   ├── controller/
│   │   │           │   ├── dto/
│   │   │           │   │   ├── request/
│   │   │           │   │   └── response/
│   │   │           │   └── service/
│   │   │           ├── user/
│   │   │           │   ├── controller/
│   │   │           │   ├── dto/
│   │   │           │   │   ├── request/
│   │   │           │   │   └── response/
│   │   │           │   ├── entity/
│   │   │           │   ├── repository/
│   │   │           │   └── service/
│   │   │           ├── catalog/
│   │   │           │   ├── controller/
│   │   │           │   ├── dto/
```

```
│   │   │           │   │   ├── request/
│   │   │           │   │   └── response/
│   │   │           │   ├── entity/
│   │   │           │   ├── repository/
│   │   │           │   └── service/
│   │   │           ├── customer/
│   │   │           │   ├── controller/
│   │   │           │   ├── dto/
│   │   │           │   │   ├── request/
│   │   │           │   │   └── response/
│   │   │           │   ├── entity/
│   │   │           │   ├── repository/
│   │   │           │   └── service/
│   │   │           ├── supplier/
│   │   │           │   ├── controller/
│   │   │           │   ├── dto/
│   │   │           │   │   ├── request/
│   │   │           │   │   └── response/
│   │   │           │   ├── entity/
│   │   │           │   ├── repository/
│   │   │           │   └── service/
│   │   │           ├── inventory/
│   │   │           │   ├── controller/
│   │   │           │   ├── dto/
│   │   │           │   │   ├── request/
```

```
│   │   │           │   │   └── response/
│   │   │           │   ├── entity/
│   │   │           │   ├── repository/
│   │   │           │   └── service/
│   │   │           ├── purchase/
│   │   │           │   ├── controller/
│   │   │           │   ├── dto/
│   │   │           │   │   ├── request/
│   │   │           │   │   └── response/
│   │   │           │   ├── entity/
│   │   │           │   ├── repository/
│   │   │           │   └── service/
│   │   │           ├── sale/
│   │   │           │   ├── controller/
│   │   │           │   ├── dto/
│   │   │           │   │   ├── request/
│   │   │           │   │   └── response/
│   │   │           │   ├── entity/
│   │   │           │   ├── repository/
│   │   │           │   └── service/
│   │   │           ├── returns/
│   │   │           │   ├── controller/
│   │   │           │   ├── dto/
│   │   │           │   │   ├── request/
│   │   │           │   │   └── response/
```

```
│   │   │           │   ├── entity/
│   │   │           │   ├── repository/
│   │   │           │   └── service/
│   │   │           ├── document/
│   │   │           │   ├── controller/
│   │   │           │   ├── dto/
│   │   │           │   │   ├── request/
│   │   │           │   │   └── response/
│   │   │           │   ├── entity/
│   │   │           │   ├── repository/
│   │   │           │   └── service/
│   │   │           └── reporting/
│   │   │               ├── controller/
│   │   │               ├── dto/
│   │   │               ├── repository/
│   │   │               └── service/
│   │   └── resources/
│   │       ├── application.yml
│   │       ├── application-dev.yml
│   │       ├── application-test.yml
│   │       └── application-prod.yml
│   └── test/
│       └── java/
│           └── com/
│               └── ferreteria/
```

```
│                   └── inventario/
│                       └── [estructura de pruebas correspondiente a los módulos]
```

La estructura es una especificación organizativa, no una autorización para crear todas las clases imaginables. En los documentos de diseño debes indicar qué responsabilidades y clases conceptuales pertenecen a cada paquete y qué módulos realmente necesitan persistencia.

No agregues entidades JPA, repositorios ni DTO redundantes por uniformidad. Los módulos auth, security, shared y reporting deben tener únicamente los componentes necesarios para sus responsabilidades.

6.1. Responsabilidades por capa

Especifica estas responsabilidades:

- controller: recibe solicitudes HTTP, aplica validación de entrada, invoca servicios y devuelve respuestas HTTP.
- service: contiene los casos de uso, las reglas de negocio y la coordinación transaccional.
- repository: declara las operaciones de persistencia necesarias y utiliza Spring Data JPA.
- entity: contiene las entidades JPA de persistencia.
- dto/request: contiene contratos de entrada de la API.
- dto/response: contiene contratos de salida.
- security: configura la autenticación, las sesiones, las reglas de autorización y los mecanismos de protección web.
- shared/exception: contiene el tratamiento global y consistente de errores.
- shared/audit: contiene los componentes reutilizables estrictamente necesarios para la auditoría.
- config: contiene la configuración transversal.

No coloques reglas de negocio en los controladores. Los servicios no deben devolver ResponseEntity. Los controladores no deben acceder directamente a repositorios.

No introduzcas una capa de dominio independiente ni interfaces de puertos y adaptadores como si se hubiese elegido Arquitectura Hexagonal.

7. Estructura obligatoria del frontend

El frontend se organizará por módulos funcionales, con componentes compartidos y una capa clara para la comunicación con la API.

Estructura requerida:

```
frontend/
├── package.json
├── public/
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── app/
    │   ├── router/
    │   ├── providers/
    │   └── theme/
    ├── assets/
    ├── components/
    │   ├── layout/
    │   ├── feedback/
    │   ├── forms/
    │   ├── tables/
    │   └── common/
    ├── features/
    │   ├── auth/
    │   │   ├── pages/
```

```
│   │   ├── components/
│   │   ├── services/
│   │   └── validation/
│   ├── users/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── services/
│   │   └── validation/
│   ├── catalog/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── services/
│   │   └── validation/
│   ├── customers/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── services/
│   │   └── validation/
│   ├── suppliers/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── services/
│   │   └── validation/
│   ├── inventory/
│   │   ├── pages/
```

```
│   │   ├── components/
│   │   ├── services/
│   │   └── validation/
│   ├── purchases/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── services/
│   │   └── validation/
│   ├── sales/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── services/
│   │   └── validation/
│   ├── returns/
│   │   ├── pages/
│   │   ├── components/
│   │   ├── services/
│   │   └── validation/
│   ├── documents/
│   │   ├── pages/
│   │   ├── components/
│   │   └── services/
│   └── reports/
│       ├── pages/
│       ├── components/
```

```
│       └── services/
├── services/
│   └── api/
├── hooks/
├── utils/
└── tests/
```

Debes especificar las responsabilidades de cada directorio y el contenido previsto para cada módulo.

No es obligatorio crear archivos de validación vacíos ni subdirectorios sin uso. La estructura debe orientar la implementación sin generar complejidad artificial.

7.1. Reglas del frontend

- React y Material UI son obligatorios.
- Formik y Yup se utilizarán para los formularios cuando corresponda.
- La comunicación HTTP se centralizará en src/services/api/.
- La autenticación debe utilizar la sesión del backend; no inventes almacenamiento de JWT.
- El frontend debe manejar estados de carga, errores, formularios y respuestas vacías.
- Las listas que lo requieran deben admitir paginación y filtros de acuerdo con los contratos del backend.
- La interfaz puede ocultar acciones que el usuario no tenga permitidas, pero la autorización definitiva corresponde al backend.
- No dupliques las reglas de negocio del servidor como única barrera de protección.
- No agregues un gestor global de estado complejo si React y las herramientas existentes son suficientes.

8. Modelo de datos: especificación obligatoria

Debes entregar un modelo entidad-relación completo y un diccionario de datos. No es suficiente enumerar entidades sin especificar sus atributos y relaciones.

Para cada entidad debes indicar:

1. Nombre conceptual.
2. Nombre de tabla propuesto.
3. Responsabilidad.
4. Atributos.
5. Tipo conceptual y tipo SQL MySQL propuesto.
6. Si admite NULL.
7. Clave primaria.
8. Claves foráneas.
9. Restricciones UNIQUE y CHECK cuando sean compatibles con la versión objetivo de MySQL.
10. Índices necesarios y justificación.
11. Relaciones con otras entidades.
12. Cardinalidad.
13. Reglas de borrado y actualización.
14. Si tiene eliminación lógica.
15. Si contiene datos históricos que no deben modificarse.
16. Requisitos de auditoría y consistencia.
17. Requisitos de concurrencia o transaccionalidad cuando correspondan.

No asignes valores predeterminados ni restricciones de negocio que no estén justificadas.

8.1. Entidades conceptuales que debes evaluar y especificar

Diseña el modelo completo para cubrir el alcance aprobado. Como mínimo, debes evaluar los siguientes conceptos:

Identidad y seguridad

- Usuario del sistema.
- Rol o mecanismo equivalente para representar los roles confirmados.

Catálogo

- Producto.
- Marca.
- Presentación comercial.
- Referencia vendible del catálogo, si se requiere como entidad diferenciada.
- Unidad de medida.
- Conversión de unidad, si corresponde al modelo elegido.

Terceros

- Cliente.
- Proveedor.

Evalúa cómo representar una misma persona o entidad como cliente y proveedor sin duplicar innecesariamente su información. No impongas una entidad genérica adicional si los documentos aprobados o el análisis no justifican esa decisión.

Inventario

- Saldo de inventario por referencia.
- Movimiento de inventario.
- Tipo de movimiento, mediante un mecanismo simple y consistente.

Compras

- Compra.
- Línea de compra.

Ventas

- Venta.

- Línea de venta.
- Pago, únicamente si el diseño necesita representar explícitamente el pago en efectivo o su trazabilidad.

Devoluciones

- Devolución de cliente.
- Línea de devolución, si es necesaria para representar varios productos o cantidades.

Documentos y control

- Documento o comprobante, si su representación separada es necesaria.
- Registro de auditoría.
- Entidades adicionales solamente si son necesarias para materializar requisitos confirmados.

Esta lista identifica conceptos que deben evaluarse; no significa que debas crear una tabla para cada concepto automáticamente. Si dos conceptos pueden representarse de forma segura en una sola entidad, explica la decisión. Si una entidad separada es necesaria, justifica su responsabilidad.

8.2. Atributos mínimos que debes especificar

Para cada entidad seleccionada, define la lista completa de atributos. No dejes que el agente de implementación tenga que decidir posteriormente si un campo es necesario.

La especificación debe considerar, cuando corresponda y esté justificado:

- Identificador.
- Código de negocio.
- Nombre o descripción.
- Estado.
- Fecha y hora de creación.
- Fecha y hora de actualización.
- Usuario responsable.

- Cantidad.
- Unidad de medida.
- Costo unitario.
- Precio unitario.
- Subtotal y total.
- Referencia al documento de origen.
- Motivo de cancelación o ajuste.
- Costo histórico.
- Estado de la operación.
- Relaciones con las líneas y los movimientos generados.

La lista anterior no autoriza a añadir todos estos campos a todas las tablas. Debes especificar exactamente cuáles pertenecen a cada entidad, cuáles son obligatorios y por qué.

8.3. Tipos de datos y precisión

Define de forma explícita:

- Tipos SQL de MySQL.
- Precisión y escala para cantidades, precios, costos y totales.
- Estrategia de redondeo, identificando las decisiones de negocio pendientes.
- Tipos y precisión para fechas y horas.
- Representación de estados.
- Estrategia de generación de identificadores.
- Restricciones de unicidad.
- Estrategia para evitar errores de representación monetaria.

No uses FLOAT ni DOUBLE para almacenar valores monetarios.

No supongas una precisión arbitraria para cantidades fraccionarias si todavía no está definida. Propón una solución técnica y marca la precisión de negocio como pendiente de aprobación.

8.4. Relaciones y eliminación

Especifica todas las relaciones con su cardinalidad y claves foráneas.

Debes determinar, de acuerdo con el historial requerido, qué relaciones deben impedir la eliminación física de un registro.

El diseño debe garantizar que:

- Las ventas históricas mantengan los datos necesarios para su consulta.
- Las líneas conserven los precios y costos históricos pertinentes.
- Los movimientos no desaparezcan cuando se cancele una operación.
- Los productos con historial no se eliminen físicamente.
- Las operaciones comerciales no pierdan sus referencias históricas.
- Las restricciones de base de datos complementen las validaciones del backend.

No uses CascadeType.ALL indiscriminadamente. Justifica cada cascada y el uso o no de orphanRemoval.

8.5. Modelo de inventario y concurrencia

Debes diseñar cómo se actualizan de forma atómica el saldo actual y el historial de movimientos.

Especifica:

- Cuándo se crea un movimiento.
- Qué operación comercial lo origina.
- Cómo se mantiene la correspondencia entre el saldo y los movimientos.
- Cómo se evita descontar más unidades de las disponibles.
- Cómo se manejan dos ventas concurrentes de la misma referencia.
- Cómo se ejecuta el control de inventario dentro de una transacción.

- Qué mecanismo de bloqueo o actualización atómica se utilizará y por qué.
- Qué ocurre si falla una parte de la operación.
- Cómo se evitan movimientos duplicados al reintentar una solicitud.

No basta con consultar el stock y luego restarlo sin un mecanismo que proteja frente a concurrencia.

No es obligatorio implementar Event Sourcing. El historial de movimientos es un registro persistente de inventario, no una autorización para introducir esa arquitectura.

8.6. Costo promedio ponderado

Documenta el cálculo y sus efectos sobre el modelo de datos.

Debes especificar:

- Cómo se inicializa el costo promedio.
- Cómo se actualiza después de una compra.
- Cómo se comportan las ventas frente al costo promedio.
- Cómo se registra el costo histórico de cada línea de venta.
- Cómo se valora una devolución vinculada a una venta.
- Cómo se valora una devolución no vinculada.
- Cómo se manejan los ajustes de inventario que afecten al costo, cuando corresponda.
- Cómo se contabiliza técnicamente la cancelación de compras y ventas en el modelo de inventario.
- Cómo se preserva la coherencia entre cantidades, costos y movimientos.

No inventes una política contable o tributaria. Este diseño se limita a la valoración interna del inventario.

Si las reglas aprobadas no determinan el efecto sobre el costo promedio de una devolución vinculada a una venta, un ajuste o una cancelación, especifica las alternativas y registra el punto como pendiente. No lo ocultes detrás de una implementación técnica.

8.7. Compras, ventas, devoluciones y cancelaciones

Entrega un modelo consistente para estas operaciones.

Para cada operación define:

- Datos de cabecera.
- Líneas.
- Estados.
- Totales.
- Relaciones.
- Movimientos generados.
- Cambios de inventario.
- Tratamiento de costos.
- Auditoría.
- Condiciones de cancelación.
- Respuesta ante fallos.

Los documentos originales deben conservarse. No diseñes cancelaciones mediante borrado físico.

Debes señalar expresamente los casos que requieren aprobación:

1. Cancelar una venta que ya tiene devoluciones parciales.
2. Devolver mercancía de una venta antigua después de que haya cambiado el costo promedio.
3. Cancelar una compra cuando parte de la mercancía ya fue vendida o utilizada por operaciones posteriores.
4. Determinar el efecto sobre el costo promedio cuando se revierten movimientos históricos.

No conviertas las propuestas sobre estos casos en reglas confirmadas sin aprobación.

9. API REST: contratos completos

Diseña los contratos de la API REST de acuerdo con los módulos y los casos de uso aprobados.

Debes especificar cada endpoint que forme parte del alcance. No basta con mostrar unos cuantos ejemplos.

Para cada endpoint incluye:

- Método HTTP.
- Ruta.
- Descripción.
- Rol o permiso requerido.
- Parámetros de ruta.
- Parámetros de consulta.
- DTO de solicitud, si aplica.
- DTO de respuesta.
- Validaciones.
- Códigos HTTP de éxito.
- Códigos HTTP de error relevantes.
- Reglas de paginación, filtrado y ordenamiento, cuando corresponda.
- Efectos sobre el inventario y las transacciones, cuando correspondan.
- Relación con los requisitos de la Etapa 1.

Adopta convenciones consistentes para nombres, pluralización, fechas, estados, paginación y filtros.

No diseñes un envoltorio universal obligatorio para todas las respuestas.

9.1. Manejo de errores

Utiliza ProblemDetail de Spring Framework siguiendo RFC 9457 para los errores de la API.

Especifica:

- Forma de los errores de validación.

- Errores de autenticación.
- Errores de autorización.
- Recursos inexistentes.
- Conflictos de estado.
- Stock insuficiente.
- Restricciones de negocio.
- Cancelaciones no permitidas.
- Conflictos por operaciones concurrentes.
- Errores inesperados.

Distingue entre 401 Unauthorized para solicitudes no autenticadas cuando corresponda y 403 Forbidden para solicitudes autenticadas sin autorización suficiente.

No expongas trazas, contraseñas, datos internos ni mensajes técnicos sensibles en las respuestas.

9.2. DTO

Define los DTO de entrada y salida necesarios para cada endpoint.

Para cada DTO especifica:

- Nombre.
- Propósito.
- Campos.
- Tipos.
- Obligatoriedad.
- Validaciones.
- Campos que no deben ser enviados por el cliente.
- Datos que debe calcular el backend.
- Relaciones o identificadores referenciados.

No permitas que el frontend decida el costo promedio, el saldo final de inventario, los roles del usuario ni otros valores autoritativos del servidor.

En una venta, por ejemplo, el cliente puede enviar referencias y cantidades, pero el backend debe determinar los valores que le corresponden de acuerdo con el catálogo, el precio autorizado y las reglas aprobadas.

9.3. Numeración y comprobantes

Define el contrato para consultar o generar los comprobantes internos y sus consecutivos.

La numeración debe ser consistente ante operaciones concurrentes. Si la política exacta de numeración, su alcance o su reinicio no está definida, regístrala como pendiente.

No implementes una integración fiscal real ni agregues reglas tributarias.

10. Autenticación y seguridad

La autenticación es mediante sesiones de Spring Security. **No utilices JWT como sustituto.**

El diseño debe especificar:

- Flujo de inicio de sesión.
- Flujo de cierre de sesión.
- Validación de la sesión actual.
- Expiración y renovación de la sesión según la configuración seleccionada.
- Invalidación de la sesión al cerrar sesión.
- Almacenamiento seguro de contraseñas mediante un algoritmo apropiado de hashing.
- Protección de endpoints según roles y permisos.
- Gestión inicial y segura del primer administrador.
- Prevención de escalamiento de privilegios.
- Protección frente a CSRF.

- Configuración CORS entre los dos repositorios desplegados por separado.
- Configuración segura de cookies.
- Uso de HTTPS en los entornos correspondientes.
- Respuestas para usuarios no autenticados y usuarios sin permiso.
- Protección de operaciones administrativas y cancelaciones.
- Validación de entrada y tratamiento seguro de errores.
- Manejo de secretos y credenciales de base de datos mediante configuración externa apropiada.

Como React y Spring Boot están en repositorios independientes, explica la interacción de las cookies de sesión, credentials en las solicitudes, CORS y CSRF.

No desactives CSRF indiscriminadamente por tratarse de una API REST. Diseña una estrategia coherente con la autenticación basada en cookies y sesiones.

No inventes credenciales iniciales ni permitas registrar libremente administradores desde el frontend. Especifica un procedimiento seguro de aprovisionamiento inicial y registra cualquier detalle pendiente.

11. Configuración y generación del esquema de base de datos

La decisión actual es utilizar JPA/Hibernate sin Flyway ni Liquibase.

Debes documentar una estrategia práctica para:

- Configuración local de desarrollo.
- Configuración de pruebas.
- Configuración de producción.
- Creación inicial del esquema.
- Protección de datos existentes.
- Manejo de cambios posteriores en las entidades.

- Prevención de alteraciones automáticas peligrosas en entornos con datos persistentes.
- Inicialización de datos de prueba sin contaminar datos reales.

No presentes spring.jpa.hibernate.ddl-auto=update como estrategia suficiente para evolucionar de manera segura un esquema de producción.

Puedes proponer un modo de generación inicial del esquema y una estrategia controlada para cambios posteriores, pero debes explicar sus límites, los pasos operativos y la decisión que falta tomar sobre la evolución del esquema sin herramientas de migración.

No incorpores Flyway o Liquibase por tu cuenta. Si concluyes que mantener el esquema sin una herramienta de migración introduce un riesgo importante, documenta el riesgo y solicita aprobación para cualquier cambio de decisión.

12. Diagramas requeridos

Entrega diagramas legibles y coherentes con los documentos.

Como mínimo:

1. Diagrama de arquitectura general.
2. Diagrama entidad-relación completo.
3. Diagramas de secuencia para:
    - Inicio de sesión.
    - Registro de compra.
    - Registro de venta.
    - Devolución de cliente.
    - Ajuste de inventario.
    - Cancelación de compra.
    - Cancelación de venta.
4. Diagrama de componentes o estructura de módulos cuando aporte claridad.

Utiliza Mermaid o PlantUML en archivos de texto que puedan versionarse.

Los diagramas deben coincidir exactamente con las entidades, paquetes, roles, endpoints y reglas especificadas en los documentos. No incluyas componentes que no aparezcan en el diseño.

Si algún flujo depende de una decisión pendiente, representa el punto de decisión o identifica explícitamente el tramo provisional.

13. Documentación que debes generar

Crea los siguientes documentos en un directorio dedicado a la Etapa 2, por ejemplo docs/etapa-2-diseno/. No sobrescribas los documentos aprobados de la Etapa 1.

Documento 1: 01_diseno_arquitectura.md

Debe contener:

- Objetivo del documento.
- Arquitectura general.
- Tecnologías y decisiones confirmadas.
- Diagrama de arquitectura.
- Responsabilidades por capa.
- Módulos y dependencias.
- Comunicación entre repositorios.
- Límites de responsabilidad.
- Justificación de las decisiones.
- Restricciones y exclusiones.

Documento 2: 02_modelo_entidad_relacion.md

Debe contener:

- Descripción del modelo.
- Diagrama entidad-relación.
- Entidades y relaciones.
- Cardinalidades.

- Restricciones de integridad.
- Reglas de eliminación.
- Relación entre documentos comerciales y movimientos.
- Modelo de inventario y costos.
- Casos que dependen de decisiones pendientes.

Incluye el archivo fuente del diagrama, por ejemplo 02_modelo_entidad_relacion.mmd.

Documento 3: 03_diccionario_datos_mysql.md

Debe contener el diccionario de datos completo con atributos, tipos SQL, nulabilidad, claves, restricciones, índices, relaciones y justificaciones.

No omitas entidades secundarias ni tablas de líneas si el diseño las necesita.

Documento 4: 04_diseno_api_rest.md

Debe contener:

- Convenciones REST.
- Catálogo completo de endpoints.
- DTO de solicitud y respuesta.
- Validaciones.
- Paginación y filtros.
- Códigos HTTP.
- Errores Problem Detail.
- Permisos por endpoint.
- Efectos transaccionales.
- Trazabilidad con los requisitos.

Documento 5: 05_diagramas_secuencia.md

Debe contener los diagramas de los flujos críticos y una explicación breve de cada uno.

Incluye sus archivos fuente cuando resulte práctico.

Documento 6: 06_diseno_frontend.md

Debe contener:

- Estructura de directorios completa.
- Responsabilidad de cada módulo.
- Páginas previstas según los casos de uso aprobados.
- Componentes compartidos.
- Navegación.
- Comunicación con la API.
- Validación de formularios.
- Gestión de errores y estados de carga.
- Manejo de sesiones.
- Comportamiento de la interfaz según permisos.

Documento 7: 07_diseno_seguridad.md

Debe contener:

- Autenticación mediante sesiones.
- Autorización por roles y permisos.
- Protección CSRF.
- CORS y cookies.
- Gestión de contraseñas.
- Aprovisionamiento inicial del administrador.
- Protección de endpoints.
- Riesgos identificados y mitigaciones.

Documento 8: 08_estructura_repositorios.md

Debe describir por separado la estructura del backend y del frontend.

Para cada paquete y directorio indica:

- Responsabilidad.
- Clases o componentes que debe contener.
- Dependencias permitidas.
- Elementos que no deben ubicarse allí.
- Relación con los módulos funcionales.

Debe ser suficientemente concreto para orientar la creación posterior de archivos y clases.

Documento 9: 09_registro_decisiones_tecnicas.md

Incluye las decisiones arquitectónicas y técnicas, su justificación, las alternativas descartadas, las consecuencias y su estado de aprobación.

Como mínimo, registra:

- Arquitectura tradicional por capas.
- Dos repositorios.
- MySQL.
- Autenticación por sesiones.
- REST y DTO.
- Problem Detail conforme a RFC 9457.
- Generación inicial del esquema con Hibernate.
- Ausencia de herramienta de migración.
- Modelo de inventario y costo promedio ponderado.
- Estrategia de concurrencia.
- Estrategia de auditoría.

Documento 10: 10_matriz_trazabilidad_diseno.md

Relaciona los requisitos aprobados de la Etapa 1 con:

- Entidades.

- Atributos y restricciones.
- Casos de uso.
- Servicios.
- Endpoints.
- Pantallas.
- Diagramas.
- Criterios de aceptación.

Conserva los identificadores originales de los requisitos. No inventes identificadores que aparenten proceder de los documentos originales.

Documento 11: 11_riesgos_y_pendientes_diseno.md

Incluye:

- Decisiones pendientes.
- Contradicciones detectadas.
- Supuestos provisionales.
- Riesgos técnicos.
- Riesgos de consistencia de datos.
- Riesgos de seguridad.
- Consecuencias de no resolver cada punto.
- Propuesta de resolución.
- Responsable o instancia que debe aprobar la decisión.
- Impacto sobre los documentos afectados.

14. Metadatos obligatorios de los documentos

Cada documento debe comenzar con una sección de metadatos que incluya:

- Nombre.
- Descripción.
- Fecha de creación.

- Fecha de última actualización.
- Autor.
- Versión.
- Estado: borrador, provisional o aprobado.

Usa fechas reales de generación. No inventes una aprobación que no haya ocurrido. El autor debe identificarse de forma honesta, por ejemplo, como agente de IA, si no se ha indicado otro autor.

Todos los documentos deben usar terminología coherente y referencias cruzadas cuando sea necesario.

15. Normas y calidad del diseño

Utiliza como referencias:

- ISO/IEC/IEEE 29148:2018 para mantener trazabilidad con los requisitos de ingeniería de sistemas y software.
- ISO/IEC 25010:2023 para considerar las características de calidad relevantes para el producto.

No afirmes que el sistema está certificado ni que cumple íntegramente una norma solo por citarla.

Relaciona las decisiones con las características de calidad pertinentes, especialmente:

- Adecuación funcional.
- Fiabilidad.
- Seguridad.
- Mantenibilidad.
- Eficiencia de desempeño.
- Compatibilidad.
- Usabilidad.

Aplica estas características de forma proporcional al alcance académico del sistema. No crees requisitos nuevos sin documentarlos y obtener aprobación.

16. Criterios de aceptación de la Etapa 2

El trabajo se considera listo para revisión cuando:

- La arquitectura coincide con las tecnologías y decisiones confirmadas.
- Las estructuras de paquetes y directorios de backend y frontend están especificadas.
- Las responsabilidades de cada módulo están documentadas.
- Las entidades y tablas están definidas con sus atributos y tipos.
- Las relaciones, cardinalidades y claves foráneas están definidas.
- Las restricciones de integridad y reglas de eliminación están justificadas.
- El inventario mantiene saldo actual e historial persistente.
- El diseño protege frente a ventas concurrentes que podrían generar stock negativo.
- El costo promedio ponderado está especificado.
- El costo histórico de las ventas se conserva.
- Las devoluciones y cancelaciones tienen flujos definidos o decisiones pendientes explícitas.
- La API incluye los contratos de los endpoints dentro del alcance.
- Los DTO, validaciones, permisos y errores están especificados.
- La autenticación se basa en sesiones, no en JWT.
- CORS, CSRF, cookies y cierre de sesión tienen una estrategia coherente.
- El plan de creación del esquema MySQL respeta la decisión de no incorporar una herramienta de migración.
- Los diagramas coinciden con el modelo y los contratos.
- Los requisitos de la Etapa 1 tienen trazabilidad.
- Las decisiones pendientes están identificadas sin presentarlas como aprobadas.

- No se han incorporado funcionalidades fuera del alcance.
- Todos los documentos incluyen los metadatos obligatorios.
- No se ha escrito código de implementación.

17. Restricciones finales de ejecución

Durante esta etapa:

- No implementes clases Java.
- No implementes entidades JPA en código.
- No implementes controladores ni servicios.
- No implementes componentes React.
- No generes un proyecto ejecutable.
- No ejecutes instalaciones ni despliegues.
- No modifiques las decisiones aprobadas de la Etapa 1.
- No sobrescribas documentos previos.
- No conviertas propuestas pendientes en reglas definitivas.
- No agregues tecnologías, módulos o patrones fuera del alcance.

Puedes incluir definiciones técnicas, tablas de atributos, contratos JSON de ejemplo, diagramas y estructuras de directorios porque son artefactos de diseño, no implementación funcional.

Antes de finalizar, verifica la consistencia cruzada entre todos los documentos. Si el diccionario define un atributo, el modelo ER debe representarlo; si un endpoint requiere un permiso, este debe coincidir con la matriz de permisos; si una operación modifica inventario, su secuencia debe coincidir con las reglas de movimientos y costos.

18. Entrega final

Al terminar, presenta un resumen que indique:

1. Los documentos creados y su ubicación.
2. Las decisiones que se han podido especificar completamente.

3. Las decisiones pendientes que requieren aprobación.
4. Las contradicciones o información faltante de la Etapa 1.
5. Los riesgos de diseño más relevantes.
6. Una lista de verificación de los criterios de aceptación.

No comiences la Etapa 3. Detente al finalizar el diseño y espera la revisión y aprobación explícita del usuario.

PROMPT #3:

ETAPA 3 — PLANIFICACIÓN DE LA IMPLEMENTACIÓN Y PREPARACIÓN TÉCNICA DE LOS REPOSITORIOS

1. Rol y objetivo

Actúa como un ingeniero de software senior, arquitecto de software y responsable técnico de un proyecto web empresarial desarrollado con Java, Spring Boot, React y MySQL.

Tu responsabilidad durante esta etapa es preparar técnicamente los dos repositorios del sistema de inventario para una ferretería, establecer un plan detallado de implementación y demostrar mediante evidencias reales que la base técnica funciona.

No debes implementar funcionalidades de negocio durante esta etapa.

Debes trabajar sobre los repositorios existentes, respetar las decisiones aprobadas en las etapas anteriores, evitar la sobreingeniería y mantener trazabilidad de todos los cambios.

El resultado debe ser una base profesional y práctica sobre la que se pueda comenzar posteriormente la implementación funcional del sistema.

2. Contexto tecnológico aprobado

El proyecto utiliza el siguiente stack:

Backend

- Java 21.
- Spring Boot 3.
- Spring MVC.
- Spring Data JPA e Hibernate.
- Spring Security.

- Maven.
- MySQL.
- API REST con JSON.
- OpenAPI/Swagger.
- JUnit, Mockito y pruebas de integración con Spring.

Frontend

- React.
- Material UI.
- Formik y Yup cuando resulten apropiados.
- Enrutamiento del lado del cliente.
- Cliente HTTP centralizado para consumir la API.
- Pruebas con Playwright. Selenium solo si existe una justificación concreta.

Contexto funcional y de arquitectura

- Aplicación web para una ferretería con una sola sede y un único inventario.
- Backend y frontend mantenidos en repositorios separados.
- Arquitectura backend tradicional por capas, de acuerdo con el diseño aprobado.
- Autenticación basada en sesiones de Spring Security, no JWT.
- MySQL como base de datos.
- Moneda COP.
- Contexto colombiano.
- Comprobantes internos y facturación simulada, sin integración real con la DIAN.
- No investigar ni incorporar normativa tributaria.

El contexto anterior no sustituye los documentos aprobados de las etapas previas. Debes consultarlos y verificar sus decisiones antes de modificar los repositorios.

3. Fuente de verdad y jerarquía de decisiones

Los documentos aprobados de la Etapa 1 —levantamiento de requisitos— y la Etapa 2 —diseño técnico y arquitectónico— constituyen la fuente de verdad del proyecto.

Antes de trabajar:

1. Localiza los documentos aprobados y verifica sus versiones y estados.
2. Lee los documentos pertinentes para las tareas que vas a ejecutar.
3. Identifica decisiones aprobadas, decisiones técnicas derivadas, decisiones provisionales, contradicciones y asuntos pendientes.
4. No supongas que un documento está aprobado únicamente porque existe.
5. No inventes requisitos ni afirmes haber consultado documentos que no pudiste leer.

No puedes modificar unilateralmente:

- La arquitectura principal.
- El modelo entidad-relación aprobado.
- Los atributos y relaciones de las entidades de negocio.
- Los contratos de la API REST.
- Las reglas de negocio.
- Los roles y permisos aprobados.
- La estrategia de autenticación por sesiones.
- El alcance funcional.
- Las decisiones explícitamente aprobadas sobre tecnologías y herramientas.

3.1. Documentos ausentes

Si no encuentras los documentos aprobados, puedes continuar con tareas técnicas independientes y reversibles, como inspeccionar los repositorios, evaluar las herramientas instaladas y preparar un plan provisional.

Debes:

- Identificar qué documentos faltan.
- Marcar como provisionales las decisiones dependientes de esos documentos.
- Evitar cambios que requieran conocer decisiones no disponibles.
- No afirmar que la preparación cumple con el diseño aprobado cuando no hayas podido verificarlo.

No debes bloquear tareas independientes por documentos que no sean necesarios para ejecutarlas.

3.2. Contradicciones documentales

Si encuentras contradicciones:

1. Registra los documentos y apartados en conflicto.
2. Explica la contradicción y sus consecuencias técnicas.
3. Propón alternativas cuando resulte útil.
4. Identifica qué tareas quedan bloqueadas.
5. Continúa con los bloques independientes.
6. Solicita aprobación antes de adoptar una solución que afecte decisiones aprobadas.

No resuelvas unilateralmente contradicciones de negocio o de arquitectura principal.

4. Alcance exacto de la Etapa 3

Esta etapa tiene dos objetivos:

1. Elaborar un plan detallado y verificable para la implementación posterior.

2. Preparar técnicamente ambos repositorios y comprobar su funcionamiento básico.

4.1. Actividades permitidas

Puedes:

- Inspeccionar y normalizar los repositorios existentes.
- Crear o ajustar los esqueletos de Spring Boot y React.
- Configurar dependencias compatibles y justificadas.
- Preparar perfiles y variables de entorno.
- Configurar una conexión a MySQL mediante parámetros externos.
- Preparar la estructura de paquetes y módulos aprobada.
- Configurar la infraestructura inicial de pruebas.
- Preparar la configuración base de Spring Security conforme al diseño.
- Configurar una comprobación técnica de salud del backend.
- Preparar el cliente HTTP centralizado del frontend.
- Configurar Material UI y el enrutamiento básico.
- Crear README e instrucciones de ejecución.
- Preparar documentación técnica, decisiones y riesgos.
- Ejecutar compilaciones, pruebas y comprobaciones de integración.
- Incorporar herramientas adicionales cuando exista una justificación técnica concreta.
- Crear commits descriptivos de bloques que hayan superado sus criterios de aceptación.

4.2. Actividades prohibidas

No debes implementar:

- CRUD funcional de productos.
- Gestión funcional de marcas, presentaciones o unidades.

- Clientes o proveedores funcionales.
- Operaciones de compras.
- Operaciones de ventas.
- Gestión funcional de inventario.
- Movimientos de inventario comerciales.
- Cálculo de costos promedio ponderados.
- Devoluciones.
- Ajustes de inventario.
- Gestión funcional de precios.
- Cancelaciones de compras o ventas.
- Generación funcional de comprobantes comerciales.
- Facturación simulada funcional.
- Reportes comerciales.
- Pantallas operativas del sistema.
- Flujos completos de registro, inicio de sesión o administración de usuarios, salvo que el alcance aprobado de la Etapa 2 exija expresamente alguna preparación mínima para verificar la infraestructura.

Tampoco debes crear entidades, tablas, repositorios o servicios de negocio con el argumento de dejar preparada la arquitectura.

Preparar la estructura no significa implementar el sistema funcional.

5. Estrategia de trabajo

Trabaja de forma incremental y secuencial, siguiendo este orden:

1. Inspección y planificación.
2. Preparación del backend.
3. Preparación del frontend.
4. Verificación de integración.

5. Auditoría y cierre.

Cada bloque debe tener:

- Objetivo.
- Dependencias.
- Archivos que se crearán o modificarán.
- Tareas concretas.
- Pruebas y verificaciones.
- Criterios de aceptación.
- Riesgos.
- Evidencias esperadas.
- Condiciones para avanzar.

No avances al siguiente bloque si el actual presenta fallos críticos.

Puedes ejecutar varias tareas consecutivas de forma autónoma si cada bloque cumple sus criterios de aceptación. Detente cuando aparezcan contradicciones relevantes, decisiones pendientes que afecten el trabajo, riesgos de pérdida de datos o fallos críticos.

6. Bloque 1 — Inspección y planificación

Antes de modificar archivos:

1. Identifica la ubicación real de los dos repositorios.
2. Examina la estructura existente.
3. Inspecciona el estado de Git en ambos repositorios.
4. Identifica ramas, cambios sin commit, archivos sin seguimiento y modificaciones preexistentes.
5. Comprueba las versiones disponibles de Java, Maven, Node.js y el gestor de paquetes del frontend.
6. Determina si MySQL está disponible y si puedes conectarte a una base de pruebas autorizada.

7. Localiza los documentos aprobados de las etapas anteriores.
8. Identifica las diferencias entre el estado actual de los repositorios y el diseño aprobado.
9. Registra las decisiones que deban resolverse antes de continuar.

No presupongas que los repositorios están vacíos.

No descartes cambios locales ni ejecutes comandos destructivos para limpiar el entorno.

Entregable del bloque

Un plan detallado de implementación que incluya:

- Orden de los incrementos posteriores.
- Dependencias entre incrementos.
- Módulos afectados.
- Tareas.
- Archivos previstos.
- Estrategia de pruebas.
- Criterios de aceptación.
- Riesgos.
- Decisiones pendientes.
- Condiciones para avanzar.

Distingue entre la preparación técnica que ejecutarás ahora y la implementación funcional que se realizará en etapas posteriores.

7. Bloque 2 — Preparación del backend

7.1. Proyecto y dependencias

Prepara un proyecto compatible con:

- Java 21.
- Spring Boot 3.

- Maven.

Selecciona versiones compatibles y coherentes con las decisiones aprobadas. No actualices versiones principales aprobadas de forma unilateral.

Incluye únicamente las dependencias necesarias para la preparación técnica y las que estén justificadas por el diseño.

Evalúa, cuando corresponda:

- Spring Web.
- Spring Data JPA.
- MySQL Connector/J.
- Spring Security.
- Bean Validation.
- Spring Boot Actuator.
- OpenAPI/Swagger.
- Spring Boot Test.
- Mockito y utilidades de pruebas necesarias.

No incorpores automáticamente todas estas dependencias: comprueba si son necesarias, si ya existen y si están justificadas por los documentos aprobados.

No añadas Flyway ni Liquibase en esta etapa, porque la decisión aprobada actualmente es no incorporarlos por ahora.

7.2. Estructura de paquetes

Respeta la estructura aprobada en la Etapa 2. La referencia acordada es:

com.ferreteria.inventario

Con las áreas principales:

- config
- security
- shared
- auth

- user
- catalog
- customer
- supplier
- inventory
- purchase
- sale
- returns
- document
- reporting

La estructura interna de cada módulo debe seguir el diseño aprobado.

Puedes crear paquetes vacíos, marcadores mínimos o configuraciones necesarias para conservar la estructura, pero no crees clases de negocio ficticias para rellenar paquetes.

Si la inspección revela que la estructura aprobada necesita un ajuste puramente mecánico para funcionar con las herramientas elegidas, documenta la razón y el impacto. No aproveches ese ajuste para rediseñar la arquitectura.

7.3. Configuración por entornos

Prepara una configuración coherente para desarrollo, pruebas y producción.

Utiliza propiedades externas y variables de entorno para los valores que deban variar entre entornos.

Como mínimo, contempla:

- URL JDBC de MySQL.
- Usuario de la base de datos.
- Contraseña.
- Puerto del backend.
- Perfil activo.

- Configuración de CORS conforme al diseño.
- Parámetros de sesión y cookies.
- Parámetros de configuración de seguridad.
- URL base de la API cuando corresponda a la integración con el frontend.

No incluyas credenciales reales, contraseñas personales ni secretos en archivos versionados.

Puedes incluir ejemplos con valores ficticios y archivos de plantilla, siempre que quede claro que deben sustituirse por valores locales.

No publiques configuraciones sensibles ni habilites accesos inseguros para facilitar las pruebas.

7.4. MySQL y esquema

Prepara la configuración para conectarse a MySQL sin depender de credenciales escritas directamente en el código.

No crees tablas funcionales de negocio.

No ejecutes migraciones ni alteraciones sobre bases de datos existentes sin autorización expresa.

No configures Hibernate para crear o modificar automáticamente el esquema de negocio en entornos que puedan contener datos reales.

Para pruebas que requieran base de datos, utiliza un entorno de pruebas controlado y documenta los requisitos previos.

Si no tienes acceso a MySQL, verifica las propiedades y la configuración que sea posible, registra la limitación y proporciona los pasos exactos para comprobar la conexión posteriormente.

7.5. Spring Security

Prepara la infraestructura de seguridad conforme al diseño aprobado:

- Autenticación basada en sesiones.
- Configuración de cookies.
- CORS para el frontend separado.

- Protección CSRF.
- Expiración e invalidación de sesiones.
- Configuración de endpoints públicos y protegidos conforme a los requisitos aprobados.

No sustituyas sesiones por JWT.

No desactives CSRF globalmente solo para evitar problemas de integración.

No utilices configuraciones permisivas que permitan todas las solicitudes como solución definitiva.

No crees un mecanismo ficticio de autenticación ni afirmes que la seguridad está completamente implementada si solo has preparado su estructura.

Si necesitas una configuración temporal para pruebas, debe estar aislada del entorno de producción, identificada explícitamente y no debe convertirse en una solución insegura permanente.

7.6. Comprobación técnica de salud

Prepara una comprobación técnica de disponibilidad del backend.

Puedes utilizar Spring Boot Actuator si resulta apropiado y compatible con el diseño aprobado.

La exposición debe ser mínima y controlada. No expongas públicamente endpoints de configuración, entorno, beans, métricas detalladas ni información interna innecesaria.

La comprobación debe permitir distinguir, como mínimo, si el proceso del backend está disponible.

Si el diseño requiere que el estado de salud también refleje la disponibilidad de MySQL, documenta la diferencia entre disponibilidad del proceso y disponibilidad de sus dependencias.

7.7. Pruebas base

Prepara la infraestructura mínima de pruebas con JUnit, Mockito y las herramientas de Spring que correspondan.

Incluye pruebas iniciales representativas de la infraestructura, por ejemplo:

- Carga del contexto de Spring cuando sea viable.
- Comprobación de una configuración esencial.
- Prueba del endpoint técnico de salud.
- Verificación de que la aplicación puede arrancar con la configuración disponible.

No crees pruebas de negocio ficticias.

No utilices mocks para afirmar que una conexión real a MySQL ha sido verificada.

Criterios de aceptación del backend

- El proyecto compila con Java 21 y Maven.
- Las pruebas que puedan ejecutarse pasan.
- Spring Boot arranca correctamente cuando el entorno lo permite.
- La configuración de MySQL es externa y configurable.
- No existen secretos reales en los archivos versionados.
- La configuración de seguridad respeta el diseño aprobado.
- La comprobación de salud está correctamente configurada y controlada.
- La documentación explica cómo ejecutar y probar el backend.
- No se han implementado funcionalidades de negocio.
- Los bloqueos externos están documentados con claridad.

8. Bloque 3 — Preparación del frontend

8.1. Proyecto y dependencias

Prepara el frontend con React y Material UI.

Selecciona versiones compatibles entre sí y con las herramientas disponibles. Respeta las versiones ya aprobadas, cuando existan.

Utiliza el gestor de paquetes que corresponda al repositorio existente. Si el repositorio no tiene uno establecido, selecciona una opción apropiada y documenta la decisión.

Mantén el archivo de bloqueo de dependencias correspondiente al gestor utilizado para favorecer instalaciones reproducibles.

8.2. Estructura

Respeta la estructura aprobada en la Etapa 2. La referencia acordada incluye:

- src/app/router
- src/app/providers
- src/app/theme
- src/components/layout
- src/components/feedback
- src/components/forms
- src/components/tables
- src/components/common
- src/features/auth
- src/features/users
- src/features/catalog
- src/features/customers
- src/features/suppliers
- src/features/inventory
- src/features/purchases
- src/features/sales
- src/features/returns
- src/features/documents
- src/features/reports
- src/services/api
- Directorios para hooks, utilidades y pruebas conforme al diseño aprobado.

No crees pantallas funcionales de inventario, ventas, compras u otros módulos de negocio.

Puedes preparar el enrutamiento técnico, el tema global, los proveedores de contexto necesarios y una pantalla inicial neutra que permita comprobar que la aplicación arranca.

8.3. Configuración de API

Prepara un cliente HTTP centralizado para consumir el backend.

La URL base debe poder configurarse mediante variables de entorno.

La configuración debe ser coherente con:

- Backend alojado por separado.
- Sesiones mediante cookies.
- CORS.
- CSRF.
- Tratamiento básico de errores de conectividad.
- Entornos de desarrollo y producción.

No implementes servicios de negocio ni endpoints ficticios para simular operaciones comerciales.

8.4. Formularios y validación

Formik y Yup pueden incluirse cuando exista un formulario técnico que los justifique o cuando la preparación acordada los requiera.

No crees formularios de registro, productos, compras o ventas durante esta etapa.

8.5. Pruebas y build

Prepara la infraestructura de pruebas del frontend de acuerdo con el diseño aprobado.

Utiliza Playwright cuando corresponda. No incorpores Selenium salvo que exista una justificación concreta.

Como mínimo, verifica:

- Instalación reproducible de dependencias.
- Compilación o build de producción.
- Funcionamiento de la aplicación base.
- Pruebas iniciales disponibles.

Criterios de aceptación del frontend

- Las dependencias se instalan correctamente.
- El build de producción termina sin errores.
- La estructura modular está preparada.
- La URL base de la API es configurable.
- No se han implementado pantallas ni funcionalidades comerciales.
- La documentación permite ejecutar el proyecto.
- Los resultados de las pruebas están registrados.

9. Bloque 4 — Verificación de integración

Una vez que el backend y el frontend estén preparados individualmente, verifica su coherencia.

No es necesario implementar flujos comerciales para esta comprobación.

9.1. Comprobaciones mínimas

Verifica, según las capacidades reales del entorno:

1. El backend puede arrancar.
2. La comprobación técnica de salud responde correctamente.
3. El frontend puede ejecutarse.
4. El frontend utiliza la URL base de API configurada.
5. La configuración de CORS coincide con los orígenes autorizados.
6. La estrategia de cookies y sesiones es coherente entre ambos proyectos.
7. La protección CSRF está contemplada de acuerdo con el diseño.
8. Los errores de conectividad no se presentan como operaciones exitosas.

9. Las variables de entorno y los puertos están documentados.
10. La configuración no requiere secretos incorporados al código fuente.

Cuando el entorno permita una comprobación real entre frontend y backend, ejecútala y registra el resultado.

Si una política de seguridad impide una solicitud, investiga la causa y corrige únicamente los problemas compatibles con las decisiones aprobadas.

No desactives medidas de seguridad de manera global para conseguir que la prueba pase.

9.2. Límites de la comprobación

Distingue claramente entre:

- Backend compilado.
- Backend arrancado.
- Endpoint de salud comprobado.
- Conexión a MySQL comprobada.
- Frontend compilado.
- Frontend ejecutado.
- Solicitud entre frontend y backend comprobada.
- Comportamiento de sesión y CSRF probado.

Son comprobaciones distintas. El éxito de una no demuestra automáticamente las demás.

Criterios de aceptación de integración

- La configuración de ambos proyectos es coherente.
- Las comprobaciones posibles se ejecutaron y sus resultados se registraron.
- Los errores encontrados se corrigieron o quedaron documentados.
- No existen fallos críticos sin resolver.

- Las comprobaciones imposibilitadas por el entorno están identificadas como pendientes.
- No se implementaron funcionalidades de negocio.

10. Herramientas adicionales y autonomía técnica

Puedes incorporar herramientas adicionales cuando aporten un beneficio concreto para reproducibilidad, calidad, pruebas o ejecución.

Evalúa cada incorporación considerando:

- Problema que resuelve.
- Beneficio esperado.
- Coste de configuración y mantenimiento.
- Alternativas disponibles.
- Compatibilidad con el stack.
- Impacto en los dos repositorios.
- Riesgos o complejidad añadida.

Docker Compose, formateadores, análisis estático u otras herramientas pueden considerarse si están justificadas. No debes incorporarlas automáticamente.

No añadas Kafka, Redis, Kubernetes, microservicios, CQRS, Event Sourcing ni otras tecnologías de infraestructura avanzada sin una necesidad aprobada y demostrable.

Registro de decisiones

Para las decisiones técnicas nuevas, registra:

- Identificador.
- Decisión.
- Contexto.
- Problema que resuelve.
- Alternativas consideradas.
- Justificación.

- Consecuencias positivas y negativas.
- Archivos afectados.
- Riesgos.
- Estado: propuesta, provisional o adoptada.

Puedes resolver decisiones técnicas menores y estructurales con autonomía, pero no cambiar decisiones de arquitectura principal o negocio.

Solicita aprobación antes de adoptar cualquier cambio que afecte las decisiones protegidas de los documentos anteriores.

11. Gestión de los repositorios y Git

Debes inspeccionar y normalizar los repositorios existentes sin destruir trabajo previo.

Antes de modificar:

- Revisa git status.
- Inspecciona los cambios locales.
- Identifica archivos sin seguimiento.
- Comprueba la rama activa.
- Identifica configuraciones y documentación existentes.

No elimines ni sobrescribas cambios previos sin autorización.

Si necesitas modificar una configuración existente, documenta:

- Estado inicial.
- Cambio aplicado.
- Motivo.
- Impacto.
- Forma de recuperación o reversión.

Commits por bloque

Al terminar cada bloque:

1. Ejecuta las comprobaciones definidas.
2. Revisa los resultados.
3. Comprueba los criterios de aceptación.
4. Revisa el diff completo.
5. Verifica que no se incluyan secretos, archivos temporales ni cambios ajenos.
6. Comprueba que no se hayan incorporado accidentalmente modificaciones preexistentes.
7. Crea un commit descriptivo únicamente si el bloque ha superado sus criterios.

Utiliza mensajes claros que identifiquen el propósito del bloque.

No hagas push automático.

Si un bloque presenta un fallo crítico, no crees un commit que lo presente como completado. Puedes conservar cambios de trabajo cuando sea necesario para evitar pérdidas, pero debes distinguirlos de los bloques aprobados y no marcarlos como finalizados.

No ejecutes comandos destructivos de Git para forzar un estado limpio.

12. Documentación y entregables

Cada repositorio debe conservar su propio README e instrucciones técnicas.

Si ya existe una ubicación común para la documentación del proyecto, úsala para centralizar el plan general, decisiones y evidencias. No crees un tercer repositorio exclusivamente para documentar esta etapa.

Si no existe una ubicación común adecuada, conserva los entregables en los lugares disponibles y documenta dónde están.

Entregables mínimos

1. **Plan detallado de implementación:** incrementos futuros, dependencias, tareas, pruebas y criterios de aceptación.
2. **Backend preparado:** estructura, dependencias, perfiles, configuración externa, seguridad base y pruebas.

3. **Frontend preparado:** estructura, tema, enrutamiento, configuración de API y pruebas.
4. **Verificación de integración:** resultados reales y limitaciones.
5. **README del backend:** requisitos, configuración, ejecución, pruebas y solución de problemas comunes.
6. **README del frontend:** requisitos, instalación, configuración, ejecución y pruebas.
7. **Registro de decisiones técnicas:** decisiones adoptadas, provisionales y pendientes.
8. **Registro de riesgos y bloqueos:** causa, impacto, tareas afectadas y siguiente acción.
9. **Informe de evidencias:** comandos ejecutados, resultados, fallos y verificaciones no realizadas.
10. **Resumen de Git:** commits creados y estado final de cada repositorio.

Cada documento formal debe incluir metadatos apropiados:

- Nombre.
- Descripción.
- Fecha de creación.
- Fecha de actualización.
- Autor o responsable.
- Versión.
- Estado del documento.

No inventes autores ni fechas de aprobaciones anteriores.

Conserva los documentos de requisitos y diseño aprobados. No los reescribas para que coincidan con decisiones nuevas.

13. Gestión de fallos y pendientes

Distingue tres estados:

A. Verificado correctamente

La comprobación se ejecutó y produjo el resultado esperado. Registra evidencia suficiente para respaldarlo.

B. Falló

La comprobación se ejecutó y detectó un defecto. Debes corregirlo o mantener el bloque como incompleto.

Un fallo crítico impide considerar completado el bloque afectado.

C. No verificable por limitaciones del entorno

No fue posible ejecutar la comprobación por falta de acceso, herramientas, servicios o permisos.

Registra:

- Verificación pendiente.
- Motivo concreto.
- Consecuencia.
- Archivos o configuraciones preparados.
- Pasos necesarios para verificar posteriormente.
- Resultado esperado.

No presentes una comprobación pendiente como exitosa.

Condiciones de cierre

Puedes cerrar el trabajo ejecutable de la etapa con verificaciones externas pendientes únicamente cuando:

- Los criterios esenciales que sí podían ejecutarse se hayan cumplido.
- No existan fallos críticos conocidos.
- Las limitaciones estén identificadas expresamente.
- Se documenten las instrucciones para completar las verificaciones.
- El informe distinga entre preparación realizada y funcionamiento no comprobado.

Si existe un fallo crítico, el estado final debe ser incompleto.

14. Criterios globales de aceptación de la Etapa 3

La etapa se considera completada cuando:

- Existe un plan detallado y verificable de implementación posterior.
- El backend compila y sus pruebas ejecutables pasan.
- El frontend instala sus dependencias y genera el build.
- Ambos proyectos tienen instrucciones de ejecución.
- La configuración de MySQL está preparada y su estado de verificación está documentado.
- La seguridad base respeta el diseño aprobado.
- La comprobación de salud está controlada.
- La integración se verificó en la medida permitida por el entorno.
- No se implementaron funcionalidades de negocio.
- Los documentos aprobados no se modificaron unilateralmente.
- Las decisiones técnicas nuevas están justificadas y registradas.
- Los cambios son recuperables y están documentados.
- Los commits reflejan únicamente bloques que superaron sus criterios.
- No se hizo push automático.
- No existen fallos críticos ocultos.
- Los pendientes y limitaciones se reportaron de manera explícita.

Si alguno de los criterios esenciales no se cumple, no declares la etapa completamente finalizada.

15. Informe final obligatorio

Al finalizar, presenta un informe ejecutivo con estas secciones:

1. **Estado general:** completada, incompleta o completada con verificaciones externas pendientes.
2. **Resumen de cambios:** qué se preparó en cada repositorio.

3. **Plan de implementación:** ubicación y resumen de los incrementos futuros.
4. **Pruebas ejecutadas:** comando, resultado y evidencia relevante.
5. **Pruebas fallidas:** causa, impacto y corrección pendiente.
6. **Pruebas no ejecutadas:** motivo y pasos para realizarlas.
7. **Decisiones técnicas:** nuevas decisiones adoptadas, con sus justificaciones.
8. **Documentación generada o actualizada:** rutas reales.
9. **Git:** commits creados y estado final de ambos repositorios.
10. **Pendientes y riesgos:** prioridad y siguiente acción.
11. **Confirmación de alcance:** declaración explícita de que no se implementaron funcionalidades de negocio.

No inventes rutas, resultados de comandos, commits, pruebas ni archivos. Utiliza las ubicaciones reales encontradas durante la ejecución.

16. Instrucción final

Empieza por inspeccionar el entorno y los repositorios. No comiences modificando archivos sin conocer su estado.

Trabaja por bloques, verifica cada resultado y documenta tus decisiones.

Resuelve con autonomía los problemas técnicos menores y las decisiones estructurales compatibles con el diseño aprobado. Solicita aprobación ante cambios de arquitectura principal, reglas de negocio o decisiones protegidas.

Si faltan documentos, avanza provisionalmente solo en tareas independientes. Si existen contradicciones, detén el trabajo afectado y continúa con el resto cuando sea seguro.

Tu objetivo no es construir el sistema de inventario todavía. Tu objetivo es dejar ambos repositorios técnicamente preparados, verificables, documentados y listos para comenzar la implementación funcional en la siguiente etapa.

PROMPT #4

ETAPA 4 — IMPLEMENTACIÓN COMPLETA, INTEGRACIÓN Y ENTREGA DEL SISTEMA DE INVENTARIO PARA FERRETERÍA

1. Rol y responsabilidad

Actúa como un equipo de ingeniería de software senior compuesto por arquitectos de software, desarrolladores backend Java/Spring Boot, desarrolladores frontend React, especialistas en bases de datos MySQL, seguridad de aplicaciones y pruebas de software.

Tu responsabilidad es implementar, integrar, probar, documentar y entregar el sistema web de inventario para una ferretería, utilizando los repositorios existentes.

Esta etapa tiene como objetivo terminar el desarrollo funcional del sistema definido en las etapas anteriores. No debes limitarte a preparar estructuras, crear esqueletos, producir documentación o implementar un prototipo.

El resultado esperado es una aplicación funcional que permita ejecutar las operaciones de negocio aprobadas, persistir los datos correctamente, controlar los permisos, mantener la consistencia del inventario y ofrecer una interfaz utilizable.

Debes trabajar de forma autónoma dentro de los límites establecidos en este documento. Resuelve problemas técnicos, implementa las funcionalidades y corrige los defectos que encuentres. No solicites aprobación para cada clase, archivo o decisión técnica menor.

Sin embargo, no puedes cambiar unilateralmente las reglas de negocio, el modelo de datos aprobado, los contratos protegidos de la API ni la arquitectura principal.

2. Contexto del proyecto

El sistema corresponde a una ferretería con una sola sede y un único inventario.

2.1. Tecnologías aprobadas

Backend

- Java 21.
- Spring Boot 3.
- Spring MVC.
- Spring Data JPA e Hibernate.
- Spring Security.
- Maven.
- MySQL.
- API REST con JSON.
- OpenAPI/Swagger.
- JUnit, Mockito y pruebas de integración con Spring.

Frontend

- React.
- Material UI.
- Formik y Yup cuando corresponda.
- Enrutamiento del lado del cliente.
- Cliente HTTP centralizado.
- Playwright para pruebas de flujos de usuario.

Persistencia

- MySQL como base de datos.
- Scripts SQL versionados, revisados y aplicados explícitamente.
- No utilizar Flyway ni Liquibase durante esta etapa.
- No depender de la creación o modificación automática del esquema mediante Hibernate.

Seguridad

- Autenticación mediante sesiones de Spring Security.
- No utilizar JWT como sustituto de las sesiones aprobadas.
- Control de acceso basado en los roles definidos en los documentos del proyecto.

Arquitectura

- Arquitectura backend tradicional por capas: Controller → Service → Repository.
- Organización modular de acuerdo con la Etapa 2.
- Repositorios independientes para backend y frontend.

No cambies estas tecnologías sin una razón imprescindible y documentada. Las decisiones que alteren el stack o la arquitectura principal requieren aprobación.

3. Jerarquía de documentos y decisiones

Los documentos aprobados de la Etapa 1 —requisitos— y de la Etapa 2 —diseño técnico y arquitectónico— son la fuente de verdad funcional y estructural.

El resultado real de la Etapa 3 es la referencia para el estado de los repositorios, la configuración inicial y las decisiones técnicas ya materializadas.

Antes de implementar funcionalidades, debes:

1. Localizar los documentos aprobados y comprobar sus versiones y estados.
2. Leer los requisitos, reglas de negocio, casos de uso, matriz de permisos, criterios de aceptación y riesgos.
3. Revisar el diseño de arquitectura, modelo entidad-relación, diccionario de datos, contratos REST, diagramas, seguridad y estructura de repositorios.
4. Revisar el código y la configuración que ya existen.
5. Comparar los documentos con el estado real del código.

6. Identificar las funcionalidades pendientes y las contradicciones relevantes.
7. Elaborar un plan de implementación que contemple dependencias, pruebas y criterios de aceptación.

No debes asumir que una funcionalidad está terminada porque existen clases, interfaces, controladores o pantallas con nombres relacionados.

Tampoco debes reemplazar implementaciones existentes sin comprobar primero su propósito y estado.

3.1. Si faltan documentos o existen contradicciones

Si falta un documento, continúa con las tareas independientes que puedan ejecutarse sin inventar decisiones.

Si existe una contradicción:

- Identifica los documentos y apartados involucrados.
- Explica la consecuencia técnica o funcional.
- Propón alternativas cuando sea útil.
- Detén únicamente el trabajo que dependa de la decisión.
- Continúa con tareas independientes.
- Solicita aprobación si resolverla implica cambiar una decisión de negocio, un contrato protegido o la arquitectura principal.

No inventes reglas para conseguir que las pruebas pasen.

4. Alcance funcional obligatorio

Debes implementar todas las funcionalidades aprobadas para el sistema, incluyendo las siguientes áreas.

4.1. Autenticación, usuarios y permisos

Implementar:

- Inicio de sesión.
- Cierre de sesión.
- Gestión de sesiones.

- Manejo de expiración e invalidación de sesión.
- Gestión de usuarios según los requisitos aprobados.
- Asignación y validación de roles y permisos.
- Inicialización segura del primer administrador.
- Protección de los endpoints.
- Manejo de accesos no autorizados.
- Validaciones de seguridad necesarias.

Roles definidos en el proyecto:

- ADMINISTRADOR
- USUARIO_OPERATIVO

Debes respetar la matriz de permisos aprobada. No deduzcas permisos nuevos por conveniencia.

La interfaz puede ocultar acciones que el usuario no tiene autorizadas, pero la autorización debe imponerse en el backend.

No utilices cuentas predeterminadas con contraseñas conocidas, secretos versionados ni mecanismos de inicialización que permitan crear administradores sin control.

4.2. Catálogo de productos

Implementar el catálogo conforme al modelo aprobado, incluyendo los conceptos aplicables de:

- Productos.
- Marcas.
- Presentaciones.
- Referencias comercializables.
- Unidades base.
- Conversiones de unidades.
- Fracciones permitidas.

- Códigos de barras.
- Precios de venta.
- Activación y desactivación de productos.
- Consultas, filtros, ordenamiento y paginación cuando estén definidos.

La referencia comercializable debe respetar la regla aprobada de producto, marca y presentación.

Los códigos de barras deben cumplir la unicidad definida por el modelo.

Los productos que tengan historial no se deben eliminar físicamente cuando eso comprometa la integridad histórica. Debe utilizarse el mecanismo de desactivación aprobado.

Los cambios de precio deben conservar la auditoría requerida.

4.3. Clientes y proveedores

Implementar:

- Registro.
- Consulta.
- Edición.
- Validación de datos.
- Activación o desactivación cuando esté contemplada en el diseño.
- Búsqueda y paginación según los contratos aprobados.

Los clientes y proveedores deben estar registrados antes de operar con ellos.

Una misma entidad podrá actuar como cliente y proveedor si así lo establece el modelo aprobado. No dupliques personas o empresas únicamente por desempeñar ambos papeles.

Toda venta debe estar asociada a un cliente registrado.

4.4. Núcleo del inventario

Implementar:

- Consulta de existencias.

- Registro persistente de movimientos.
- Actualización transaccional de los saldos.
- Validación de cantidades.
- Conversión de unidades.
- Control de precisión decimal.
- Costos asociados al inventario.
- Trazabilidad de los movimientos.
- Protección contra existencias negativas.
- Protección frente a operaciones concurrentes.
- Auditoría de operaciones.

El inventario debe mantener un saldo actual coherente con los movimientos que lo modifican.

No se debe permitir que una operación deje actualizados los movimientos, pero no los saldos, o viceversa.

Las reglas de transacción y concurrencia deben implementarse realmente, no limitarse a comprobar el stock antes de iniciar una operación.

Utiliza el mecanismo de bloqueo, control de versiones o serialización que corresponda al diseño y que demuestre mantener la consistencia de los datos.

4.5. Inventario inicial

Implementar la inicialización de existencias mediante el movimiento específico aprobado.

Debe permitir registrar las cantidades iniciales y el costo de referencia conforme al diseño.

No simules una compra para representar el inventario inicial.

Debe existir control sobre las condiciones en que se permite inicializar o modificar la carga inicial, con trazabilidad de quién realizó la operación.

4.6. Compras

Implementar:

- Registro de compras.
- Selección de proveedor.
- Varias líneas por compra.
- Cantidades y costos unitarios.
- Cálculo de subtotales y totales.
- Validaciones de datos.
- Recepción completa en una operación.
- Actualización de existencias.
- Actualización del costo promedio ponderado.
- Registro de movimientos.
- Auditoría.
- Consulta del historial.

No implementar recepciones parciales.

La compra, sus líneas, los movimientos y los cambios de inventario deben mantener la consistencia transaccional establecida.

4.7. Ventas de contado

Implementar:

- Registro de ventas.
- Asociación obligatoria con un cliente registrado.
- Selección de productos.
- Varias líneas por venta.
- Validación de cantidades y precios.
- Cálculo de subtotales y totales.
- Registro del pago en efectivo.
- Descuento transaccional de existencias.
- Conservación del costo histórico de cada línea.

- Registro de movimientos.
- Auditoría.
- Consulta de ventas.

No implementar ventas a crédito.

Una venta no debe completarse si no puede satisfacer las reglas de stock, persistencia, precios y autorización aprobadas.

Las operaciones concurrentes no deben provocar que el inventario disponible sea negativo.

Los precios y costos históricos deben conservarse de forma que una modificación posterior del catálogo no altere las operaciones ya registradas.

4.8. Devoluciones de clientes

Implementar las modalidades aprobadas:

- Devolución vinculada a una venta original.
- Devolución no vinculada, cuando esté permitida por el diseño.
- Registro de líneas y cantidades.
- Validaciones.
- Registro de movimientos.
- Reintegro de existencias aceptadas.
- Auditoría.

Las devoluciones vinculadas deben respetar el costo histórico de la venta original.

Las devoluciones no vinculadas deben aplicar la regla aprobada, que contempla el costo promedio vigente cuando corresponda.

No permitas devolver cantidades superiores a las permitidas por las reglas aprobadas ni procesar repetidamente una devolución que viole sus restricciones.

4.9. Ajustes y cancelaciones

Implementar los ajustes de inventario y las cancelaciones de compras y ventas conforme a la matriz de permisos.

Cuando corresponda, exigir:

- Usuario autorizado.
- Motivo obligatorio.
- Validaciones previas.
- Registro de auditoría.
- Conservación del documento original.
- Movimientos compensatorios.
- Reglas que impidan operaciones que comprometan la consistencia histórica.

No borres físicamente las operaciones comerciales para simular una cancelación.

Si una operación posterior hace insegura una reversión, el sistema debe aplicar la restricción aprobada y explicar el motivo.

Antes de implementar este bloque, verifica expresamente las reglas sobre:

- Cancelar ventas con devoluciones previas.
- Revertir compras cuando las existencias ya se consumieron.
- Recalcular o compensar correctamente el costo promedio.
- Evitar duplicar movimientos al cancelar.
- Mantener la trazabilidad de los documentos y operaciones relacionadas.

Si alguna regla no está definida en los documentos aprobados, registra la decisión pendiente y solicita autorización antes de implementar el comportamiento afectado.

4.10. Comprobantes y documentos internos

Implementar:

- Numeración consecutiva interna.

- Documentos asociados a las operaciones correspondientes.
- Estados y relaciones definidos por el diseño.
- Detalle de líneas y totales.
- Representación imprimible.
- Conservación del historial.

No implementar integración real con la DIAN ni asumir obligaciones tributarias que no formen parte del alcance aprobado.

La facturación contemplada es simulada y los documentos son internos.

4.11. Consultas y reportes

Implementar las consultas y reportes definidos en los requisitos aprobados.

Como mínimo, revisar los requerimientos de:

- Existencias actuales.
- Historial de movimientos.
- Compras.
- Ventas.
- Productos y precios.
- Costos.
- Devoluciones.
- Ajustes y cancelaciones.
- Auditoría cuando corresponda.

Respetar filtros, ordenamiento, paginación y restricciones de permisos.

Evitar cargar todos los registros innecesariamente o introducir problemas N+1 que perjudiquen de forma relevante el rendimiento.

No agregar reportes no solicitados únicamente para aumentar el alcance.

5. Arquitectura y calidad del backend

Respeta la arquitectura por capas aprobada:

Controller → Service → Repository

Utiliza la estructura modular definida en la Etapa 2.

Reglas de implementación

- Mantener los controladores delgados.
- Concentrar las reglas de negocio en los servicios correspondientes.
- Utilizar DTO para las entradas y salidas de la API.
- Aplicar Bean Validation.
- Evitar exponer directamente entidades JPA.
- Usar inyección por constructor.
- Evitar dependencias de Spring Data JPA en capas que no deban conocer la persistencia.
- Evitar ResponseEntity como tipo de retorno de servicios de negocio.
- Centralizar el manejo de errores mediante el mecanismo aprobado, incluyendo @RestControllerAdvice y ProblemDetail.
- Utilizar transacciones donde sean necesarias.
- Evitar duplicación de reglas de negocio.
- Mantener nombres, responsabilidades y dependencias comprensibles.
- No introducir abstracciones genéricas sin una necesidad demostrable.

No conviertas el proyecto en microservicios ni introduzcas CQRS, Event Sourcing, Kafka, Redis o Kubernetes sin una necesidad aprobada.

No añadas patrones arquitectónicos por moda ni refactorices módulos completos sin una justificación vinculada a un defecto o requisito.

6. Gestión de MySQL y scripts SQL

La estrategia aprobada es utilizar scripts SQL versionados y revisados, sin Flyway ni Liquibase.

Debes implementar el esquema necesario para todas las funcionalidades, respetando el modelo aprobado.

6.1. Procedimiento obligatorio

1. Inspeccionar el esquema existente.
2. Identificar el cambio necesario.
3. Crear un script versionado y descriptivo.
4. Revisar tipos, restricciones, relaciones e índices.
5. Comprobar el impacto sobre datos existentes.
6. Aplicar el script explícitamente en una base de datos de desarrollo o pruebas autorizada.
7. Verificar el resultado.
8. Actualizar la documentación.
9. Registrar qué scripts se aplicaron y en qué entorno.

Establece una convención sencilla para evitar ejecutar dos veces scripts que no sean idempotentes y detectar discrepancias entre el esquema esperado y el real.

No ejecutes scripts automáticamente durante el arranque de Spring Boot.

No elimines tablas ni datos existentes para simplificar la implementación.

No utilices ddl-auto=update como mecanismo de evolución del esquema. Configura Hibernate de forma que no altere automáticamente esquemas que puedan contener datos importantes.

6.2. Seguridad de los datos

- Utilizar una base de pruebas aislada.
- No incluir contraseñas reales en Git.
- No ejecutar operaciones destructivas sobre bases de datos cuya naturaleza no esté confirmada.
- Documentar copias de seguridad y recuperación cuando corresponda.
- No afirmar que se verificó MySQL real si solamente se utilizaron mocks.

7. Seguridad funcional

Implementa y prueba la seguridad del sistema de extremo a extremo.

Obligaciones

- Autenticación basada en sesiones.
- Cierre de sesión e invalidación de sesión.
- Expiración de sesiones.
- Configuración correcta de cookies.
- CORS limitado a los orígenes necesarios.
- Protección CSRF compatible con la aplicación.
- Contraseñas almacenadas mediante un algoritmo de hash apropiado.
- Autorización por roles y operaciones.
- Validación de entradas.
- Tratamiento seguro de errores.
- Protección de endpoints y datos.
- Configuración segura de secretos.
- Inicialización controlada del primer administrador.

Prueba los escenarios de acceso autorizado y no autorizado, incluidos los intentos de un usuario operativo de ejecutar funciones administrativas.

No confíes en que el frontend oculte botones para proteger una operación.

No desactives CSRF globalmente ni permitas todos los orígenes como solución permanente.

No expongas contraseñas, tokens de sesión, secretos, datos internos de configuración ni información sensible en mensajes de error o logs.

8. Contratos REST y documentación de API

Implementa los contratos de la Etapa 2.

Si se detectan defectos técnicos, puedes corregirlos cuando no alteren el comportamiento aprobado. Si es necesario modificar un contrato protegido, debes documentar la incompatibilidad y solicitar aprobación.

La documentación OpenAPI debe coincidir con los endpoints implementados.

Para cada endpoint relevante deben estar definidos:

- Método y ruta.
- Parámetros.
- DTO de entrada y salida.
- Validaciones.
- Códigos HTTP.
- Errores.
- Permisos.
- Comportamiento de paginación cuando aplique.

Al finalizar el backend, sus contratos deben ser suficientes para desarrollar el frontend sin inventar parámetros, campos, códigos de error o comportamientos.

9. Pruebas obligatorias del backend

Utiliza JUnit, Mockito y las herramientas de Spring que correspondan.

9.1. Pruebas unitarias

Cubrir las reglas de negocio relevantes, incluidos:

- Validaciones.
- Cálculos.
- Restricciones.
- Transiciones de estado.
- Casos de error.
- Permisos que puedan comprobarse en la capa correspondiente.

9.2. Pruebas de integración

Utiliza MySQL real en un entorno de pruebas controlado para comprobar persistencia, transacciones y comportamiento específico del motor.

No dependas exclusivamente de H2 para validar las características de MySQL.

Las pruebas deben comprobar, según corresponda:

- Repositorios y consultas.
- Persistencia de relaciones.
- Restricciones de la base de datos.
- Transacciones.
- Rollback.
- Manejo de excepciones.
- Integración con Spring Security.
- Endpoints REST.
- Contratos de respuesta.

9.3. Escenarios críticos

Debes comprobar, como mínimo:

1. Venta con stock suficiente.
2. Venta rechazada por stock insuficiente.
3. Dos ventas concurrentes sobre las mismas existencias.
4. Compra que actualiza correctamente el inventario.
5. Cálculo del costo promedio ponderado.
6. Fallo durante una operación y rollback de sus cambios.
7. Devolución vinculada a una venta.
8. Restricciones de devolución.
9. Cancelación permitida y cancelación rechazada.
10. Preservación del historial.
11. Acceso administrativo y acceso operativo.
12. Rechazo de solicitudes no autorizadas.

13. Manejo de sesión expirada.
14. Protección CSRF.
15. Validación de entradas incorrectas.

No es necesario alcanzar un porcentaje arbitrario de cobertura. Es obligatorio cubrir las reglas críticas y los escenarios de fallo relevantes.

10. Criterios de aceptación del backend

El backend solo se considerará completo cuando:

- Todos los módulos de negocio incluidos en el alcance estén implementados.
- La persistencia sea real.
- El esquema y sus scripts estén documentados.
- Las operaciones críticas mantengan la consistencia transaccional.
- Los permisos se apliquen realmente.
- Los contratos REST estén implementados y documentados.
- Las pruebas críticas pasen.
- La compilación se complete correctamente.
- La aplicación pueda arrancar con una configuración válida.
- Las verificaciones realizadas tengan evidencia.
- No queden fallos críticos conocidos sin resolver.

No comiences el desarrollo funcional del frontend hasta que el backend haya superado estos criterios, salvo que exista una excepción expresamente documentada y aprobada.

11. Implementación completa del frontend

Una vez aceptado el backend, implementa la interfaz completa en React y Material UI.

No te limites a crear una página de inicio o una colección de componentes vacíos.

11.1. Estructura y navegación

Respeta la estructura modular aprobada, incluyendo las áreas de autenticación, usuarios, catálogo, clientes, proveedores, inventario, compras, ventas, devoluciones, documentos y reportes.

Implementa:

- Navegación funcional.
- Rutas.
- Layout común.
- Tema global.
- Componentes reutilizables cuando aporten valor.
- Manejo de sesión.
- Manejo de errores.
- Estados de carga y vacío.
- Mensajes de confirmación y validación.

11.2. Pantallas funcionales

Implementa las pantallas necesarias para ejecutar las funcionalidades del alcance aprobado.

Las operaciones de consulta, creación, edición, desactivación, compras, ventas, devoluciones, ajustes, cancelaciones y reportes deben consumir la API real cuando correspondan a las reglas aprobadas.

No utilices datos ficticios para aparentar que una operación fue persistida.

11.3. Formularios y tablas

- Utiliza Formik y Yup cuando resulten apropiados.
- Reutiliza componentes de formulario y tabla cuando exista una ventaja real.
- Valida entradas antes de enviarlas.
- Muestra errores devueltos por el backend.

- Mantén las validaciones de negocio definitivas en el servidor.
- Implementa filtros y paginación de acuerdo con los contratos REST.
- Evita duplicar reglas de negocio complejas en el frontend.

11.4. Experiencia de usuario

La interfaz debe ser profesional, consistente y práctica.

Debe contemplar:

- Adaptación a diferentes tamaños de pantalla.
- Etiquetas comprensibles.
- Navegación clara.
- Confirmaciones para operaciones sensibles.
- Estados de carga.
- Mensajes de error útiles.
- Estados vacíos.
- Prevención de envíos duplicados.
- Formato coherente de cantidades, moneda COP y fechas.
- Accesibilidad básica de formularios y controles.

No agregues animaciones, complejidad visual ni dependencias innecesarias.

11.5. Seguridad de la interfaz

- Gestionar correctamente las sesiones.
- Enviar cookies de sesión según la configuración acordada.
- Integrar la protección CSRF.
- Manejar respuestas de sesión expirada.
- Mostrar acciones según los permisos conocidos del usuario.
- Tratar los errores 401 y 403 de forma coherente con los contratos.
- No almacenar credenciales ni secretos innecesarios en el navegador.

El frontend no debe asumir que ocultar una acción reemplaza la autorización del backend.

12. Pruebas del frontend

Utiliza Playwright para probar los flujos principales.

Las pruebas deben cubrir, según corresponda:

- Acceso y cierre de sesión.
- Navegación.
- Restricciones visuales por rol.
- Formularios y validaciones.
- Consultas y paginación.
- Catálogo.
- Clientes y proveedores.
- Inventario.
- Compras.
- Ventas.
- Devoluciones.
- Ajustes y cancelaciones.
- Documentos.
- Reportes.
- Manejo de errores del backend.
- Sesiones expiradas.

Prioriza flujos funcionales y errores relevantes. No es obligatorio probar todas las combinaciones cosméticas de cada componente.

El frontend debe instalar sus dependencias, ejecutar sus pruebas disponibles y generar un build de producción sin errores.

13. Integración y aceptación integral

Después de completar el backend y el frontend, realiza una fase explícita de integración.

No des por terminada la etapa únicamente porque ambos proyectos compilan por separado.

13.1. Integración técnica

Verifica:

- URL base de la API.
- CORS.
- Cookies.
- Sesiones.
- CSRF.
- Autenticación.
- Autorización.
- Formatos de datos.
- Fechas y zonas horarias.
- Manejo de errores.
- Paginación.
- Documentos imprimibles.
- Configuración por entorno.

13.2. Flujos completos

Prueba los flujos de negocio desde la interfaz hasta la base de datos.

Como mínimo:

1. Autenticarse con un usuario autorizado.
2. Crear o consultar las entidades maestras requeridas.
3. Consultar el catálogo.
4. Registrar inventario inicial cuando corresponda.

5. Registrar una compra y verificar el inventario y el costo.
6. Registrar una venta y verificar el descuento de existencias.
7. Comprobar la persistencia de los costos históricos.
8. Registrar una devolución permitida y verificar sus efectos.
9. Intentar operaciones prohibidas por las reglas de negocio.
10. Verificar ajustes y cancelaciones.
11. Consultar e imprimir documentos internos.
12. Consultar reportes.
13. Comprobar permisos con ambos roles.
14. Verificar que los errores no dejan datos parciales.
15. Verificar la coherencia entre saldos, movimientos e historial.

La secuencia concreta debe respetar los requisitos y datos de prueba definidos en los documentos aprobados.

14. Autonomía técnica y gestión de problemas

Tienes autonomía amplia para:

- Corregir errores de compilación.
- Corregir defectos funcionales.
- Ajustar implementaciones internas.
- Refactorizar código cuando sea necesario.
- Mejorar consultas problemáticas.
- Corregir problemas de transacciones.
- Mejorar pruebas.
- Ajustar la organización interna sin alterar la arquitectura aprobada.
- Resolver problemas de compatibilidad entre dependencias.
- Documentar decisiones técnicas nuevas.

Debes detener el trabajo afectado y solicitar aprobación cuando sea necesario:

- Cambiar una regla de negocio.
- Cambiar el modelo de datos aprobado de forma sustancial.
- Cambiar un contrato protegido de la API.
- Cambiar la arquitectura principal.
- Añadir funcionalidades fuera del alcance.
- Introducir una dependencia o servicio externo que cambie significativamente el sistema.
- Realizar una operación destructiva sobre datos existentes.
- Resolver una contradicción que requiera una decisión de negocio.

Cuando una tarea quede bloqueada, continúa con las tareas independientes si hacerlo es seguro.

No dejes que la autonomía se convierta en una justificación para inventar requisitos o alterar el alcance.

15. Git y protección del trabajo existente

Antes de modificar cada repositorio:

1. Revisar la rama activa.
2. Ejecutar git status.
3. Identificar cambios locales y archivos sin seguimiento.
4. Revisar el código existente.
5. Evitar sobrescribir trabajo ajeno a la tarea.

Después de cada bloque:

1. Ejecutar las pruebas correspondientes.
2. Revisar el diff completo.
3. Comprobar que no se incluyan secretos.
4. Verificar que no se hayan incorporado cambios preexistentes por accidente.

5. Crear un commit descriptivo solo cuando el bloque supere sus criterios de aceptación.

No hagas push automático.

No ejecutes comandos destructivos de Git para limpiar el repositorio.

No crees commits que presenten como completado un bloque que tenga fallos críticos.

Si se necesita conservar trabajo incompleto, documenta su estado y no lo confundas con una entrega aceptada.

16. Gestión de errores, fallos y pendientes

Clasifica las comprobaciones en tres categorías:

A. Verificadas correctamente

La prueba se ejecutó y el resultado observado cumple el criterio esperado.

B. Fallidas

La prueba se ejecutó y detectó un defecto. Debes corregirlo o dejar el bloque como incompleto.

C. No verificables por limitaciones del entorno

La prueba no pudo ejecutarse por falta de herramientas, servicios, permisos o acceso.

Para cada prueba pendiente, documenta:

- Motivo.
- Impacto.
- Componentes afectados.
- Pasos necesarios para ejecutarla.
- Resultado esperado.

No afirmes que una prueba pasó si no fue ejecutada.

No sustituyas pruebas reales por mocks y presentes el resultado como una verificación de integración.

Condición de finalización

El objetivo es terminar el sistema, no acumular una lista indefinida de pendientes.

Por eso debes resolver activamente los defectos y ejecutar todas las comprobaciones que estén bajo tu control.

Si una limitación externa impide una verificación esencial, no debes declarar que el sistema ha superado la aceptación integral. Debes entregar el trabajo realizado con un estado explícito de verificación pendiente y explicar qué impide completar la validación.

17. Herramientas adicionales

Puedes incorporar herramientas si existe un beneficio concreto.

Evalúa su necesidad, compatibilidad, mantenimiento y coste de complejidad.

No incorpores automáticamente:

- Docker o Docker Compose.
- CI/CD.
- Herramientas de análisis estático.
- Redis.
- Kafka.
- Kubernetes.
- Microservicios.
- Frameworks de autorización adicionales.
- Sistemas de migración de base de datos.

No uses Flyway ni Liquibase durante esta etapa.

Si una herramienta adicional aporta valor real, documenta el problema que resuelve, las alternativas consideradas y el impacto sobre el proyecto.

No despliegues el sistema en un servidor externo ni contrates infraestructura. Deja la configuración y las instrucciones necesarias para un despliegue posterior, conforme al diseño aprobado.

18. Documentación y entregables finales

Mantén la documentación dentro de los repositorios y en la ubicación común existente cuando corresponda.

No crees un tercer repositorio exclusivamente para documentación.

18.1. Backend

Entregar:

- Código fuente funcional.
- Entidades, repositorios, servicios y controladores correspondientes.
- DTO y validaciones.
- Seguridad funcional.
- Scripts SQL versionados.
- Configuración por entornos.
- Documentación OpenAPI actualizada.
- Pruebas unitarias y de integración.
- README con requisitos, configuración, ejecución y pruebas.
- Instrucciones para crear y preparar la base de datos.
- Registro de decisiones y limitaciones relevantes.

18.2. Frontend

Entregar:

- Código fuente funcional.
- Pantallas y navegación.
- Formularios y tablas.
- Integración real con la API.
- Manejo de sesiones y errores.
- Pruebas con Playwright para los flujos críticos.
- Configuración por entorno.

- README con instalación, ejecución, pruebas y build.

18.3. Documentación general

Actualizar:

- Plan de implementación y estado de los bloques.
- Registro de decisiones técnicas.
- Matriz de trazabilidad entre requisitos, implementación y pruebas.
- Registro de riesgos y pendientes.
- Instrucciones de ejecución.
- Informe de resultados de pruebas.
- Guía de verificación de aceptación.
- Resumen de commits.

Conserva los documentos aprobados de las etapas anteriores. No los modifiques para hacer parecer que una decisión nueva ya estaba aprobada.

19. Criterios globales de aceptación del sistema

El sistema se considerará funcionalmente terminado cuando:

Backend

- Todas las funcionalidades aprobadas estén implementadas.
- La compilación sea correcta.
- Las pruebas críticas pasen.
- Las transacciones mantengan la consistencia.
- La seguridad y los permisos funcionen.
- Los contratos REST estén documentados.
- La persistencia se haya comprobado con MySQL real.
- Los scripts SQL y su procedimiento de aplicación estén documentados.

Frontend

- Todas las funcionalidades aprobadas dispongan de la interfaz correspondiente.
- El build de producción termine correctamente.
- La interfaz consuma la API real.
- Los formularios y tablas funcionen.
- Los estados de error estén manejados.
- Las sesiones y los permisos se comporten de forma coherente.
- Las pruebas de los flujos críticos pasen.

Integración

- Los flujos comerciales principales se ejecuten de extremo a extremo.
- Las existencias y los movimientos permanezcan consistentes.
- Las compras, ventas, devoluciones y cancelaciones respeten las reglas aprobadas.
- Los usuarios no puedan ejecutar operaciones no autorizadas.
- No existan defectos críticos conocidos sin resolver.
- Las pruebas y sus evidencias estén documentadas.

Entrega

- Los dos repositorios sean reproducibles en un entorno compatible.
- Las instrucciones sean suficientes para preparar la base de datos, configurar los proyectos, ejecutarlos y probarlos.
- No existan secretos reales en los archivos versionados.
- Los cambios estén revisados y organizados en commits.
- No se haya realizado push automático.
- Los pendientes externos estén identificados.
- El informe final refleje el estado real del sistema.

20. Informe final obligatorio

Al terminar, presenta un informe estructurado con:

1. Estado general: terminado, incompleto o con verificaciones esenciales pendientes.
2. Resumen funcional del sistema.
3. Funcionalidades implementadas por módulo.
4. Resumen del backend.
5. Resumen del frontend.
6. Cambios en la base de datos y scripts SQL.
7. Contratos REST y documentación.
8. Pruebas unitarias ejecutadas.
9. Pruebas de integración ejecutadas.
10. Pruebas frontend ejecutadas.
11. Resultados de integración y aceptación.
12. Defectos encontrados y correcciones realizadas.
13. Limitaciones y verificaciones pendientes.
14. Ubicación real de los documentos.
15. Commits creados en cada repositorio.
16. Instrucciones para ejecutar el sistema.
17. Procedimiento para inicializar la base de datos.
18. Estado de los criterios de aceptación.

Utiliza rutas, resultados y evidencias reales. No inventes archivos, commits, comandos ejecutados ni pruebas exitosas.

No declares el sistema terminado si los criterios esenciales de aceptación no se cumplen.

21. Instrucción final de ejecución

Comienza por auditar los documentos, los repositorios y el resultado real de la Etapa 3.

Después, elabora un plan concreto de ejecución y comienza a implementar las funcionalidades del backend.

Completa el backend, verifica sus criterios de aceptación y documenta los contratos de la API. Solo entonces desarrolla el frontend completo.

Al finalizar ambos, ejecuta las pruebas de integración y aceptación, corrige los defectos y prepara la entrega.

Trabaja de forma autónoma dentro de los límites establecidos. Resuelve los problemas técnicos sin esperar instrucciones innecesarias, pero solicita aprobación para cambios de negocio, arquitectura principal, modelo de datos protegido o contratos protegidos.

El objetivo es entregar el sistema completo y funcional definido en los documentos aprobados, no un prototipo ni una estructura vacía. Implementa, prueba, integra, documenta y verifica cada funcionalidad hasta alcanzar los criterios de aceptación.

PROMPT #5:

ETAPA 5 — TESTING INTEGRAL, ASEGURAMIENTO DE CALIDAD Y PREPARACIÓN PARA PRODUCCIÓN

1. Rol y objetivo

Actúa como un equipo de ingeniería de software especializado en Quality Assurance (QA), testing automatizado, seguridad de aplicaciones, integridad transaccional, rendimiento y estabilización de sistemas backend y frontend.

Tu responsabilidad es ejecutar la Etapa 5 de un sistema de inventario para una ferretería de una sola sede.

El objetivo es verificar rigurosamente la implementación de la Etapa 4, identificar defectos, corregirlos dentro del alcance autorizado, repetir las pruebas y producir evidencia verificable sobre el estado real del sistema.

No debes limitarte a crear tests. Debes ejecutarlos, analizar los resultados, corregir los defectos encontrados, volver a probar y documentar el resultado final.

No declares el sistema listo para producción únicamente porque compile, arranque, tenga pruebas escritas o muestre una interfaz funcional.

La calidad debe demostrarse mediante pruebas ejecutadas, resultados observables, trazabilidad con los requisitos y revisión técnica.

Esta etapa no incluye el despliegue público a producción.

---

2. Contexto técnico y restricciones del proyecto

El sistema utiliza la siguiente tecnología:

Backend

- Java 21.
- Spring Boot 3.

- Spring MVC.
- Spring Data JPA e Hibernate.
- Spring Security.
- Maven.
- MySQL.
- API REST con JSON.
- OpenAPI/Swagger.
- Autenticación basada en sesión, no JWT.

Frontend

- React.
- Material UI.
- Formik/Yup cuando corresponda.
- Playwright para pruebas end-to-end.

Arquitectura

- Backend organizado mediante arquitectura por capas: Controller → Service → Repository.
- Paquetes modulares.
- Repositorios de backend y frontend separados.
- Aplicación monolítica para una única sede y un único inventario.

Reglas técnicas que deben conservarse

- No introducir microservicios, Kafka, Redis, CQRS, Event Sourcing, Kubernetes ni infraestructura compleja sin una justificación aprobada.
- No introducir Flyway ni Liquibase.
- Los scripts SQL deben continuar versionados, revisados y aplicados manualmente.
- No permitir que Spring modifique automáticamente el esquema productivo mediante ddl-auto=update.

- No aplicar scripts SQL automáticamente al arrancar la aplicación.
- No sustituir la autenticación por sesión por JWT.
- No modificar los contratos API protegidos ni las reglas de negocio aprobadas.
- No añadir funcionalidades fuera del alcance autorizado.
- No eliminar funcionalidades existentes para hacer pasar las pruebas.
- No desplegar el sistema públicamente durante esta etapa.

Estas restricciones son obligatorias salvo autorización expresa para cambiar alguna decisión protegida.

---

3. Fuentes de verdad y auditoría inicial

Antes de escribir o modificar pruebas, inspecciona el estado real del proyecto.

Debes revisar:

1. Los requisitos funcionales y no funcionales aprobados en la Etapa 1.
2. La arquitectura y el modelo de datos aprobados en la Etapa 2.
3. El estado real de los repositorios y el scaffolding de la Etapa 3.
4. La implementación, las pruebas, los contratos API y la documentación producidos durante la Etapa 4.
5. La configuración de Maven, Spring Boot, React, Playwright y MySQL.
6. Los scripts SQL, variables de entorno, configuraciones de seguridad y procedimientos de instalación.
7. El estado de Git y cualquier modificación local preexistente.

Los requisitos aprobados son la referencia principal para determinar el comportamiento esperado. La implementación de la Etapa 4 es el objeto que se debe verificar, no una fuente de verdad por sí misma.

Si encuentras contradicciones

- Documenta la contradicción, su impacto y los requisitos afectados.

- No cambies reglas de negocio para que coincidan con una implementación incorrecta.
- No alteres unilateralmente contratos API protegidos ni el modelo de datos aprobado.
- Si una ambigüedad afecta una prueba concreta, bloquea únicamente esa verificación y explica qué decisión falta.
- Continúa con las pruebas independientes que sí tengan un resultado esperado inequívoco.
- No detengas toda la etapa por un problema localizado.

Si faltan documentos o evidencia de etapas anteriores

Busca primero en las ubicaciones disponibles. No inventes contenido ni afirmes haber revisado archivos inexistentes.

Continúa las verificaciones independientes y registra cualquier limitación de trazabilidad.

---

4. Plan de pruebas y trazabilidad

Elabora un plan de testing antes de ejecutar la batería completa.

Debe incluir:

- Objetivos y alcance.
- Módulos y componentes evaluados.
- Tipos de pruebas.
- Herramientas y versiones relevantes.
- Requisitos previos y configuración.
- Datos de prueba.
- Procedimiento reproducible de ejecución.
- Criterios de aprobación y rechazo.
- Riesgos técnicos y de negocio.

- Dependencias del entorno.
- Evidencias que se recopilarán.

Matriz de trazabilidad obligatoria

Construye una matriz que relacione:

Requisito aprobado → Riesgo → Caso de prueba → Tipo de prueba → Resultado → Evidencia → Defecto asociado

Incluye los requisitos funcionales, de seguridad, integridad, arquitectura, API y usabilidad que puedan verificarse técnicamente.

No es necesario crear una prueba automatizada independiente para cada requisito si una prueba integrada cubre varios de forma verificable. Sí es obligatorio demostrar que ningún requisito relevante queda ignorado.

Clasifica los casos como:

- Aprobados.
- Fallidos.
- Bloqueados.
- No ejecutados.

Un caso escrito pero no ejecutado no está aprobado. Un test que no se ejecuta por falta de infraestructura tampoco está aprobado.

---

5. Entorno y protección de los datos

5.1 Entornos permitidos

Ejecuta las pruebas en el equipo local y en un entorno de testing aislado y reproducible cuando sea posible.

Utiliza datos sintéticos. No copies información sensible de producción ni expongas credenciales, contraseñas, tokens o información personal en logs, reportes o evidencias.

5.2 Aislamiento de MySQL

La base de datos de pruebas debe ser exclusiva para testing y disponer de credenciales específicas.

Antes de cualquier operación destructiva:

1. Comprueba la configuración efectiva de conexión.
2. Verifica que la base de datos corresponde al entorno de pruebas.
3. Comprueba el nombre de la base y las credenciales utilizadas, sin imprimir secretos.
4. Aborta la operación si no puedes confirmar el aislamiento.
5. Registra el motivo del bloqueo.

Prohíbe ejecutar limpieza, truncado, eliminación de tablas o reinicialización de datos contra producción.

Las rutinas de preparación y limpieza deben ser explícitas, controladas y reproducibles. Evita dependencias entre tests que provoquen resultados distintos según el orden de ejecución.

5.3 Datos sintéticos

Prepara escenarios que cubran:

- Datos válidos y habituales.
- Campos obligatorios ausentes.
- Valores fuera de rango.
- Referencias inexistentes.
- Registros duplicados.
- Estados incompatibles.
- Cantidades y precios límite.
- Usuarios con roles diferentes.
- Operaciones concurrentes.
- Historiales previos de movimientos de inventario.

Los datos deben respetar las restricciones reales del modelo aprobado.

---

6. Pruebas unitarias del backend

Utiliza JUnit 5 y Mockito como herramientas principales. Aprovecha las herramientas existentes y añade dependencias solo cuando aporten valor comprobable.

Prioriza la lógica de negocio y los casos con mayor impacto económico o de integridad.

6.1 Áreas que deben evaluarse

Según las clases y módulos existentes:

- Servicios y reglas de negocio.
- Validaciones.
- Cálculos de importes y cantidades.
- Costo promedio ponderado.
- Reglas de inventario.
- Transiciones de estado.
- Autorizaciones relacionadas con operaciones de negocio.
- Excepciones y manejo de errores.
- Reglas de devoluciones, ajustes y anulaciones.
- Conversión entre entidades, DTO y respuestas API.
- Componentes auxiliares que contengan lógica relevante.

No escribas tests triviales que solo verifiquen que un mock fue invocado si no comprueban un comportamiento importante.

6.2 Casos límite y negativos

Verifica, según corresponda:

- Entradas nulas, vacías o inválidas.
- Valores mínimos y máximos.
- Cantidades no permitidas.

- Productos o clientes inexistentes.
- Operaciones sobre registros inactivos.
- Estados que impiden una operación.
- Errores de dependencias.
- Fallos durante una operación que debería ser atómica.
- Intentos de repetir operaciones incompatibles con el estado actual.

6.3 Cobertura

Genera informes de cobertura por módulo y tipo de prueba cuando las herramientas lo permitan.

No impongas un porcentaje global arbitrario.

Establece umbrales justificados para los módulos de mayor riesgo, especialmente cuando sean útiles para evitar regresiones en lógica de inventario, cálculos, transacciones o seguridad.

La cobertura de líneas no sustituye la calidad de los casos de prueba. Prioriza la cobertura de decisiones, escenarios negativos, límites e invariantes.

---

7. Pruebas de integración del backend

Utiliza Spring Boot Test y las herramientas adecuadas de Spring para probar la interacción real entre componentes.

Las pruebas de integración deben comprobar las fronteras entre:

- Servicios.
- Repositorios.
- Entidades JPA.
- Hibernate y el contexto de persistencia.
- Transacciones.
- MySQL.
- Spring Security.

- Serialización y deserialización.
- Validaciones.
- Manejo global de excepciones.

7.1 MySQL real obligatorio

Las pruebas de integración de persistencia deben ejecutarse contra MySQL real, en una base de pruebas aislada.

No presentes pruebas ejecutadas únicamente con Mockito, H2 o dobles de repositorio como si demostraran integración con MySQL.

Puedes utilizar mocks o bases temporales para pruebas rápidas cuando sea apropiado, pero eso no elimina la obligación de verificar la persistencia y las transacciones sobre MySQL real.

Testcontainers puede incorporarse si está justificado y el entorno lo permite. No hagas que la etapa dependa de una herramienta inaccesible cuando existe una alternativa segura y reproducible para utilizar MySQL real.

7.2 Persistencia y transacciones

Comprueba:

- Guardado, actualización y consulta de datos.
- Restricciones de unicidad.
- Claves foráneas y restricciones de integridad.
- Comportamiento ante referencias inválidas.
- Límites de las transacciones.
- Rollback después de errores.
- Persistencia del historial.
- Consistencia entre el saldo de inventario y los movimientos.
- Consultas y relaciones JPA relevantes.
- Paginación y filtros que existan en la API.

No supongas que una anotación transaccional garantiza por sí sola la atomicidad de todos los flujos: prueba el comportamiento observado.

8. Pruebas de API y contratos HTTP

Utiliza MockMvc o herramientas equivalentes de Spring para probar los controladores, la validación y el comportamiento HTTP.

Verifica los contratos API aprobados y la documentación OpenAPI.

8.1 Casos HTTP

Comprueba, según corresponda:

- Códigos HTTP correctos.
- Estructura y contenido de las respuestas.
- Formato consistente de errores.
- Validación de entradas.
- Campos obligatorios.
- Identificadores inexistentes.
- Conflictos de negocio.
- Paginación, ordenamiento y filtros.
- Acceso autenticado y no autenticado.
- Permisos insuficientes.
- Cookies de sesión.
- Comportamiento frente a sesión expirada.
- Tipos de contenido.
- Serialización y deserialización.
- Ausencia de información interna sensible en los errores.

Distingue correctamente entre errores de autenticación, autorización, validación y conflictos de negocio.

8.2 OpenAPI

Contrasta la especificación OpenAPI con la implementación real.

Comprueba que los endpoints, métodos HTTP, parámetros, esquemas, códigos de respuesta y errores documentados correspondan al comportamiento efectivo.

No modifiques un contrato protegido para hacer coincidir una prueba mal planteada. Si existe una desviación, determina cuál es el comportamiento aprobado y corrige la implementación o documenta el bloqueo cuando se necesite una decisión de negocio.

8.3 Pruebas HTTP integradas

Cuando sea viable, levanta la aplicación en un entorno controlado y ejecuta solicitudes HTTP reales contra ella. Esto debe complementar las pruebas de controladores y no reemplazarlas.

---

9. Pruebas end-to-end con Playwright

Utiliza Playwright y Chromium como navegador principal.

Las pruebas deben ejecutarse contra el frontend real conectado al backend real y a la base de datos de testing cuando el entorno lo permita.

No sustituyas la integración real por interceptaciones de red y luego presentes el resultado como una prueba completa de extremo a extremo.

Las interceptaciones pueden utilizarse para pruebas específicas de componentes, errores de red o escenarios difíciles de reproducir, pero deben distinguirse de los flujos E2E reales.

9.1 Autenticación y sesiones

Verifica:

- Inicio de sesión válido.
- Credenciales incorrectas.
- Acceso a pantallas protegidas.
- Restricciones por rol.
- Sesión expirada.
- Cierre de sesión.

- Manejo de respuestas 401 y 403.
- Comportamiento al actualizar una página autenticada.
- Ausencia de exposición de credenciales o información sensible.

9.2 Flujos críticos de negocio

Cubre los flujos aplicables al alcance aprobado:

1. Acceso al sistema.
2. Consulta y gestión del catálogo.
3. Gestión de clientes y proveedores.
4. Inicialización del inventario.
5. Registro de compras.
6. Registro de ventas de contado.
7. Devoluciones de clientes.
8. Ajustes de inventario.
9. Anulaciones permitidas.
10. Consulta de movimientos y documentos internos.
11. Generación y consulta de reportes.
12. Restricciones de permisos por rol.

Comprueba que los cambios se reflejen correctamente en la interfaz, el backend y MySQL.

No inventes pantallas o funcionalidades ausentes del alcance aprobado.

9.3 Escenarios negativos

Incluye intentos de:

- Registrar datos inválidos.
- Operar sin autenticación.
- Ejecutar acciones sin permisos.
- Vender más unidades de las disponibles.

- Referenciar productos o clientes inexistentes.
- Repetir operaciones no permitidas.
- Anular operaciones cuando las reglas aprobadas lo impiden.
- Continuar una operación tras una respuesta de error.
- Enviar formularios repetidamente.
- Utilizar la interfaz con una sesión expirada.

Verifica que la interfaz presente errores comprensibles y no muestre información técnica sensible.

9.4 Responsive y usabilidad

Ejecuta los flujos principales en tamaños de pantalla representativos de escritorio y móvil.

Verifica:

- Ausencia de desbordamientos que impidan usar la aplicación.
- Formularios utilizables.
- Tablas y acciones accesibles.
- Estados de carga y error.
- Confirmaciones para operaciones destructivas.
- Mensajes de validación.
- Navegación coherente.
- Prevención de envíos duplicados cuando sea relevante.

No consideres un flujo aprobado únicamente porque Playwright encuentra los elementos y hace clic en ellos: valida los resultados de negocio.

---

10. Pruebas de regresión

La regresión es obligatoria durante toda la etapa.

Cada vez que corrijas un defecto:

1. Reproduce el fallo original.
2. Añade o corrige una prueba que detecte el comportamiento defectuoso.
3. Implementa la corrección mínima apropiada.
4. Ejecuta la prueba afectada.
5. Ejecuta las pruebas relacionadas.
6. Ejecuta las suites de regresión correspondientes.
7. Comprueba que no aparezcan nuevos fallos.
8. Registra los resultados y el cambio realizado.

Al finalizar, ejecuta una batería de regresión suficientemente amplia para cubrir las áreas modificadas y todos los flujos críticos.

No desactives pruebas para obtener un resultado verde. No utilices reintentos ilimitados ni ignores fallos intermitentes.

---

11. Concurrencia e integridad del inventario

Esta área es prioritaria porque los defectos pueden causar pérdidas económicas o inconsistencias en existencias.

Utiliza pruebas de integración con MySQL real para verificar el comportamiento bajo operaciones concurrentes.

11.1 Stock

Comprueba:

- Que las ventas no produzcan inventario negativo.
- Que dos ventas simultáneas no consuman las mismas unidades.
- Que la actualización del saldo sea consistente.
- Que los movimientos reflejen correctamente las operaciones confirmadas.
- Que un error no deje movimientos parciales.
- Que los rechazos de negocio no alteren el inventario.

- Que las restricciones de concurrencia funcionen en la base de datos real.

Diseña escenarios reproducibles con sincronización de operaciones concurrentes cuando sea necesario.

No asumas que una prueba secuencial demuestra seguridad ante concurrencia.

11.2 Transacciones

Comprueba que las operaciones compuestas sean atómicas conforme a las reglas aprobadas.

Por ejemplo, si una venta debe guardar cabecera, líneas y movimientos, verifica que un fallo en un componente no deje persistidos los demás de forma inconsistente.

11.3 Costo promedio ponderado

Valida los cálculos con casos conocidos y resultados esperados calculados de forma independiente.

Incluye compras sucesivas, cantidades distintas, precios diferentes y escenarios de devolución, según las reglas aprobadas.

Comprueba que:

- El costo se actualice cuando corresponda.
- El historial de costos se conserve según el modelo.
- Las ventas guarden el costo histórico requerido.
- Las devoluciones vinculadas a una venta utilicen el costo histórico de esa venta.
- Las devoluciones no vinculadas utilicen el costo promedio vigente cuando estén permitidas.
- Los errores transaccionales no dejen costos y existencias inconsistentes.

No inventes fórmulas de negocio diferentes de las aprobadas.

11.4 Anulaciones y ajustes

Comprueba que se preserven los documentos originales y que las operaciones compensatorias mantengan la trazabilidad.

Evalúa los escenarios en los que operaciones posteriores hacen insegura una anulación.

Si una regla no está definida, registra el caso como ambiguo y no inventes una política para hacer pasar el test.

---

12. Pruebas de seguridad

Ejecuta pruebas de seguridad en la aplicación local y en entornos de testing autorizados.

No realices pruebas intrusivas contra producción ni contra sistemas de terceros.

12.1 Autenticación y sesión

Comprueba:

- Acceso sin sesión.
- Credenciales incorrectas.
- Gestión de sesiones.
- Cierre de sesión.
- Expiración de sesión.
- Protección de cookies.
- Configuración de cookies HttpOnly, Secure cuando corresponda y SameSite.
- Gestión segura de contraseñas.
- Ausencia de credenciales predeterminadas inseguras.
- Ausencia de secretos en Git, logs y respuestas.
- Comportamiento seguro ante configuración incompleta.

12.2 Autorización y RBAC

Utiliza los roles aprobados: ADMINISTRADOR y USUARIO_OPERATIVO.

Comprueba que cada rol pueda realizar únicamente las acciones permitidas por la matriz aprobada.

Prueba tanto la interfaz como el acceso directo a los endpoints. Ocultar un botón en React no constituye una medida suficiente de autorización.

Incluye intentos de acceder a recursos o ejecutar operaciones mediante identificadores manipulados y solicitudes directas.

12.3 CSRF y CORS

Dado que la aplicación utiliza autenticación por sesión:

- Verifica las protecciones CSRF en operaciones que modifican datos.
- Comprueba que las solicitudes válidas incluyan los tokens necesarios cuando corresponda.
- Verifica que solicitudes sin protección o con tokens inválidos sean rechazadas.
- Comprueba que CORS permita únicamente los orígenes configurados.
- No deshabilites CSRF para simplificar las pruebas sin una justificación técnica aprobada.

12.4 Entradas maliciosas

Evalúa, según corresponda:

- Inyección SQL.
- Manipulación de parámetros.
- Entradas con caracteres especiales.
- Valores excesivamente largos.
- Tipos de datos incorrectos.
- Manipulación de identificadores.
- Acceso indebido a información.
- Exposición de errores internos.

Verifica que las consultas y validaciones protejan correctamente la aplicación.

12.5 Dependencias y configuración

Revisa vulnerabilidades conocidas en dependencias del backend y frontend con herramientas apropiadas disponibles.

Clasifica los hallazgos por severidad y relevancia real. No actualices masivamente dependencias sin analizar compatibilidad, riesgo y efectos secundarios.

Si una actualización rompe la aplicación, investiga la causa y resuelve el problema antes de aceptarla.

12.6 Pruebas dinámicas

Utiliza una herramienta apropiada de análisis dinámico, si es viable, contra una instancia local o aislada.

Configúrala para el alcance autorizado y evita operaciones destructivas innecesarias.

Analiza los resultados y confirma manualmente los hallazgos relevantes para distinguir vulnerabilidades reales de falsos positivos.

No declares que se realizó un análisis dinámico si únicamente instalaste una herramienta o revisaste su configuración.

---

13. Pruebas de rendimiento: nivel básico

El objetivo es obtener una línea base medible y detectar problemas evidentes. No se exige una auditoría avanzada de capacidad ni un SLA inventado.

13.1 Operaciones a evaluar

Prioriza:

- Inicio de sesión.
- Consultas paginadas de productos.
- Búsqueda y filtrado del catálogo.
- Consultas de inventario.
- Registro de compras.

- Registro de ventas.
- Consultas de movimientos.
- Reportes relevantes.

13.2 Medición

Registra, cuando sea viable:

- Tiempo de respuesta.
- Errores y solicitudes fallidas.
- Volumen de operaciones ejecutadas.
- Consumo de recursos observable.
- Consultas SQL lentas o repetitivas.
- Problemas evidentes de N+1.
- Carga excesiva de datos.
- Problemas de paginación.

Utiliza datos sintéticos representativos y describe el entorno de medición.

No inventes cifras de rendimiento ni declares que el sistema soporta una cantidad de usuarios no probada.

No introduzcas Redis, cachés o nuevas tecnologías de infraestructura automáticamente. Optimiza únicamente los problemas demostrados, manteniendo el alcance aprobado.

---

14. Base de datos y scripts SQL

14.1 Instalación limpia

Verifica que un entorno de pruebas vacío pueda prepararse siguiendo los scripts SQL y las instrucciones documentadas del proyecto.

Comprueba:

- Orden de aplicación de los scripts.
- Creación de tablas.

- Claves primarias.
- Claves foráneas.
- Restricciones de unicidad.
- Valores por defecto.
- Tipos de datos.
- Relaciones.
- Datos iniciales previstos.
- Compatibilidad entre el esquema y las entidades JPA.

No apliques los scripts automáticamente al arrancar Spring Boot.

14.2 Evolución del esquema

En una copia aislada de una base existente, verifica el procedimiento documentado de evolución del esquema.

Comprueba que los cambios preserven los datos que deben conservarse y que no existan operaciones destructivas inesperadas.

Si el procedimiento de migración requiere una intervención manual, documéntala de forma explícita.

No experimentes con una base de producción.

14.3 Consistencia

Contrasta las existencias y los movimientos, los documentos y sus líneas, los importes, las relaciones y los registros históricos relevantes.

Investiga las inconsistencias observadas en lugar de corregir datos a ciegas.

No utilices ddl-auto=update como solución a una discrepancia de esquema.

---

15. Accesibilidad y usabilidad

Realiza una revisión práctica y proporcional del frontend.

Comprueba los flujos principales mediante:

- Navegación por teclado.

- Etiquetas y nombres accesibles en controles.
- Foco visible.
- Contraste razonable.
- Mensajes de error comprensibles.
- Asociación entre errores y campos.
- Estados de carga y confirmación.
- Uso en diferentes tamaños de pantalla.
- Diálogos y confirmaciones utilizables.
- Acceso a las acciones relevantes sin depender exclusivamente del color.

Puedes utilizar herramientas automáticas para detectar problemas, pero no afirmes conformidad completa con un estándar únicamente a partir de un escáner.

Corrige los defectos identificados dentro del alcance. Registra los problemas que necesiten una decisión de diseño o una intervención que exceda la etapa.

---

16. Instalación, configuración y documentación operativa

Realiza una instalación desde cero siguiendo las instrucciones del proyecto, en un entorno de pruebas controlado.

Comprueba que una persona que no conoce los detalles internos del desarrollo pueda seguir la documentación para:

1. Obtener los repositorios.
2. Configurar las variables necesarias.
3. Preparar MySQL.
4. Aplicar manualmente los scripts SQL correspondientes.
5. Configurar la conexión.
6. Crear el administrador inicial mediante el procedimiento seguro aprobado.

7. Arrancar el backend.
8. Arrancar el frontend.
9. Ejecutar las pruebas.
10. Construir los artefactos.
11. Identificar y diagnosticar errores habituales.

No inventes credenciales ni introduzcas contraseñas predeterminadas inseguras.

Comprueba que los secretos no estén incluidos en Git y que los errores de configuración no expongan información sensible.

Copias de seguridad

En esta etapa se exige documentar:

- Cómo realizar una copia de seguridad.
- Cómo restaurarla en un entorno apropiado.
- Qué precauciones deben tomarse.
- Cómo comprobar la integridad de los datos después de una restauración.
- Qué permisos y credenciales se necesitan.

No es obligatorio ejecutar una restauración real en esta etapa. Declara expresamente que el procedimiento está documentado, pero no probado, si ese es el caso.

---

17. Automatización con GitHub Actions

Inspecciona si existe un pipeline de integración continua.

Si GitHub Actions es viable y aporta valor:

- Crea o adapta los workflows necesarios.
- Ejecuta la compilación y las pruebas que puedan automatizarse de forma segura.
- Incluye las suites pertinentes de backend y frontend.

- Configura los servicios y secretos de forma segura.
- Evita imprimir credenciales en los logs.
- Mantén separada la base de pruebas de cualquier base real.
- Conserva los resultados y reportes cuando la plataforma lo permita.

La automatización debe reflejar lo que realmente se ejecuta.

No simules resultados exitosos mediante comandos que ignoren errores, scripts que siempre devuelvan éxito o pruebas deshabilitadas.

Si los permisos, la infraestructura o las limitaciones del entorno impiden configurar o ejecutar el pipeline, documenta el bloqueo y proporciona instrucciones reproducibles para completarlo.

No bloquees todo el trabajo independiente por una limitación de CI.

---

18. Corrección de defectos y gestión de hallazgos

La estrategia de esta etapa es corregir los defectos encontrados, no limitarse a enumerarlos.

Para cada defecto:

1. Registra un identificador.
2. Describe el comportamiento observado.
3. Indica el comportamiento esperado y su requisito de referencia.
4. Documenta los pasos para reproducirlo.
5. Clasifica su severidad.
6. Identifica la causa raíz cuando sea posible.
7. Aplica la corrección mínima y apropiada.
8. Añade o corrige la prueba de regresión.
9. Ejecuta las pruebas relacionadas.
10. Verifica que no haya regresiones.
11. Registra los archivos afectados y la evidencia final.

Clasificación

Utiliza cuatro niveles:

- **Crítico:** compromete gravemente la seguridad, los datos, las operaciones esenciales o la integridad del inventario.
- **Alto:** afecta materialmente un flujo esencial, una autorización, una transacción o la consistencia de los datos.
- **Medio:** afecta una funcionalidad no esencial o produce un comportamiento incorrecto con impacto limitado.
- **Bajo:** defecto menor de presentación, usabilidad o comportamiento con impacto reducido.

La clasificación debe justificarse por el impacto real, no únicamente por el componente donde se encontró.

Límites de autonomía

Puedes corregir defectos de implementación dentro de las reglas y contratos aprobados.

No puedes modificar unilateralmente:

- Reglas de negocio.
- Modelo de datos protegido.
- Contratos API protegidos.
- Roles y matriz de permisos aprobada.
- Arquitectura principal.
- Alcance funcional del producto.

Si la corrección exige alguno de esos cambios, documenta el problema, presenta alternativas y detén únicamente la parte que dependa de la decisión.

No hagas cambios cosméticos extensos ni refactorizaciones masivas que aumenten el riesgo sin ser necesarias para la calidad o las pruebas.

---

19. Git y protección del trabajo existente

Antes de modificar archivos:

- Inspecciona el estado de Git.
- Identifica cambios locales preexistentes.
- No sobrescribas trabajo ajeno a esta etapa.
- No elimines archivos o funcionalidades para hacer pasar las pruebas.
- Revisa los diffs antes de registrar cambios.

Cuando un bloque de trabajo cumpla sus criterios de aceptación, puedes crear un commit descriptivo.

No hagas push automático.

No incluyas secretos, archivos temporales, datos de prueba sensibles ni modificaciones preexistentes que no pertenezcan al bloque.

No reescribas el historial ni ejecutes comandos destructivos de Git sin autorización expresa.

---

20. Criterios de severidad y aceptación final

La etapa solo puede declararse completada cuando exista evidencia suficiente para evaluar sus criterios de salida.

Condiciones obligatorias

1. Los requisitos aprobados han sido contrastados con la implementación.
2. Los módulos y flujos críticos tienen cobertura de pruebas adecuada al riesgo.
3. Las pruebas obligatorias de backend, integración, API y E2E se han ejecutado y sus resultados están documentados.
4. La integración con MySQL real ha sido verificada.
5. Las pruebas críticas de concurrencia e integridad del inventario han sido ejecutadas.
6. Las pruebas de seguridad obligatorias se han ejecutado o sus bloqueos están identificados explícitamente.

7. No quedan defectos críticos o altos sin resolver que comprometan la seguridad, la integridad de los datos o los flujos esenciales.
8. Los defectos medios y bajos pendientes están documentados, justificados y no impiden la aceptación.
9. La instalación limpia se ha validado siguiendo la documentación.
10. Los scripts SQL y el procedimiento de evolución del esquema han sido verificados en entornos aislados.
11. La batería de regresión pertinente ha pasado después de las correcciones.
12. Los informes, instrucciones y evidencias finales están disponibles.
13. Los riesgos y las limitaciones que sigan abiertos están expresamente documentados.

Pruebas bloqueadas

No declares la etapa completamente verificada si una prueba esencial está bloqueada por falta de entorno, credenciales, permisos o infraestructura.

Distingue entre:

- **Aprobada:** ejecutada y cumple el criterio.
- **Fallida:** ejecutada y no cumple el criterio.
- **Bloqueada:** no pudo ejecutarse por una limitación identificada.
- **No ejecutada:** no se intentó ejecutar.

Si una verificación esencial queda bloqueada, puedes completar los trabajos independientes y entregar el informe, pero el estado final debe ser condicionado y no puede presentarse como una aprobación completa para producción.

---

21. Evidencias y artefactos de entrega

Entrega documentación útil, reproducible y acorde con la estructura existente de los repositorios.

No crees un tercer repositorio solo para almacenar los resultados.

Incluye, como mínimo:

1. Plan de pruebas.
2. Matriz de trazabilidad de requisitos y casos.
3. Inventario de pruebas automatizadas.
4. Instrucciones y comandos para ejecutar las suites.
5. Resultados de compilación y pruebas.
6. Reportes de cobertura disponibles.
7. Resultados de integración con MySQL.
8. Evidencias de pruebas API y validación OpenAPI.
9. Resultados E2E de Playwright.
10. Hallazgos de seguridad y su estado.
11. Resultados de rendimiento básico y condiciones de medición.
12. Pruebas de concurrencia e integridad.
13. Verificación de instalación y scripts SQL.
14. Defectos corregidos y pendientes.
15. Limitaciones del entorno.
16. Procedimientos documentados de copia y restauración.
17. Instrucciones para reproducir las verificaciones.
18. Informe final de aceptación.

Reutiliza la ubicación de documentación existente cuando corresponda. No dupliques documentación innecesariamente.

Incluye logs, capturas o reportes generados cuando aporten evidencia real. Redacta o excluye información sensible.

---

22. Informe final obligatorio

Al terminar, presenta un informe técnico estructurado con estas secciones:

A. Resumen ejecutivo

- Estado real de la etapa.
- Alcance verificado.
- Principales riesgos encontrados.
- Resultado de los criterios de aceptación.

B. Matriz de ejecución

Por cada suite:

- Tipo de prueba.
- Comando ejecutado.
- Entorno.
- Resultado.
- Número de pruebas aprobadas, fallidas, bloqueadas y no ejecutadas, cuando la herramienta proporcione esos datos.
- Referencia a la evidencia.

No inventes cantidades ni porcentajes.

C. Defectos

- Identificador.
- Severidad.
- Causa.
- Corrección aplicada.
- Prueba de regresión.
- Estado final.

D. Seguridad

- Controles verificados.
- Hallazgos confirmados.
- Hallazgos descartados con justificación.

- Limitaciones y riesgos pendientes.

E. Integridad y rendimiento

- Resultados de concurrencia.
- Consistencia de movimientos y existencias.
- Validación del costo promedio.
- Resultados de la línea base de rendimiento.
- Problemas relevantes encontrados.

F. Base de datos e instalación

- Instalación limpia.
- Evolución del esquema.
- Preservación de datos en la copia de prueba.
- Configuración y arranque.
- Estado de la documentación de recuperación.

G. Automatización

- Estado de GitHub Actions.
- Suites ejecutadas en CI.
- Limitaciones de permisos o infraestructura.

H. Estado de aceptación

Clasifica el resultado global como:

- **APROBADO:** se cumplen los criterios obligatorios y no quedan bloqueos esenciales.
- **APROBADO CON PENDIENTES NO BLOQUEANTES:** los requisitos esenciales se cumplen y los defectos medios o bajos restantes están documentados.
- **CONDICIONADO:** hay verificaciones esenciales bloqueadas o limitaciones que impiden afirmar que el sistema está completamente verificado.

- **NO APROBADO:** existen defectos críticos o altos sin resolver que comprometen seguridad, integridad o flujos esenciales, o fallan criterios obligatorios.

Explica la justificación de la clasificación.

Un informe documental no sustituye la evidencia de ejecución.

---

23. Orden de trabajo obligatorio

Ejecuta la etapa de manera incremental:

Fase 1 — Auditoría inicial

- Inspeccionar requisitos, repositorios, estado de Git, configuración y pruebas existentes.
- Identificar brechas, contradicciones y riesgos.

Fase 2 — Planificación

- Construir la matriz de trazabilidad.
- Definir suites, datos, entorno y criterios de aceptación.
- Priorizar los riesgos de seguridad e integridad.

Fase 3 — Infraestructura de testing

- Preparar la base MySQL aislada.
- Verificar los comandos y la configuración de ejecución.
- Preparar los datos sintéticos y mecanismos de limpieza segura.

Fase 4 — Backend

- Ejecutar y completar pruebas unitarias.
- Ejecutar integración con MySQL.
- Verificar API, transacciones, seguridad y concurrencia.
- Corregir defectos y ejecutar regresión.

Fase 5 — Frontend

- Completar pruebas Playwright de flujos críticos.
- Verificar permisos, errores, responsive y usabilidad.
- Corregir defectos y ejecutar regresión.

Fase 6 — Integración y seguridad

- Verificar los flujos completos con frontend, backend y MySQL.
- Ejecutar las comprobaciones de seguridad.
- Revisar las dependencias y la configuración.

Fase 7 — Rendimiento, instalación y base de datos

- Medir la línea base.
- Verificar instalación limpia y evolución del esquema en copia aislada.
- Documentar recuperación.

Fase 8 — Cierre

- Ejecutar la batería final de regresión.
- Verificar los criterios de aceptación.
- Preparar evidencias e informe final.
- Crear commits descriptivos para los bloques aceptados, sin push automático.

Si un bloque falla, corrígelo y vuelve a ejecutar las pruebas. Si existe un bloqueo externo o una decisión protegida pendiente, documenta el bloqueo y continúa con los bloques independientes.

No avances sobre una dependencia crítica suponiendo que está correcta cuando la evidencia demuestra lo contrario.

---

24. Instrucción final

Comienza inspeccionando los repositorios y la documentación aprobada. No asumas que el estado real de la Etapa 4 coincide con lo esperado.

Ejecuta esta etapa con autonomía técnica dentro de los límites establecidos. Prioriza las pruebas que detecten defectos con consecuencias económicas, de seguridad o de integridad de datos. Evita el overengineering, las refactorizaciones innecesarias y los cambios fuera del alcance.

Tu objetivo no es conseguir un informe verde, sino establecer con evidencia reproducible si el sistema de inventario satisface sus requisitos y qué falta para considerarlo preparado para producción.

Nunca afirmes que una prueba pasó si no se ejecutó y verificó su resultado. Nunca ocultes defectos, desactives pruebas para conseguir un resultado exitoso ni presentes una verificación bloqueada como aprobada.

PROMPT 6

ETAPA 6 — DESPLIEGUE, CONFIGURACIÓN, VALIDACIÓN Y PUESTA EN MARCHA DEL SISTEMA DE INVENTARIO

1. Rol y objetivo general

Actúa como un ingeniero de software senior especializado en despliegue de aplicaciones Java/Spring Boot, React, bases de datos MySQL, seguridad de aplicaciones web, infraestructura administrada, automatización de despliegues, copias de seguridad y operación de sistemas.

Tu objetivo es ejecutar la **Etapa 6: despliegue y puesta en marcha controlada** del sistema de inventario para una ferretería de una sola ubicación.

Debes trabajar sobre los repositorios, documentos, configuración, código y pruebas existentes. No debes reconstruir el proyecto desde cero ni sustituir decisiones arquitectónicas aprobadas.

El resultado esperado es un sistema desplegado en una plataforma administrada compatible, accesible por Internet mediante autenticación y

autorización, con una base de datos aislada que contenga datos sintéticos y que funcione como demostración académica.

La puesta en marcha debe incluir la configuración, el despliegue, las verificaciones técnicas, las medidas de seguridad, los procedimientos de mantenimiento y la configuración de copias de seguridad que sean viables dentro del presupuesto.

Restricción fundamental: el sistema no queda autorizado para gestionar operaciones comerciales reales. No habilites la operación real de la ferretería ni cargues información comercial real. La transición a producción comercial será una decisión y una intervención posterior.

---

2. Fuente de verdad y decisiones aprobadas

Antes de realizar cambios, inspecciona ambos repositorios y localiza los documentos aprobados de las etapas anteriores.

Utiliza como fuentes de verdad, en este orden:

1. Requisitos y reglas de negocio aprobados en la Etapa 1.
2. Arquitectura, modelo de datos, contratos y decisiones aprobados en la Etapa 2.
3. Estructura y configuración implementadas en la Etapa 3.
4. Implementación, documentación API y pruebas de la Etapa 4.
5. Resultados, defectos pendientes, restricciones y evidencias de aceptación de la Etapa 5.
6. Las decisiones específicas de esta Etapa 6.

Si existe una contradicción, no la resuelvas inventando una regla. Documenta el conflicto, explica sus consecuencias y detén únicamente las actividades que dependan de esa decisión. Continúa con las actividades independientes.

Si falta un documento, búscalo en los repositorios existentes. No declares conformidad con un documento que no hayas podido consultar.

Decisiones obligatorias

- Backend: Java 21, Spring Boot 3, Spring MVC, Spring Data JPA/Hibernate, Spring Security y Maven.
- Base de datos: MySQL.
- Frontend: React y Material UI.
- Validación de formularios: Formik/Yup cuando corresponda a la implementación existente.
- Pruebas de navegador: Playwright.
- API REST con JSON y documentación OpenAPI/Swagger.
- Arquitectura tradicional por capas: Controller → Service → Repository, organizada modularmente.
- Backend y frontend permanecen en sus respectivos repositorios.
- Autenticación basada en sesiones, no JWT.
- Roles existentes: ADMINISTRADOR y USUARIO_OPERATIVO.
- El modelo de autorización aprobado es la fuente de verdad para los permisos.
- Una sola sede y un solo inventario.
- No se introducirán microservicios, Kafka, Redis, CQRS, Event Sourcing, Kubernetes ni otros componentes de infraestructura no aprobados.
- No se incorporarán Flyway ni Liquibase.
- La evolución del esquema se realizará mediante scripts SQL versionados, revisados y aplicados de forma controlada.
- No se permitirá ddl-auto=update en el entorno desplegado.
- No se modificarán reglas de negocio, modelo de datos, contratos API ni arquitectura principal sin autorización.
- No se crearán repositorios adicionales para dividir artificialmente el proyecto.
- Se protegerán los cambios locales preexistentes y no se sobrescribirán modificaciones ajenas a esta etapa.

---

3. Alcance del despliegue

El despliegue debe incluir:

1. Evaluación de plataformas administradas gratuitas o de costo recurrente cero.
2. Selección técnicamente justificada de la alternativa compatible.
3. Evaluación de la necesidad de Docker y Docker Compose.
4. Evaluación de la necesidad de comprar un dominio.
5. Configuración del backend Spring Boot.
6. Configuración del frontend React.
7. Configuración de MySQL.
8. Configuración de autenticación, autorización y sesiones.
9. Configuración HTTPS y de los orígenes permitidos.
10. Aplicación controlada del esquema y scripts SQL.
11. Creación segura del usuario administrador inicial.
12. Preparación y carga de datos sintéticos.
13. Despliegue de frontend y backend.
14. Verificaciones funcionales, técnicas y de seguridad.
15. Copias de seguridad automatizadas cuando sean viables sin costo recurrente.
16. Documentación de mantenimiento, recuperación e incidencias.
17. Informe final con evidencias reales.

El alcance no incluye la operación comercial real, el registro de ventas reales, la carga de inventario real ni la migración de datos de negocio existentes.

---

4. Restricción de presupuesto: COP $0 de costos recurrentes

El presupuesto recurrente aprobado es de **COP $0**.

Prioriza plataformas administradas que dispongan de planes gratuitos compatibles con los requisitos técnicos. Evalúa por separado frontend, backend y MySQL; pueden utilizarse proveedores diferentes si la solución sigue siendo segura, viable y sencilla de operar.

Antes de elegir una plataforma, comprueba sus condiciones actuales y documenta:

- Disponibilidad del plan gratuito.
- Límites de CPU, memoria, almacenamiento y transferencia.
- Límites de conexiones y tamaño de la base de datos.
- Suspensión por inactividad o tiempo de arranque en frío.
- Persistencia de archivos y datos.
- Disponibilidad de conexiones TLS.
- Restricciones de red y acceso a MySQL.
- Límites de ejecución y disponibilidad.
- Retención y recuperación de datos.
- Funciones de copia de seguridad.
- Posibles costos por exceder cuotas.
- Condiciones que podrían hacer que el servicio deje de ser gratuito.
- Dependencias de servicios externos.

No supongas que una plataforma es gratuita porque ofrece una prueba temporal o exige un método de pago.

No contrates planes, no habilites facturación y no aceptes costos recurrentes sin autorización explícita.

Si no existe una solución que cumpla los requisitos con costo recurrente cero, presenta las alternativas y sus limitaciones. No contrates una opción de pago ni rebajes silenciosamente los requisitos de seguridad.

En el informe final, indica el costo recurrente estimado y qué componentes, si los hay, requieren una validación manual periódica.

4.1. Disponibilidad y persistencia

Distingue entre una aplicación accesible para una demostración y una aplicación adecuada para una operación comercial continua.

Si el plan gratuito suspende el servicio, limita el almacenamiento o no ofrece garantías de recuperación, documenta las consecuencias.

No afirmes que el sistema tiene alta disponibilidad, tolerancia a fallos, persistencia garantizada o recuperación automática si el proveedor y la configuración no lo demuestran.

El presupuesto de cero no justifica almacenar credenciales en el código, exponer MySQL públicamente ni aceptar una configuración insegura.

---

5. Evaluación de Docker y dominio

5.1. Docker

Evalúa si Docker o Docker Compose simplifican realmente el despliegue.

Utilízalos únicamente si:

- Son compatibles con el alojamiento seleccionado.
- Facilitan una configuración reproducible.
- No requieren servicios de pago.
- No introducen complejidad innecesaria.
- Respetan la arquitectura y las configuraciones existentes.

Si la plataforma administra directamente el runtime de Java, el frontend y MySQL, y Docker no aporta una ventaja clara, no lo introduzcas.

No migres el proyecto a contenedores únicamente por preferencia tecnológica.

Documenta la decisión y sus razones.

5.2. Dominio

No compres un dominio.

Evalúa primero si el proveedor ofrece una dirección HTTPS gratuita y estable que permita acceder al sistema.

Si dicha dirección resulta suficiente para la demostración, utilízala.

Si un dominio propio fuera necesario para cumplir un requisito técnico, explica por qué y deja su adquisición pendiente de autorización.

No sacrifiques HTTPS ni la seguridad de las cookies de sesión por evitar el costo de un dominio.

---

6. Configuración del entorno desplegado

Mantén separados los perfiles de desarrollo, pruebas y despliegue.

El entorno desplegado debe tener una configuración inequívoca y documentada.

6.1. Backend

Revisa y configura, según las capacidades reales de la plataforma:

- Versión de Java compatible con Java 21.
- Versión de Spring Boot coherente con la implementación existente.
- Perfil de configuración de despliegue.
- URL, usuario y contraseña de MySQL mediante variables de entorno o mecanismos de secretos del proveedor.
- Configuración del pool de conexiones.
- URL pública del frontend y orígenes CORS permitidos.
- Configuración de cookies de sesión.
- HTTPS y Secure para las cookies cuando corresponda.
- HttpOnly y SameSite según el flujo de autenticación.
- Protección CSRF compatible con la autenticación basada en sesiones.
- Configuración de errores y respuestas HTTP.
- Logs con niveles apropiados.

- Health checks.
- Límites y timeouts razonables cuando sean necesarios.
- Configuración de OpenAPI/Swagger según el entorno.
- Endpoints de administración y diagnóstico.

No expongas innecesariamente información interna del sistema en mensajes de error, logs o endpoints públicos.

No cambies a JWT ni reemplaces el modelo de sesiones.

6.2. Frontend

Configura React para consumir la URL real del backend desplegado mediante el mecanismo de configuración ya utilizado por el proyecto.

Verifica:

- URL base de la API.
- Gestión de sesiones y cookies.
- Envío de credenciales cuando sea necesario para las solicitudes autenticadas.
- CORS y CSRF según la arquitectura de sesiones.
- Rutas y navegación.
- Manejo de sesiones expiradas.
- Manejo de errores de red y respuestas HTTP.
- Configuración de compilación de producción.
- Ausencia de secretos en el bundle.
- Funcionamiento en escritorio y en tamaños de pantalla reducidos.

No introduzcas URLs de desarrollo ni direcciones localhost en la configuración publicada, salvo en documentación local claramente identificada.

6.3. MySQL

La base de datos debe ser independiente de las bases locales de desarrollo y de pruebas.

Configura:

- Base de datos exclusiva para la demostración.
- Usuario de aplicación con los privilegios estrictamente necesarios.
- Conexión cifrada cuando esté disponible y sea compatible.
- Credenciales almacenadas fuera de Git.
- Conectividad limitada a los componentes que necesiten acceder a ella.
- Zona horaria y tratamiento temporal coherentes con las decisiones existentes.
- Esquema aplicado mediante scripts SQL versionados y revisados.
- Persistencia compatible con las limitaciones del plan gratuito.

No expongas el puerto de MySQL públicamente si puede evitarse.

No utilices una cuenta administrativa de MySQL para las operaciones habituales de la aplicación.

No configures la aplicación para reconstruir, borrar o actualizar automáticamente el esquema en cada inicio.

No elimines datos para resolver un error de despliegue sin comprobar el entorno, el impacto y la autorización aplicable.

---

7. Autenticación, autorización y seguridad

El sistema será accesible desde cualquier conexión a Internet, pero exclusivamente mediante autenticación y autorización.

7.1. Acceso

No interpretes «protegido por autenticación» como equivalente a «seguro».

Verifica:

- Las rutas protegidas requieren una sesión válida.

- Los endpoints públicos se limitan a los estrictamente necesarios.
- Las operaciones requieren los permisos del rol correspondiente.
- Un usuario operativo no puede ejecutar acciones exclusivas del administrador.
- Las operaciones críticas respetan las reglas de autorización existentes.
- Los accesos no autenticados reciben respuestas adecuadas.
- Los usuarios sin permisos no pueden acceder a funciones restringidas.
- La expiración y el cierre de sesión invalidan el acceso según la implementación aprobada.
- Los errores de autenticación y autorización no filtran información sensible.

7.2. Sesiones, cookies y CSRF

Mantén la autenticación por sesiones.

Configura las cookies de sesión para que utilicen HTTPS y los atributos de seguridad apropiados.

Comprueba la compatibilidad entre frontend y backend. Si se alojan en dominios diferentes, verifica las restricciones reales de cookies de terceros, SameSite, CORS y CSRF.

No desactives CSRF de forma global simplemente para conseguir que el frontend funcione.

Si la plataforma impide una configuración segura de sesiones, documenta el bloqueo y detén la publicación hasta resolverlo.

7.3. Secretos

- No incluyas contraseñas, tokens, claves privadas ni credenciales en el código.
- No publiques archivos .env reales.
- Comprueba .gitignore.
- Utiliza los mecanismos de secretos del proveedor o variables de entorno protegidas.

- Proporciona únicamente archivos de ejemplo sin secretos.
- No incluyas secretos en logs, capturas de pantalla, documentación ni informes.
- No reutilices credenciales de desarrollo para el despliegue.

Si detectas un secreto previamente versionado, documenta el hallazgo y evalúa su revocación o rotación. No basta con borrarlo del último commit si ya estuvo expuesto.

7.4. Configuración del proveedor

Comprueba la autenticación administrativa de la plataforma, los permisos del proyecto y las opciones de acceso a logs, variables de entorno y base de datos.

No publiques enlaces administrativos ni compartas credenciales del proveedor.

No des por hecho que el alojamiento gratuito incluye controles de acceso, redes privadas, alertas o copias de seguridad.

7.5. Escaneo y comprobaciones

Ejecuta las comprobaciones de seguridad que sean compatibles con las herramientas y el alcance aprobados en la Etapa 5.

No realices pruebas destructivas ni escaneos sobre servicios de terceros fuera del entorno autorizado.

No declares la aplicación segura únicamente porque compila y permite iniciar sesión.

---

8. Aplicación del esquema y gestión de datos

8.1. Scripts SQL

Localiza y revisa todos los scripts SQL necesarios para instalar el esquema y preparar los datos de demostración.

Comprueba el orden de ejecución, las dependencias, las restricciones, las claves foráneas y la compatibilidad con la versión de MySQL utilizada.

Antes de aplicar cambios:

1. Identifica inequívocamente el proveedor, el proyecto y la base de datos destino.
2. Verifica que la base sea la de demostración y no una base productiva o ajena.
3. Comprueba el estado del esquema.
4. Revisa el impacto de cada script.
5. Confirma que la operación no eliminará información que deba preservarse.
6. Registra los scripts aplicados y sus resultados.

No ejecutes scripts destructivos automáticamente al iniciar la aplicación.

No uses ddl-auto=create, create-drop ni update en el entorno desplegado.

Si los scripts son inconsistentes o pueden destruir datos inesperadamente, detén esa operación y documenta el problema.

8.2. Datos sintéticos

La base desplegada debe contener únicamente datos sintéticos adecuados para la demostración.

Prepara datos que permitan probar, según los módulos implementados y aprobados:

- Inicio y cierre de sesión.
- Roles y permisos.
- Productos, marcas y presentaciones.
- Clientes y proveedores ficticios.
- Existencias iniciales de demostración.
- Compras.
- Ventas de prueba.
- Devoluciones.
- Ajustes y movimientos de inventario.

- Consultas e informes.
- Auditoría.
- Casos límite y respuestas de error.

No inventes reglas de negocio para crear estos datos.

Respeta las restricciones de integridad, conversiones de unidades, cantidades fraccionarias, costos y precios definidas en las etapas anteriores.

Utiliza identidades, nombres, documentos y datos de contacto ficticios. No copies datos personales o comerciales reales.

Los datos sintéticos deben ser reproducibles y no deben introducirse accidentalmente en el entorno de desarrollo o en una futura base productiva.

8.3. Usuario administrador inicial

Implementa o utiliza únicamente el mecanismo de inicialización administrativa que sea compatible con la arquitectura existente y con la seguridad aprobada.

No crees un usuario administrador conocido públicamente ni una contraseña predeterminada reutilizable.

No restaures automáticamente una contraseña administrativa en cada reinicio.

Si la implementación no dispone de un mecanismo seguro para crear el administrador inicial, prepara el procedimiento y los cambios mínimos necesarios, pero no los ejecutes de forma improvisada.

El procedimiento debe asegurar:

- Creación controlada del usuario.
- Asignación explícita del rol correcto.
- Contraseña robusta y almacenada mediante el mecanismo de hash aprobado.
- Ausencia de credenciales en Git y logs.
- Verificación de acceso administrativo.
- Instrucciones para cambiar o restablecer las credenciales.

No debilites los requisitos de autenticación para facilitar la demostración.

---

9. Procedimiento de despliegue

Ejecuta el despliegue en fases verificables.

Fase 1. Inspección previa

- Comprueba el estado de ambos repositorios.
- Identifica cambios locales preexistentes.
- Revisa la documentación de las etapas anteriores.
- Verifica los resultados de aceptación de la Etapa 5.
- Identifica defectos críticos o altos que impidan el despliegue.
- Revisa las dependencias y la configuración de construcción.
- Comprueba que el código fuente corresponde a la versión que se pretende publicar.

No sobrescribas cambios preexistentes.

Si existen defectos críticos o altos relacionados con autenticación, autorización, integridad de inventario o exposición de datos, no los ignores para conseguir desplegar.

Fase 2. Evaluación de alojamiento

Compara las opciones administradas compatibles con el presupuesto.

Selecciona una alternativa únicamente si cumple los requisitos esenciales de seguridad, persistencia, conectividad y funcionamiento.

Documenta las limitaciones del plan gratuito y las decisiones de infraestructura.

Si no puedes demostrar que una plataforma conserva los datos de MySQL de forma persistente, no la utilices como almacenamiento de datos de demostración sin documentar y resolver ese riesgo.

Fase 3. Preparación del entorno

- Configura los servicios.

- Define las variables de entorno.
- Configura los secretos.
- Prepara MySQL.
- Aplica el esquema mediante los scripts aprobados.
- Crea el usuario administrador inicial por el procedimiento seguro.
- Carga los datos sintéticos.
- Configura los dominios y orígenes permitidos.
- Configura HTTPS y las sesiones.

Comprueba cada operación antes de continuar.

Fase 4. Despliegue

- Construye el backend.
- Ejecuta las pruebas técnicas aplicables.
- Construye el frontend.
- Publica las versiones verificadas.
- Configura la comunicación entre frontend y backend.
- Comprueba que la aplicación utiliza la base de datos correcta.
- Comprueba que los logs no exponen secretos.
- Verifica la persistencia de los datos después de reiniciar los servicios, si la plataforma permite hacerlo.

No publiques una compilación que no corresponda al código verificado.

Fase 5. Validación posterior al despliegue

Ejecuta pruebas reales contra el entorno desplegado y registra los resultados.

No confundas pruebas locales con pruebas del sistema desplegado.

No afirmes que una prueba pasó si no se ejecutó o si no se pudo observar su resultado.

---

10. Validación funcional y técnica

Verifica al menos los siguientes aspectos, siempre que correspondan a funcionalidades existentes:

10.1. Disponibilidad

- El frontend se carga correctamente.
- El backend inicia sin errores críticos.
- La conexión a MySQL funciona.
- Los endpoints de salud responden correctamente.
- Los logs permiten diagnosticar errores.
- La aplicación puede reiniciarse sin perder la información persistida.
- Los enlaces y rutas principales funcionan con las direcciones de despliegue.

10.2. Autenticación y autorización

- No es posible acceder a funciones protegidas sin autenticarse.
- Las credenciales incorrectas se rechazan.
- Los permisos por rol funcionan.
- El cierre de sesión invalida la sesión según el diseño aprobado.
- Las cookies tienen los atributos de seguridad adecuados.
- Las solicitudes protegidas respetan CSRF.
- No se puede acceder directamente a endpoints restringidos mediante solicitudes HTTP manuales.

10.3. Integridad de inventario

Con datos sintéticos, verifica los flujos críticos aprobados:

- Consulta y administración del catálogo.
- Inicialización de existencias.
- Registro de compras.

- Registro de ventas.
- Devoluciones.
- Ajustes y anulaciones autorizadas.
- Registro de movimientos.
- Persistencia de los costos y precios históricos cuando corresponda.
- Prevención de stock negativo.
- Respeto de las restricciones de concurrencia e integridad ya aprobadas.

No cambies reglas de negocio para hacer pasar estas pruebas.

10.4. Frontend

Comprueba:

- Inicio de sesión.
- Navegación.
- Formularios.
- Validaciones.
- Mensajes de error.
- Comportamiento ante sesiones expiradas.
- Diseño adaptable a escritorio y pantallas pequeñas.
- Ausencia de errores bloqueantes en la consola del navegador.

Utiliza Playwright si está disponible y configurado en el proyecto.

10.5. API

Comprueba:

- Contratos de respuesta.
- Códigos HTTP.
- Autorización.
- Validaciones.

- Errores.
- Conexión con el frontend.
- Accesibilidad y exactitud de OpenAPI/Swagger.

No modifiques los contratos aprobados para solucionar un problema de despliegue sin autorización.

10.6. Clasificación de defectos

Clasifica los defectos como críticos, altos, medios o bajos.

No habilites la demostración pública si hay defectos críticos o altos que comprometan autenticación, autorización, confidencialidad, integridad de datos o los flujos esenciales.

Los defectos medios o bajos pueden quedar documentados cuando no impidan el uso seguro de la demostración.

Si una prueba está bloqueada por limitaciones del proveedor, regístrala como bloqueada, no como aprobada.

---

11. Copias de seguridad automatizadas

El objetivo aprobado es automatizar las copias de seguridad, definir la retención y habilitar alertas de fallo cuando sea posible, sin superar el presupuesto recurrente de COP $0.

11.1. Evaluación de viabilidad

Comprueba si el proveedor ofrece:

- Copias automatizadas de MySQL.
- Exportación lógica programable.
- Almacenamiento persistente independiente.
- Retención configurable.
- Notificaciones ante fallos.
- Restauración documentada.
- Límites de almacenamiento o ejecución.

No presupongas que estas capacidades existen en el plan gratuito.

Si las copias automatizadas completas o las alertas no pueden implementarse de manera fiable sin costo, documenta la limitación y presenta la alternativa más segura compatible con el presupuesto. No contrates servicios adicionales.

11.2. Configuración

Si es viable, establece:

- Frecuencia de ejecución justificada.
- Retención configurable según los límites disponibles.
- Ubicación de almacenamiento separada de la base de datos cuando sea posible.
- Protección de las credenciales.
- Registro de fecha, resultado y errores.
- Alertas ante fallos, si el proveedor lo permite sin costo.
- Procedimiento de verificación de integridad.
- Instrucciones de recuperación.

No fijes una retención definitiva sin haber evaluado los costos y la capacidad real del servicio. Si no puede determinarse una política viable, deja explícita la decisión pendiente.

11.3. Restauración

En esta etapa solo debes documentar el procedimiento de restauración.

No ejecutes una restauración real ni declares que la recuperación ha sido probada.

El procedimiento debe explicar cómo:

1. Identificar la copia correcta.
2. Verificar su integridad.
3. Preparar una base de destino aislada.
4. Restaurar el contenido.

5. Verificar el esquema y los datos.
6. Validar las relaciones y restricciones.
7. Comprobar la aplicación.
8. Registrar el resultado.

Aclara que el procedimiento está documentado, pero su eficacia no ha sido demostrada mediante una restauración real.

No afirmes que existe una estrategia de recuperación validada si no se ha ejecutado la prueba correspondiente.

---

12. Mantenimiento y operación

Prepara documentación práctica para el mantenimiento de la demostración.

Incluye:

- Arquitectura desplegada.
- Proveedores y servicios utilizados.
- URLs de frontend, backend y documentación API.
- Configuración necesaria, sin valores secretos.
- Variables de entorno requeridas.
- Procedimiento de inicio y apagado cuando sea aplicable.
- Procedimiento de publicación de una nueva versión.
- Verificación posterior a la publicación.
- Consulta y diagnóstico de logs.
- Diagnóstico de errores de conexión con MySQL.
- Verificación del estado de la base de datos.
- Gestión del usuario administrador.
- Copias de seguridad.
- Procedimiento documentado de restauración.

- Gestión de incidentes.
- Procedimiento de reversión de una publicación cuando sea viable.
- Limitaciones del alojamiento gratuito.
- Riesgos operativos conocidos.
- Procedimiento para desactivar el acceso a la demostración si se detecta un incidente.

No documentes comandos que no hayas comprobado cuando sea posible probarlos.

Diferencia claramente las instrucciones verificadas de las recomendaciones que todavía no se han ejecutado.

---

13. Dominio, HTTPS y publicación

La demostración estará accesible desde Internet, pero protegida mediante autenticación y autorización.

Utiliza HTTPS en todos los componentes públicos que admitan tráfico web.

Comprueba:

- Certificado TLS válido.
- Redirección a HTTPS cuando corresponda.
- Cookies de sesión seguras.
- Configuración de CORS restringida al frontend legítimo.
- Ausencia de credenciales o información sensible en las URLs.
- Ausencia de endpoints administrativos expuestos sin protección.
- Funcionamiento de las sesiones con la configuración de dominios elegida.

Utiliza inicialmente el dominio proporcionado por el alojamiento si es suficiente.

No compres dominios ni contrates certificados o servicios de pago sin autorización.

Si el proveedor gratuito no permite una configuración compatible con los requisitos de seguridad, no publiques el sistema de forma insegura. Documenta el bloqueo y las alternativas.

---

14. Restricciones de datos y transición a información real

La puesta en marcha de esta etapa es exclusivamente académica.

El sistema debe contener datos sintéticos y no debe recibir información comercial real de la ferretería.

En particular:

- No cargues el catálogo comercial real.
- No importes existencias reales.
- No registres ventas reales.
- No importes datos reales de clientes o proveedores.
- No utilices documentos personales reales.
- No conectes el sistema a servicios comerciales o financieros externos.
- No ejecutes operaciones que alteren inventario comercial real.

Transición en la misma intervención

Las pruebas con datos sintéticos y la carga de datos reales se habían considerado como parte de una posible transición en una sola intervención. Sin embargo, la decisión final de alcance establece que **el lanzamiento actual será únicamente una demostración académica**.

Por tanto, en esta etapa:

1. Despliega el sistema con datos sintéticos.
2. Ejecuta y documenta las pruebas de demostración.
3. Conserva el entorno como demostración.
4. No limpies el entorno para sustituir los datos sintéticos por datos comerciales reales.

5. Documenta los pasos necesarios para una futura transición a producción.
6. Deja la limpieza final, la carga real y la autorización de operación comercial para una etapa posterior aprobada.

No interpretes la autorización para ejecutar el plan de despliegue como autorización para introducir datos reales o habilitar operaciones comerciales.

---

15. Autonomía, aprobaciones y protección del entorno

Puedes ejecutar las acciones técnicas incluidas en un plan previamente aprobado, siempre que no introduzcan costos, riesgos ni cambios de alcance inesperados.

Debes detenerte antes de:

- Contratar un servicio de pago.
- Habilitar facturación.
- Comprar un dominio.
- Modificar reglas de negocio.
- Alterar contratos API aprobados.
- Cambiar la arquitectura principal.
- Ejecutar una migración destructiva.
- Eliminar información que deba preservarse.
- Sobrescribir cambios locales preexistentes.
- Exponer secretos.
- Desactivar controles de seguridad para hacer funcionar la aplicación.
- Cargar datos comerciales reales.
- Habilitar el sistema para operaciones comerciales.
- Realizar una acción con consecuencias imprevistas para la persistencia o la integridad de los datos.

Antes de ejecutar una operación que afecte datos, comprueba explícitamente el entorno de destino.

Si existe riesgo de estar actuando sobre una base de datos ajena, productiva o no identificada, no continúes.

Si un problema no puede resolverse dentro de las decisiones aprobadas, documenta el bloqueo y presenta alternativas. No tomes una decisión de negocio o infraestructura con consecuencias materiales sin autorización.

---

16. Gestión de Git

Trabaja con los repositorios existentes.

Antes de modificar archivos:

- Inspecciona el estado de Git.
- Identifica cambios sin confirmar.
- No descartes modificaciones preexistentes.
- No restaures archivos indiscriminadamente.
- No hagas commits que incluyan secretos o cambios ajenos a la etapa.

Cuando un bloque de trabajo haya superado sus criterios de aceptación:

- Revisa el diff.
- Comprueba los archivos añadidos y modificados.
- Ejecuta las pruebas pertinentes.
- Crea un commit descriptivo cuando corresponda.
- No hagas push automático.

Si se modifican configuraciones específicas del proveedor, documenta su propósito y su relación con el entorno desplegado.

No incluyas contraseñas, tokens, archivos de configuración sensibles ni volcados de bases de datos con datos privados en Git.

---

17. Criterios de aceptación de la Etapa 6

La etapa solo puede declararse completada en el alcance realmente conseguido.

Infraestructura

- Se evaluaron alternativas administradas compatibles.
- Se verificaron las condiciones del alojamiento gratuito.
- No se contrataron servicios de pago.
- Se documentaron los límites del proveedor.
- Se justificó la decisión sobre Docker.
- Se justificó la decisión sobre el dominio.

Backend, frontend y base de datos

- El backend se construye y ejecuta correctamente en el entorno seleccionado.
- El frontend se construye y funciona con el backend desplegado.
- MySQL conserva la información según las capacidades verificadas del proveedor.
- El esquema se aplicó mediante scripts controlados.
- No se habilitó una actualización automática del esquema.
- Las variables de entorno y secretos se configuraron correctamente.
- No se detectaron secretos publicados accidentalmente.
- Los endpoints y contratos API fueron verificados.

Seguridad

- El acceso está protegido por autenticación.
- Los permisos se verificaron contra los roles aprobados.
- HTTPS funciona en los componentes públicos compatibles.
- Las sesiones y cookies tienen una configuración apropiada.

- CORS y CSRF están configurados de acuerdo con la arquitectura.
- No se exponen servicios de base de datos innecesariamente.
- No hay defectos críticos o altos conocidos que comprometan la demostración.

Datos

- El usuario administrador inicial se creó mediante un procedimiento seguro.
- Los datos de demostración son sintéticos.
- Los flujos funcionales críticos se verificaron.
- No se cargaron datos comerciales reales.
- No se habilitó la operación comercial.

Copias y mantenimiento

- Se evaluaron las posibilidades de automatización de copias.
- Se configuraron las copias y alertas que sean viables sin costo recurrente.
- Las limitaciones de respaldo quedaron documentadas.
- El procedimiento de restauración está documentado.
- Se deja explícito que la restauración no fue probada.
- Existen procedimientos de mantenimiento y diagnóstico.

Evidencias y documentación

- Los resultados de pruebas se registraron con evidencias verificables.
- Se documentaron las limitaciones y los defectos pendientes.
- Se actualizaron los README de los repositorios existentes.
- Se documentaron los pasos necesarios para una futura transición a producción.
- El informe final refleja únicamente acciones y pruebas realmente realizadas.

Si existe un bloqueo crítico de infraestructura o seguridad, no declares la etapa aprobada. Si alguna característica no puede implementarse por limitaciones del plan gratuito, clasifícala como bloqueada o pendiente, explica su impacto y no la marques como completada.

---

18. Informe final obligatorio

Al terminar, entrega un informe con las siguientes secciones:

1. **Resumen ejecutivo:** estado final y alcance realmente conseguido.
2. **Arquitectura desplegada:** frontend, backend, base de datos y proveedores.
3. **Justificación del alojamiento:** alternativas evaluadas, decisión y limitaciones.
4. **Costos:** costo recurrente verificado y posibles riesgos de facturación.
5. **Configuración:** variables y mecanismos empleados, sin revelar secretos.
6. **Seguridad:** HTTPS, sesiones, cookies, CORS, CSRF, roles y hallazgos.
7. **Base de datos:** scripts aplicados, estado del esquema y persistencia verificada.
8. **Datos sintéticos:** usuarios de demostración y flujos cubiertos, sin publicar credenciales.
9. **Pruebas ejecutadas:** comandos, resultados, evidencias y entorno de ejecución.
10. **Copias de seguridad:** configuración, frecuencia, retención disponible, alertas y limitaciones.
11. **Recuperación:** procedimiento documentado y declaración explícita de que no se probó la restauración.
12. **Mantenimiento:** ubicación de la documentación y procedimientos disponibles.
13. **Defectos y riesgos:** clasificación, impacto y estado.

14. **Pendientes:** decisiones o tareas bloqueadas y motivos.
15. **Transición futura a producción:** tareas necesarias para sustituir los datos sintéticos por datos reales de forma controlada.
16. **Estado de aceptación:** aprobado, aprobado con limitaciones documentadas o bloqueado.

Para cada prueba, utiliza uno de estos estados:

- **APROBADA:** se ejecutó y existe evidencia del resultado esperado.
- **FALLIDA:** se ejecutó y no cumplió el resultado esperado.
- **BLOQUEADA:** no pudo ejecutarse por una dependencia o limitación identificada.
- **NO EJECUTADA:** no se realizó.

No afirmes que una tarea, prueba, copia de seguridad, restauración, alerta o configuración está completada si no existe evidencia suficiente.

No inventes URLs, resultados, métricas, capturas, costos, configuraciones ni verificaciones.

---

19. Orden de ejecución

Sigue este orden de trabajo:

1. Inspeccionar repositorios, documentación y estado de Git.
2. Revisar los criterios pendientes de la Etapa 5.
3. Evaluar las plataformas administradas gratuitas.
4. Confirmar la compatibilidad del alojamiento con la arquitectura y la seguridad.
5. Elaborar el plan técnico concreto del despliegue.
6. Verificar que el plan no introduce costos ni modifica decisiones aprobadas.
7. Preparar y configurar los entornos.
8. Desplegar MySQL y aplicar los scripts controlados.

9. Crear el administrador inicial y los datos sintéticos.
10. Desplegar el backend.
11. Desplegar el frontend.
12. Ejecutar las pruebas de seguridad y funcionamiento.
13. Configurar las copias de seguridad viables.
14. Documentar mantenimiento y restauración.
15. Corregir defectos encontrados y repetir las pruebas afectadas.
16. Revisar los repositorios, cambios de Git y secretos.
17. Elaborar el informe final con evidencias.
18. Confirmar que el sistema permanece en modo de demostración académica.

Si una fase falla, corrige el problema y repite las verificaciones correspondientes. Si no es posible resolverlo de forma segura, detén la actividad afectada y continúa únicamente con las tareas independientes.

20. Instrucción final

Ejecuta la Etapa 6 de forma incremental, verificable y sin sobreingeniería.

La prioridad es obtener una demostración funcional, segura y reproducible dentro del presupuesto aprobado, no aparentar una infraestructura empresarial que el proyecto no necesita ni que el proveedor gratuito puede garantizar.

Respeta todas las decisiones aprobadas de las etapas anteriores. No alteres las reglas de negocio, los contratos API ni la arquitectura principal. No introduzcas costos recurrentes. No publiques secretos. No elimines datos de manera improvisada. No cargues datos comerciales reales ni habilites operaciones reales.

La etapa termina cuando el despliegue alcanzable ha sido verificado, las limitaciones están documentadas y el informe final permite determinar con evidencia qué está funcionando, qué está pendiente y qué sería necesario antes de una futura puesta en producción comercial.

PROMPT #7

ETAPA 7 — HARDENING, AUTOMATIZACIÓN, OBSERVABILIDAD Y MEJORA CONTINUA

1. Rol y objetivo

Actúa como un equipo multidisciplinario de ingeniería de software integrado por:

- Software Architect.
- Senior Backend Engineer especializado en Java y Spring Boot.
- Senior Frontend Engineer especializado en React y Material UI.
- QA Automation Engineer.
- Application Security Engineer.
- DevOps Engineer.
- Especialista en observabilidad, monitoreo y análisis de métricas.

Tu objetivo es realizar la séptima y última etapa de un sistema web de inventario para una ferretería de una sola sede, complementando el trabajo de las seis etapas anteriores.

Esta etapa debe cerrar las brechas de documentación, internacionalización, calidad del código, pruebas avanzadas, automatización CI/CD, despliegue, monitoreo, gestión de incidencias y seguimiento técnico del equipo.

No debes reconstruir el sistema ni repetir las etapas anteriores. Primero inspecciona lo que ya existe; después identifica las brechas y trabaja exclusivamente sobre ellas.

La implementación debe ser incremental, verificable, segura y proporcional a las necesidades del proyecto.

2. Contexto técnico obligatorio

Conserva las siguientes decisiones:

Backend

- Java 21.
- Spring Boot 3.
- Spring MVC.
- Spring Data JPA y Hibernate.
- Spring Security.
- Maven.
- API REST con intercambio JSON.
- OpenAPI/Swagger.
- Arquitectura tradicional por capas: Controller → Service → Repository.

Frontend

- React.
- Material UI.
- Formik y Yup cuando sean apropiados.
- Playwright para pruebas E2E.
- Selenium para una suite complementaria cuando aporte cobertura útil.

Persistencia y autenticación

- MySQL.
- Autenticación basada en sesiones, no JWT.
- Autorización mediante los roles ya aprobados.
- Los endpoints protegidos deben verificar los permisos en el backend.
- No cambies el modelo de datos, las reglas de negocio ni los contratos de API sin autorización explícita.

Repositorios y presupuesto

- Backend y frontend permanecen en sus repositorios actuales.
- No crees un tercer repositorio sin necesidad justificada y aprobación.
- Presupuesto recurrente máximo: **COP $0**.

- Prioriza herramientas gratuitas y mecanismos nativos de GitHub cuando sean viables.
- No compres dominios, planes, servicios ni licencias.
- No actives servicios de pago ni consumas recursos con posibles cargos sin autorización.

3. Reglas de ejecución

Antes de modificar archivos, cumple este procedimiento.

3.1 Inspección inicial

Inspecciona:

- Los repositorios completos de backend y frontend.
- La estructura de paquetes y módulos.
- Los documentos de requisitos y arquitectura aprobados.
- Las pruebas existentes y sus resultados.
- Los scripts SQL y las instrucciones de instalación.
- Los archivos de configuración y variables de entorno.
- Los workflows de GitHub Actions, si existen.
- La infraestructura actual de demostración.
- Los mecanismos de logging, monitoreo y manejo de errores.
- Los módulos administrativos existentes.
- Las incidencias o tareas registradas en GitHub, si están disponibles.

No supongas que una funcionalidad está ausente por no encontrarla inmediatamente. Busca evidencia suficiente.

3.2 Matriz de brechas

Antes de implementar, genera una matriz con estas columnas:

Campo	Contenido
ID	Identificador único

Área	Documentación, pruebas, CI/CD, monitoreo, etc.
Requisito	Resultado esperado
Estado	Cumplido, parcial, pendiente o bloqueado
Evidencia	Archivo, prueba, workflow o resultado verificable
Brecha	Trabajo pendiente concreto
Prioridad	Crítica, alta, media o baja
Dependencias	Requisitos previos
Criterio de aceptación	Condición objetiva para cerrar la brecha

La matriz debe distinguir entre una capacidad realmente implementada y una que solo está documentada o parcialmente configurada.

Conserva los documentos existentes y actualízalos de forma coherente. No dupliques documentación innecesariamente.

3.3 Plan de ejecución

Después de la inspección:

1. Presenta las brechas encontradas.
2. Agrúpalas por dependencias y riesgo.
3. Identifica los cambios necesarios en cada repositorio.
4. Define las pruebas y evidencias para cada bloque.
5. Implementa por bloques pequeños.
6. Ejecuta las verificaciones después de cada bloque.
7. Corrige las regresiones introducidas.
8. Actualiza la matriz de brechas con resultados reales.

No detengas el trabajo por decisiones técnicas menores que puedas resolver de forma segura y reversible. Sí debes detenerte ante cambios de alcance, riesgos de pérdida de datos, costes, modificaciones de reglas de negocio o decisiones arquitectónicas no autorizadas.

No afirmes haber ejecutado comandos, pruebas, despliegues o verificaciones que no hayas realizado.

---

4. Área A — Documentación y estándares de código

4.1 Metadatos uniformes

Define una convención documental para los artefactos donde tenga sentido incorporar metadatos.

Debe contemplar:

- Nombre.
- Descripción.
- Fecha de creación.
- Fecha de actualización.
- Autor o responsable.
- Versión.

Aplica encabezados de forma selectiva a archivos fuente, documentos y otros artefactos donde aporten valor.

No fuerces comentarios dentro de JSON, YAML, archivos generados, binarios o formatos que no admitan comentarios válidos. En esos casos, utiliza documentación externa, metadatos del formato o información del repositorio.

No inventes autores ni fechas históricas. Si no se pueden verificar, documenta esa limitación o utiliza un mecanismo de seguimiento que no presente datos ficticios como reales.

Evita encabezados repetitivos que añadan ruido y no aporten información. No sobrescribas licencias ni avisos legales existentes.

4.2 JavaDoc y JSDoc

Evalúa la documentación del backend y frontend.

En Java, documenta las clases, interfaces y métodos públicos relevantes, especialmente aquellos que forman parte de contratos, servicios de aplicación, integraciones o lógica no evidente.

Cuando corresponda, documenta:

- Propósito.
- Parámetros.
- Valores de retorno.
- Excepciones.
- Efectos secundarios.
- Restricciones o invariantes.
- Consideraciones de concurrencia.
- Ejemplos de uso cuando ayuden a comprender el contrato.

En JavaScript y TypeScript, si este último ya se utiliza, documenta funciones, hooks, componentes y módulos públicos relevantes mediante JSDoc o el mecanismo apropiado.

No agregues comentarios que simplemente repitan el código. No inventes comportamientos ni documentes como implementadas funciones inexistentes.

4.3 Validación automática

Incorpora comprobaciones automáticas razonables para detectar:

- Documentación pública obligatoria ausente.
- Encabezados incorrectos cuando sean exigibles.
- Errores en la generación de documentación.
- Inconsistencias entre contratos documentados e implementación.
- Comentarios o metadatos evidentemente desactualizados cuando puedan detectarse de forma fiable.

No conviertas cada advertencia documental en un bloqueo absoluto. Define qué comprobaciones son obligatorias y cuáles informativas.

4.4 OpenAPI y documentación de instalación

Verifica que OpenAPI refleje la API real:

- Rutas y métodos HTTP.

- DTO y esquemas JSON.
- Validaciones.
- Códigos de respuesta.
- Errores.
- Autenticación por sesión.
- Restricciones de autorización.
- Paginación y filtros cuando existan.

Mantén actualizados los README de ambos repositorios, incluyendo instalación, configuración, ejecución, pruebas, compilación, variables necesarias y resolución de problemas habituales.

Criterio de aceptación: documentación coherente con el código y verificaciones automatizadas ejecutadas, con las limitaciones identificadas.

---

5. Área B — Internacionalización, diseño y accesibilidad

5.1 Internacionalización

Implementa o completa la internacionalización del sistema.

Idiomas requeridos:

- Español como idioma predeterminado.
- Inglés como idioma alternativo.

En el frontend:

- Centraliza los textos traducibles.
- Evita duplicar textos visibles en múltiples componentes.
- Traduce navegación, formularios, validaciones, tablas, estados, mensajes de éxito y error, confirmaciones y componentes administrativos.
- Permite cambiar el idioma mediante un mecanismo accesible.

- Conserva el idioma seleccionado según una estrategia compatible con la aplicación actual.

En el backend:

- Centraliza los mensajes traducibles de validación y errores cuando corresponda.
- Respeta el contrato JSON de la API.
- Evita depender de textos localizados para la lógica de negocio.
- Mantén estables los códigos de error y los identificadores internos.

No traduzcas identificadores persistidos, valores internos, nombres de columnas ni contratos técnicos de manera que rompa compatibilidad.

5.2 Formatos regionales

Separa la presentación regional de la lógica de negocio.

Verifica:

- Fechas y horas.
- Números decimales.
- Cantidades fraccionarias.
- Moneda COP.
- Símbolos y separadores.
- Zonas horarias.

La localización no debe modificar cálculos de inventario, precios, costos ni persistencia.

5.3 Sistema visual

Conserva Material UI y establece, si todavía no existe, un sistema centralizado de diseño que incluya:

- Colores.
- Tipografía.
- Espaciado.

- Bordes.
- Elevaciones.
- Estados de interacción.
- Componentes reutilizables.
- Tema claro y, si resulta viable sin ampliar innecesariamente el alcance, tema oscuro.

Evita estilos duplicados y valores visuales arbitrarios dispersos.

5.4 Accesibilidad y diseño responsive

Verifica:

- Navegación mediante teclado.
- Etiquetas accesibles.
- Contraste.
- Estados de foco.
- Mensajes de error comprensibles.
- Tablas y formularios utilizables.
- Diálogos accesibles.
- Diseño en escritorio, tableta y móvil.
- Uso correcto de encabezados y regiones semánticas.

No afirmes conformidad con una norma de accesibilidad si no se ha evaluado con evidencia suficiente.

Criterio de aceptación: ambos idiomas son utilizables, no hay claves de traducción faltantes en los recorridos críticos y las comprobaciones visuales y de accesibilidad están documentadas.

---

6. Área C — Pruebas avanzadas y aseguramiento de calidad

6.1 Inventario de pruebas

Construye un catálogo de casos con:

- ID.
- Requisito relacionado.
- Objetivo.
- Precondiciones.
- Datos de prueba.
- Pasos.
- Resultado esperado.
- Prioridad.
- Tipo de prueba.
- Resultado real.
- Evidencia.
- Estado: aprobado, fallido, bloqueado o no ejecutado.

Genera una matriz de trazabilidad entre requisitos, casos, automatización y defectos.

No inventes resultados ni marques como aprobados los casos no ejecutados.

6.2 Pruebas unitarias y de integración

Utiliza las herramientas existentes cuando sean adecuadas:

- JUnit 5.
- Mockito.
- Spring Boot Test.
- MockMvc.
- Pruebas de integración con MySQL aislado.
- Herramientas de prueba del frontend compatibles con el stack existente.

Prioriza riesgos de integridad del inventario, cálculo de costos, cantidades, seguridad, permisos, reversos y consistencia transaccional.

Distingue pruebas unitarias con mocks de pruebas que verifican comportamiento real con Spring y MySQL.

6.3 Pruebas de límites y casos negativos

Incluye casos como:

- Valores nulos o vacíos.
- Longitudes mínimas y máximas.
- Cantidades cero, negativas, fraccionarias y excesivas.
- Precios y costos inválidos.
- Identificadores inexistentes.
- Duplicados.
- Usuarios sin permisos.
- Sesiones expiradas.
- Transiciones de estado inválidas.
- Existencias insuficientes.
- Cancelaciones y devoluciones inválidas.
- Datos con caracteres especiales.
- Solicitudes concurrentes.
- Errores de persistencia y transacciones.

Los casos deben derivarse de las reglas ya aprobadas; no introduzcas nuevas reglas de negocio para completar la matriz.

6.4 Pruebas de mutación

Incorpora pruebas de mutación en backend mediante PIT u otra herramienta compatible.

Evalúa StrykerJS para el frontend únicamente si aporta valor y puede integrarse sin coste recurrente.

Debes:

1. Configurar el alcance de mutación.

2. Ejecutar una línea base.
3. Identificar mutaciones sobrevivientes relevantes.
4. Añadir o mejorar pruebas donde la mutación revele una brecha real.
5. Documentar exclusiones justificadas.
6. Definir umbrales por riesgo y madurez del módulo.

No persigas un porcentaje arbitrario ni modifiques código de producción únicamente para mejorar una puntuación.

Una mutación sobreviviente no siempre representa un defecto: clasifica los resultados antes de decidir.

6.5 Pruebas de carga, estrés y concurrencia

Utiliza k6 o una alternativa gratuita apropiada.

Diseña pruebas aisladas y reproducibles que evalúen:

- Operaciones frecuentes.
- Carga concurrente.
- Latencia.
- Tasa de errores.
- Consumo de recursos disponible.
- Comportamiento ante saturación.
- Operaciones de inventario simultáneas.
- Prevención de stock negativo.
- Consistencia transaccional.

No inventes objetivos de rendimiento. Establece una línea base medida y documenta los límites del entorno.

El estrés debe ejecutarse únicamente contra entornos autorizados y aislados. No provoques saturación de la demostración pública ni de sistemas externos.

6.6 Playwright y Selenium

Conserva Playwright como herramienta principal de E2E.

Evalúa Selenium para una suite complementaria pequeña y justificada, por ejemplo, una comprobación de compatibilidad del navegador si añade cobertura real.

No dupliques todos los escenarios sin beneficio claro ni mantengas dos frameworks completos solo por cumplir una lista tecnológica.

Automatiza los flujos críticos de autenticación, catálogo, inventario, compras, ventas, devoluciones, permisos y reportes, además de los casos negativos relevantes.

6.7 Seguridad

Evalúa:

- Autenticación por sesión.
- Autorización del backend.
- CSRF.
- CORS.
- Gestión de cookies.
- Validación de entradas.
- Inyección SQL.
- XSS.
- Control de acceso a recursos.
- Manejo de secretos.
- Dependencias vulnerables.
- Exposición de logs y errores.
- Protección de endpoints administrativos.

Realiza pruebas dinámicas solo en entornos propios o autorizados.

6.8 Regresión y estabilidad

Investiga las pruebas inestables. No las deshabilites ni uses reintentos ilimitados para ocultar fallos.

Tras corregir un defecto, añade una prueba de regresión cuando corresponda.

Criterio de aceptación: matriz de pruebas completa, resultados verificables, mutación evaluada, límites y concurrencia cubiertos, suite E2E operativa y riesgos residuales documentados.

---

7. Área D — CI/CD y automatización de cambios

7.1 GitHub Actions

Evalúa la viabilidad de GitHub Actions con los límites gratuitos reales de la cuenta y los repositorios.

No supongas que todos los minutos, artefactos, runners, entornos o funciones son ilimitados. Comprueba las restricciones disponibles y documenta cualquier limitación.

Crea workflows mantenibles para backend y frontend, reutilizando componentes solo cuando simplifique el mantenimiento.

7.2 Pipeline de validación

Cuando se detecten cambios en las ramas o eventos configurados, ejecuta las verificaciones pertinentes:

1. Validación de estructura y configuración.
2. Compilación.
3. Pruebas unitarias.
4. Pruebas de integración cuando el entorno lo permita.
5. Lint y validaciones del frontend.
6. Construcción del frontend.
7. Generación y validación de documentación.
8. Análisis de dependencias y seguridad.
9. Pruebas adicionales según la criticidad del cambio.
10. Publicación de resultados y artefactos identificados por versión o commit.

Evita ejecutar pruebas destructivas contra bases de datos reales.

Las pruebas de mutación y estrés pueden ejecutarse en workflows específicos o controles de aceptación previos a una versión, siempre que formen parte de la verificación documentada. No deben desaparecer del alcance solo porque sean costosas para cada commit.

7.3 Bloqueo ante fallos

Si una validación obligatoria falla:

- El pipeline debe quedar fallido.
- No debe publicarse una nueva versión de demostración.
- Se debe conservar evidencia del error.
- Se debe crear o actualizar una incidencia cuando corresponda.
- Se debe notificar mediante mecanismos gratuitos disponibles.

No dupliques issues para el mismo fallo repetido. Utiliza una clave o mecanismo de deduplicación basado en workflow, rama, commit y tipo de fallo.

Las notificaciones deben evitar exponer secretos, datos de usuarios o información sensible.

7.4 Detección de cambios e incidencias

Automatiza el seguimiento de cambios utilizando GitHub cuando sea viable.

No confundas un cambio detectado con un cambio correctamente probado. No marques tareas como resueltas solo porque un commit existe.

Cada incidencia automática debe incluir, cuando esté disponible:

- Tipo de fallo.
- Workflow y ejecución.
- Rama y commit.
- Fecha.
- Evidencia relevante.
- Pasos para reproducir o diagnóstico.

- Estado de resolución.

Evita loops en los que el bot cree nuevos issues por sus propias modificaciones.

Criterio de aceptación: cambios detectados, validaciones ejecutadas, fallos bloqueantes correctamente tratados y evidencias accesibles sin costes recurrentes.

---

8. Área E — Despliegue automático de demostración

8.1 Regla de despliegue

La demostración académica puede desplegarse automáticamente cuando se cumplan todas las verificaciones obligatorias y la plataforma lo permita.

La producción comercial queda fuera de la autorización de esta etapa y requerirá aprobación independiente.

No cargues datos comerciales reales ni conviertas el sistema en una operación comercial real durante esta fase.

8.2 Configuración segura

Verifica:

- Variables de entorno.
- Secretos fuera del repositorio.
- Configuración de sesiones.
- HTTPS cuando lo soporte el proveedor.
- Cookies seguras según el entorno.
- Protección CSRF.
- CORS restringido.
- Acceso administrativo.
- Configuración de base de datos.
- Límites y condiciones del proveedor.

- Manejo de errores.
- Logs.
- Identificación de versión desplegada.

No publiques credenciales, tokens ni archivos de configuración sensibles.

No uses datos sintéticos débiles como credenciales reales ni publiques contraseñas administrativas en documentación pública.

8.3 MySQL y scripts SQL

Conserva MySQL y el mecanismo de scripts SQL versionados y revisados manualmente.

- No uses ddl-auto=update.
- No ejecutes scripts destructivos automáticamente al arrancar.
- No borres datos para resolver problemas de despliegue.
- No ejecutes migraciones sobre un entorno sin confirmar su destino y compatibilidad.
- Documenta el orden de ejecución y los scripts ya aplicados.
- No incluyas credenciales reales en Git.

Mantén los datos sintéticos de demostración. Documenta el futuro proceso de transición a una operación comercial, sin ejecutarlo.

8.4 Verificaciones posteriores al despliegue

Después de publicar la demostración:

- Comprueba disponibilidad del frontend.
- Comprueba disponibilidad del backend.
- Verifica la conexión entre frontend y API.
- Ejecuta smoke tests.
- Comprueba autenticación y autorización.
- Verifica los recorridos esenciales.
- Confirma la versión desplegada.

- Conserva el resultado del despliegue.

Si el proveedor no permite ejecutar una verificación, marca el resultado como bloqueado y explica la limitación.

Si no existe un rollback seguro, no improvises uno ni continúes ante una situación de riesgo.

Criterio de aceptación: demostración accesible y autenticada, pipeline verificable, smoke tests documentados y ningún despliegue comercial no autorizado.

---

9. Área F — Dashboard integrado de administración y monitoreo

9.1 Objetivo

Implementa un dashboard integrado en la aplicación existente, accesible exclusivamente para usuarios con el rol ADMINISTRADOR.

No crees microservicios ni una aplicación independiente para el dashboard.

Reutiliza la autenticación por sesión y las convenciones visuales de Material UI.

La protección debe implementarse tanto en el frontend como en el backend. Ocultar un menú no constituye autorización.

9.2 Métricas técnicas

Muestra, cuando existan fuentes de datos fiables:

- Disponibilidad.
- Estado de salud del backend.
- Latencia.
- Tasas de errores HTTP.
- Errores recientes.
- Incidencias abiertas.
- Estado de ejecuciones CI/CD.
- Resultado del último despliegue.

- Commit o versión desplegada.
- Resultados resumidos de pruebas relevantes.
- Resultados de pruebas de rendimiento.

Distingue las métricas medidas en tiempo real de los resultados históricos o periódicos.

No inventes datos cuando una fuente no exista. Muestra claramente la falta de información o la fecha de la última actualización.

No expongas públicamente endpoints internos de diagnóstico ni datos operativos sensibles.

9.3 Fuentes de datos

Evalúa fuentes reales disponibles:

- Actuator u otros mecanismos existentes de salud.
- Logs estructurados.
- Instrumentación de errores y latencia.
- GitHub Issues.
- GitHub Pull Requests.
- GitHub Actions.
- Artefactos de pruebas.
- Información de versión y despliegue.

Prefiere integraciones gratuitas y simples. Si se necesita acceder a la API de GitHub, gestiona permisos mínimos y secretos protegidos.

No guardes tokens de GitHub en el frontend. No incluyas secretos en respuestas API, logs ni navegador.

Documenta qué fuentes son automáticas, cuáles son periódicas y cuáles requieren actualización manual.

9.4 Seguridad del dashboard

- Verifica permisos en cada endpoint administrativo.

- Rechaza usuarios no autenticados.
- Rechaza usuarios autenticados sin el rol necesario.
- Limita la información expuesta.
- Protege secretos y datos personales.
- Evita que un error en la integración externa exponga credenciales.
- Registra los errores de integración sin revelar datos sensibles.

Criterio de aceptación: dashboard funcional, fuentes identificadas, datos verificables y acceso administrativo aplicado en el servidor.

---

10. Área G — Gestión de tareas, incidencias y métricas del equipo

10.1 Seguimiento de trabajo

Integra, cuando sea viable sin coste recurrente, información de GitHub Issues, Pull Requests y Actions.

El dashboard debe permitir consultar:

- Tareas por estado.
- Tareas por responsable.
- Hitos y fechas límite cuando existan.
- Incidencias por severidad.
- Antigüedad de incidencias abiertas.
- Tiempo de resolución.
- Pull Requests pendientes.
- Estado de revisión.
- Fallos recurrentes de CI.
- Tendencias de defectos.
- Bloqueos identificados.
- Relación entre tareas y entregas cuando pueda verificarse.

No dupliques manualmente información que ya esté correctamente gestionada en GitHub. Si se necesita una copia local, documenta sincronización, frecuencia y comportamiento ante fallos.

10.2 Métricas de gestión y métricas técnicas

Distingue explícitamente ambos grupos.

Métricas de gestión

- Tareas abiertas, cerradas y bloqueadas.
- Cumplimiento de hitos.
- Antigüedad de tareas.
- Incidencias pendientes.
- Tiempo de resolución.
- Trabajo pendiente de revisión.

Métricas técnicas

- Compilaciones correctas y fallidas.
- Resultados de pruebas.
- Defectos encontrados.
- Errores de ejecución.
- Latencia y disponibilidad.
- Incidencias de seguridad.
- Regresiones.
- Resultados de carga y estrés.

Explica el significado, origen, frecuencia de actualización y limitaciones de cada métrica.

10.3 Evaluación responsable

No construyas puntuaciones automáticas de productividad individual.

No uses cantidad de commits, líneas de código, tareas cerradas o Pull Requests como medidas aisladas de rendimiento profesional.

Las métricas deben servir para identificar problemas del proceso, riesgos, bloqueos y tendencias, no para establecer rankings automáticos.

Si se muestran métricas por responsable, contextualízalas y evita inferencias automáticas sobre desempeño individual.

No inventes responsables, asignaciones, fechas ni resultados.

Criterio de aceptación: el dashboard refleja información trazable, distingue métricas técnicas y de gestión y no genera calificaciones individuales automáticas.

---

11. Presupuesto, privacidad y dependencias

Antes de añadir una herramienta o servicio:

1. Evalúa si la necesidad ya está cubierta.
2. Busca una alternativa nativa o gratuita.
3. Comprueba sus límites actuales.
4. Evalúa mantenimiento, seguridad y complejidad.
5. Documenta la decisión.

No introduzcas dependencias solo por popularidad.

No actives pruebas de pago, almacenamiento facturable, runners de pago, dominios ni planes comerciales.

No guardes información personal innecesaria en métricas o logs.

Define una política razonable de retención técnica sin imponer periodos de conservación que no hayan sido aprobados. Documenta límites de almacenamiento y limpieza.

Si un requisito no puede cumplirse con presupuesto cero, no lo simules ni lo presentes como implementado. Documenta la limitación, las alternativas gratuitas evaluadas y el impacto.

---

12. Protección de Git y cambios existentes

Antes de editar:

- Comprueba rama y estado del repositorio.
- Inspecciona cambios pendientes y archivos no rastreados.
- Identifica modificaciones preexistentes.
- No sobrescribas trabajo ajeno.
- No elimines archivos ni datos sin justificación y autorización.

Después de cada bloque:

- Revisa el diff.
- Ejecuta las verificaciones relevantes.
- Comprueba que no se hayan introducido secretos.
- Registra commits descriptivos cuando el bloque esté verificado.
- No hagas push automático sin autorización.

No reformatees ni refactorices archivos ajenos al alcance únicamente para uniformar estilo.

No realices migraciones destructivas ni cambios irreversibles sin autorización explícita.

---

13. Criterios globales de aceptación

La Etapa 7 debe evaluarse con una matriz final que incluya, al menos:

ID	Criterio
E7-01	Inspección y matriz de brechas completadas con evidencia
E7-02	Convenciones de metadatos documentadas y comprobaciones implementadas cuando correspondan
E7-03	JavaDoc/JSDoc completados en el alcance definido

E7-04	OpenAPI coherente con la API real
E7-05	Internacionalización española e inglesa verificada
E7-06	Formatos regionales separados de la lógica de negocio
E7-07	Diseño responsive y accesibilidad evaluados
E7-08	Catálogo de pruebas y trazabilidad actualizados
E7-09	Pruebas de límites, concurrencia y regresión ejecutadas
E7-10	Pruebas de mutación configuradas y resultados evaluados
E7-11	Pruebas de carga y estrés documentadas con resultados medidos
E7-12	Playwright operativo y Selenium evaluado e integrado si aporta valor
E7-13	Evaluación de seguridad completada con evidencia
E7-14	Workflows de CI configurados y verificados
E7-15	Fallos obligatorios bloquean la publicación
E7-16	Gestión de incidencias y notificaciones implementada cuando sea viable
E7-17	Demostración desplegada automáticamente solo después de las verificaciones requeridas

E7-18	Smoke tests posteriores al despliegue documentados
E7-19	Dashboard administrativo protegido en frontend y backend
E7-20	Métricas técnicas proceden de fuentes verificables
E7-21	Métricas de gestión proceden de fuentes identificadas
E7-22	No existen puntuaciones automáticas individuales basadas en métricas aisladas
E7-23	Restricciones de presupuesto y servicios gratuitos verificadas
E7-24	Documentación de operación, limitaciones y mantenimiento actualizada
E7-25	No se modificaron reglas de negocio, MySQL, autenticación por sesión ni arquitectura sin autorización

Cada criterio debe registrar estado, evidencia, fecha y observaciones.

Los estados permitidos son:

- **Aprobado:** existe evidencia suficiente de cumplimiento.
- **Fallido:** la verificación se ejecutó y no cumplió.
- **Bloqueado:** no se pudo verificar por una dependencia o limitación identificada.
- **No ejecutado:** todavía no se realizó la comprobación.
- **No aplicable:** únicamente con justificación explícita.

No conviertas un estado bloqueado o no ejecutado en aprobado.

Los requisitos obligatorios críticos no pueden cerrarse únicamente con documentación si requieren una implementación funcional.

---

14. Informe final obligatorio

Al terminar, genera un informe con:

1. Resumen ejecutivo.
2. Estado general de la etapa.
3. Brechas detectadas y su resolución.
4. Archivos y módulos modificados.
5. Dependencias agregadas y justificación.
6. Pruebas ejecutadas y resultados reales.
7. Informe de mutación.
8. Resultados de carga, estrés y concurrencia.
9. Hallazgos de seguridad.
10. Estado de CI/CD.
11. Estado del despliegue de demostración.
12. URL real de la demostración, únicamente si se verificó.
13. Estado del dashboard y fuentes de datos.
14. Estado de issues y notificaciones.
15. Evaluación de costes recurrentes.
16. Riesgos residuales y limitaciones.
17. Instrucciones de mantenimiento y resolución de problemas.
18. Matriz final de aceptación.
19. Recomendaciones pendientes, priorizadas y sin implementar automáticamente.

Incluye comandos ejecutados y resultados relevantes cuando sea útil. Distingue resultados locales de resultados obtenidos en CI o en el entorno desplegado.

No inventes porcentajes, métricas, URLs, resultados, costes ni pruebas.

15. Condición de cierre

La etapa se considera completada cuando todas las verificaciones obligatorias se hayan aprobado o cuando las únicas limitaciones restantes estén explícitamente identificadas y aceptadas como bloqueos externos no críticos.

Si existen fallos críticos o altos que comprometan la seguridad, la integridad del inventario, la autenticación o los flujos esenciales, informa que la etapa no está aprobada.

No ocultes defectos ni rebajes su severidad para declarar éxito.

No inicies una octava etapa. Finaliza con el informe de la Etapa 7 y deja cualquier trabajo futuro como recomendación separada.

Instrucción final: comienza inspeccionando los repositorios y documentos existentes. No modifiques archivos hasta terminar la evaluación inicial y presentar la matriz de brechas y el plan de implementación. Después, avanza por bloques verificables, preservando las decisiones aprobadas y sin ampliar el alcance por iniciativa propia.